VarioHyve Language Specification

Ciprian Teodorov

12-10-2026

Contents

I. Motivation 1

1. Behavioral Federation 2

II. Specification 6

2. Program Structure 7

3. Literals 11

4. Objects and Classes 15

5. Slots and Accessors 24

6. Message Sends 27

7. Apply and Callable Values 32

8. Syntactic Sugar 33

9. Name Resolution 35

10. Recursion 40

11. Descriptors 42

12. Foreign Semantic Values and the Alien/Expat Boundary 45

III. Possible Seeds and Worlds 57

13. Possible Seeds and Worlds 58

14. The Host Seed 59

15. JavaHostKernel world 60

16. One Possible Native Seed 62

17. A World with Native Seed but Host Scalars and Strings 64

IV. Implementation 65

18. Runtime Object Representation 66

19. Java Interpreter 71

20. Tooling Architecture 81

V. Future 91

21. Future Directions 92

VI. Related Works 99

22. Connections and Comparisons 100

VII. Appendix 103

A. Formal Rules 104

Part I.

Motivation

1. Behavioral Federation

VarioHyve investigates how programs can coordinate systems that keep their own meaning, execution mechanisms and authority. A model repository, a simulator and a running controller can cooperate without becoming objects of one universal model or processes of one execution engine. The difficult part is making the agreements between them explicit enough to understand, change and examine.

We call this **behavioral federation**. The experiment combines an object language, a native/foreign boundary and an explicit execution model. Its working hypothesis is that a small, coherent set of mechanisms can make coordination behavior easier to reuse and its assumptions easier to inspect. An executable bootstrap now allows that hypothesis to be investigated through programs and tests. Whether the combination improves real engineering workflows remains an empirical question.

1.1. Cooperation Without Complete Control

A system-of-systems brings together participants that retain substantial operational and managerial independence. Their owners, release cycles and objectives need not coincide. Engineering environments encounter a related problem when requirements, designs, simulations and operational records live in different tools. The engineered product need not itself be a system-of-systems for its engineering models to require federation.

Consider a pumping station. A requirements model states a minimum flow, a hydraulic simulator predicts performance, and maintenance records describe the installed equipment. Each may refer to the same pump while expressing different facts about it. Assessing a proposed replacement requires a correspondence between those references, compatible units, an agreed revision of the design and a policy for interpreting the analysis. Authorizing the replacement requires additional authority. A successful assessment alone does not supply it.

API connectivity solves only part of this problem. A simulator may finish after the relevant design has changed. A service may fail after accepting a request, leaving its effects uncertain. A callback may arrive while the coordinator is waiting for a result. The engineer needs to know which observations support a decision, which assumptions remain in force and what a failed interaction means. Replacing the simulator with a recorded trace should expose the changed assumptions while preserving the intended coordination protocol.

These needs become language and tooling questions when the workflow must be maintained across many projects. Builders of modeling environments need reusable behavior over heterogeneous models, traceable evidence and controlled access to tools. Moving every model into a new metamodel can be expensive or inappropriate; reimplementing every participant's behavior can lose exactly the semantics that made it useful. Existing federation and model-management tools already address substantial parts of this problem. VarioHyve asks what a compact coordination language can contribute alongside them,

and what it costs compared with their own workflows or direct host-language code.

1.2. Agreements Expressed Through Objects

The approach starts with supplied capabilities: objects whose behavior grants access to particular services or resources. A coordinator receives the capabilities it needs and exchanges objects by sending messages. Its inputs can represent a narrow model query, an analysis operation or permission to record evidence. The supplying system determines the available authority; the coordinator's source makes its use visible.

This is also a commitment to semantic sovereignty. Foreign participants retain their own execution and interpretation of operations. The boundary states how messages, values, identity, failures and callbacks cross between them. An operation may return an ordinary value or promise an explicit outcome protocol, just as a native method may. A shared message syntax does not establish agreement about units, revisions or successful completion: those remain part of the supplied operation's contract.

Here, "contract" means an agreement about behavior and authority, not an implemented contract language or automatic proof system. The boundary makes such agreements a subject of design. It also makes their limits visible. For example, foreign code running in the same host process can terminate that process; a capability interface alone does not provide process containment.

1.3. A Language for Explicit Dependencies

The following module illustrates the shape of a coordinator:

```
class Assessment(modelSource, checker, evidenceSink) {
    models = modelSource;
    analysis = checker;
    evidence = evidenceSink;

    assess(item) {
        var snapshot = models.snapshot(item);
        evidence.record(snapshot, analysis.check(snapshot))
    }
}
```

The host supplies three existing objects when it constructs `Assessment`; it later requests an assessment by sending `assess`. The module retains its inputs in slots because constructor parameters belong to initialization, rather than becoming instance-method bindings. Nothing in this code requires the three services to be implemented in VarioHyve. Their selectors are application agreements, and the example deliberately leaves their failure and evidence policies to those agreements.

A normal module begins over an empty top activation. It receives external capabilities through explicit constructor inputs and, subsequently, message arguments and the behavior of objects already available to it. There is no ambient application namespace. Supplied objects keep their own authority and captured scopes, so a short parameter list is not by itself an isolation proof. The program-structure chapter defines this entry contract precisely.

Within the language, objects and message sends also organize access and reuse. Slots are accessed through messages; lexical scopes are activation objects and ordinary objects. Classes are first-class values. Composition uses reified contributions and application occurrences, retaining the declaration origins that determine scope and invocation. A reusable analysis stage can therefore keep access to its defining provider even when applied in a receiving module with similarly named bindings. The rules aim to make that behavior predictable without merging independently defined models into a deep class family.

For language design, the question is whether these mechanisms work together without hidden exceptions at each point of reuse. Definition scope, receiver identity and contribution selection must remain intelligible when code moves between modules or across a foreign boundary. The specification fixes these rules independently of the Java representation, giving alternative implementations a contract to preserve. That precision also exposes costs in notation, implementation and debugging that the experiments must assess.

Foreign values appear inside the language as **Aliens**; native behavior is exported as **Expats**. Sending a message to an Alien uses the ordinary source syntax. The adapter mediates the operation, and foreign code may call an Expat while the native caller waits. Language-visible VM services use this same boundary, with separately granted privileged capabilities. Explicit mirrors can offer information beyond ordinary behavior. The boundary chapter defines these interactions, including identity, failure and the default prohibition on non-local returns crossing a foreign call.

1.4. Making Execution Available for Inquiry

Coordination becomes easier to investigate when execution itself has a precise description. VarioHyve's machine exposes configurations and transitions for evaluation, invocation, storage and control. This supports a correspondence between language rules, executable behavior and observations made by tools. It does not by itself establish that correspondence as a proved theorem.

The current sequential execution agreement uses a conceptual **control token** for ordinary native execution. A native-to-foreign call transfers control to the foreign side; an admitted callback can return it to native execution before the original call completes. Multiple callback workers must negotiate admission, and the current implementation serializes their native execution. The token need not be a runtime object. Its purpose is to describe an agreement that an implementation must realize.

That agreement leaves room for explicitly confined extensions. A collector may perform concurrent maintenance if it preserves the required abstract behavior. A monitor may have a separately specified intervention capability. A future parallel machine may execute proven-independent transitions under an appropriate execution agreement. These possibilities have distinct obligations; granting one does not silently grant unrestricted execution authority. The formal appendix states the model and its limits, while Future Directions separates accepted extension principles from mechanisms still to be developed.

This opens two different lines of inquiry for formal methods. A coordinator can call an external checker as a supplied capability. Separately, an analyzer can be derived for the coordinator's own execution semantics. The first requires an agreement about the checker's inputs, results and assumptions; the second needs an abstraction and soundness argu-

ment, including a treatment of foreign effects. Neither alone verifies a complete federation or its physical environment. Descriptors can carry tooling information, but attaching information does not establish its truth. Keeping these distinctions explicit makes proposed claims testable and clarifies the evidence still required.

1.5. What the Bootstrap Establishes

The present result is an executable language bootstrap centered on `VisitorSemanticsVM`. It includes parsing and lowering, explicit module entry, object and class construction, reified composition, lexical lookup, closures and return-home checks, a native heap collector, and a synchronous Java Alien/Expat boundary. Semantic regressions exercise the interactions between these mechanisms. The implementation chapters identify concrete policies and remaining limitations; executable tests are evidence about those policies, not a substitute for their specification.

The composition case study provides a small but concrete federation slice. Native source coordinates Java fixture services for model observations, validation, capacity checking and evidence. Reusable stages preserve their defining checker capabilities despite competing bindings in the receiving module. Its acceptance cases cover successful and rejected assessments, early failure and a selected foreign operation failing without being replayed. This tests capability flow and composition using controlled participants.

The slice does not yet integrate an industrial model repository or simulator, and it establishes no comparative productivity or performance advantage. The case-study catalog develops the next experiments around explicit baselines, measurable outcomes and failure conditions. Useful findings include cases where an existing tool or direct adapter is simpler. Current measurement infrastructure also supports language implementation experiments, with workload and measurement boundaries documented separately.

1.6. Positioning and Reading the Experiment

VarioHyve draws on object languages, capability security, behavioral reflection and explicit operational semantics. Openflexo, Epsilon and CorrLang provide important comparisons for model federation and coordination; Smalltalk and Newspeak inform the language lineage. Work on monitoring and analysis supplies ways to formulate the questions raised by observable execution. The Related Works part explains the relevant connections and differences without treating them as delivered integrations or inherited proofs.

The rest of the book moves from agreement to realization. **Specification** defines language behavior, including the foreign boundary. **Implementation** describes the Java bootstrap and its tooling. **Future** records extension principles, proposals and unresolved work. **Related Works** situates the experiment, and the **Appendix** gives the formal rules. This opening **Motivation** supplies the questions behind those parts: can explicit capabilities and preserved semantic boundaries make coordination easier to understand, and can explicit execution make its behavior easier to investigate?

Part II.

Specification

2. Program Structure

Operational definition: SOS entry rules.

2.1. A Program Is a Single Expression

The grammar entry is `top: expression? EOF`. Empty input is an empty expression; normal expression evaluation returns canonical null. A named or anonymous **class literal** can instead serve as a module. Naming it retains declaration metadata without creating an ambient binding. Module instantiation requires a class literal and rejects empty input or other expression forms without executing them.

The introductory Assessment example is such a module. Its instantiation service supplies the three existing objects `modelSource`, `checker` and `evidenceSink`; a caller later sends `assess(item)`. These selectors are application contracts, not built-in services.

2.2. Sources of an Object's Capabilities

An object's behavior can access capabilities through inherited behavior, mixin contributions, lexical nesting and explicitly supplied inputs. Inputs include constructor arguments during initialization and arguments to later messages. Further capabilities may become available through message results.

These routes do not change lookup order. Contributions retain their definition scopes; calling a class from another scope does not supply the caller's bindings.

Constructor parameters are available only during initialization. Retaining these inputs for later use requires slots or deliberate captures, as specified by the definition-scope rules. These rules apply equally to native and foreign capabilities.

A capability can be a native object, including a class with factory behavior, or an Alien. Its offered behavior may return further objects or create new ones. Authority therefore includes behavior and captured scopes, not merely objects present in an initial heap. Nesting shares enclosing capabilities; it is not an isolation boundary.

2.3. Top-Level Classes and Instantiation

Normal module definition evaluates the class over a **fresh empty top activation**. The execution frame exists, but has no capability accessors, lexical parent, receiver or enclosing application scope. The class inherits only from structural root `Object` and composes no mixins. This structural relationship supplies neither an `Object` binding nor a platform namespace. Every explicit top-level superclass or mixin clause is rejected before its expressions execute, including a clause spelled `Object`.

At entry, the module's declared constructor arguments are its only external capability

inputs. Zero, one or multiple arguments are allowed, with exact arity. The service passes existing objects, preserving identity and sharing through the ordinary boundary. It does not lend the module its own scope, registry or other services. Supplied objects retain their capabilities and captures; later explicit message arguments are additional inputs.

Class definition precedes construction, so constructor arguments cannot provide that top-level class's superclass or mixins. They can be stored and used when realizing **nested** classes:

```
class Module(baseClass, loggingMixin) {
    Base = baseClass;
    Logging = loggingMixin;
    class Worker(identifier) <: Logging < Base { id = identifier; }
}
```

The host receives the constructed module instance and explicitly chooses any entry message. Instantiation does not guess or send `main` or `run`. Construction failure does not return a successful instance; effects already performed are not rolled back, and an initializer may deliberately share `self`.

2.4. The Kernel Parameter

A `kernel` is an optional ordinary bundle, not a reserved name, mandatory parameter or global lookup mechanism. Services may be separate inputs or grouped in native/Alien bundles. Passing a narrow object does not remove authority already retained by its methods, enclosing scopes or composed contributions.

2.4.1. Optional Bootstrap Bundle

A platform may bundle canonical values, libraries and I/O capabilities. Bundle inventory and the behavior of its selectors are platform contracts. Source names such as `null`, `true` and `false` select behavior through ordinary lookup; their spelling establishes no canonical identity. See names and contracts. The current tooling bundle is described in the Java implementation.

2.5. Startup Seed Installation

This contract governs atomic finalization before publishing a runtime. It grants no authority to replace the seed during application execution. The accepted profiles are described in Possible Seeds and Worlds; the staged seed design records the rationale and implementation evidence.

When a builder adopts a replacement seed before publishing its runtime, it must substitute every retained reference to a retired seed identity using a validated old-to-new map. Updating only the canonical roles is insufficient. The substitution includes values in ordinary/immutable slots and captures, class and contribution relationships, private construction code, derived layouts/caches, root owners and VM-managed boundary state. Newly supplied objects are part of the rewrite too. Unchanged objects preserve their identities; references to a deliberately retired object consistently select its replacement.

The map's final targets must be fixed points, and conflicting mappings, unresolved cycles, incompatible structural uses and unsupported key collisions must fail before publication. Every retained use of a retired identity needs a valid replacement. Old machinery with no retained use can instead be reclaimed. Reused structural infrastructure is not implicitly retired.

The provider's `Object/0` must return an ordinary native class whose direct superclass is the provisional Object by identity. Its owned contribution and each directly applied mixin must have no instance slots, no constructor parameters, no initializer body, and no routed argument expressions. Stateless mixins, methods, member classes, and owned factories are permitted. Support for state in Object, owned or contributed, is indefinitely deferred.

Root Object adoption removes the candidate's edge to the provisional Object before substitution, representing Top and preventing self-inheritance. The installed root therefore has **no superclass and no instance state**. Repair preserves valid flat metaclasses, definition scopes, application occurrences, owned-contribution boundaries and construction semantics. Pointer substitution alone does not define new slot initialization or repair arbitrary data derived from replaced identities; unsupported cases require rejection or a separately specified preparation contract.

Finalization requires exclusive builder ownership and no active execution or callbacks. It performs reference repair, invariant validation, bootstrap-owner retirement and a forced full native collection before publishing the result as one atomic operation. The collection must run even below normal GC thresholds. Success requires no retired store entries or operative references to them in the final world. A partially installed runtime must never become usable. Preparation failure preserves the preceding private stage or discards the unpublished builder; it does not roll back or replay earlier foreign effects.

Foreign payloads remain opaque. The builder must prevent unmanaged provisional references from escaping and repair or close controlled bootstrap handles. Retiring a provisional Alien includes removing its native ownership roots and wrapper; it does not imply destroying the foreign payload or general foreign garbage collection. If the boundary cannot establish these conditions, installation fails. Published boundary identity laws then apply to the final world; this startup operation grants no live object-replacement authority.

2.6. Bootstrap Conformance Requirements

Definition, construction and synchronous callbacks share the requested execution budget. Host sends supply an empty caller activation; the target's definition supplies its lexical scope. The concrete APIs and validation are documented under module entry and in the implemented entry decision.

2.7. Development and Debugging Contexts

Richer tool evaluation contexts are distinct from normal module entry. A class created with tooling authority retains those captures and cannot silently be treated as an isolated module.

2.8. Error Handling

Current execution failures abort their native continuation; they are not catchable language values. A failed FN callback reports its diagnostic to foreign code, which may handle it and complete the enclosing NF call normally. Missing dispatch does not invoke a language-level `doesNotUnderstand` handler. Uninitialized reads, unresolved holes, invalid contributors and invalid returns also fail explicitly; return rules specify dead homes, initializer completion and callback barriers. Public diagnostics expose neither raw activation addresses nor host exceptions. The boundary preserves its defined failure kinds. Conditions and restarts remain proposed work.

3. Literals

The literal forms are numbers, strings, holes, lambdas, classes and objects. Class and object syntax is defined in Objects and Classes; this chapter owns scalar syntax, closure returns and initializer completion.

`null`, `true` and `false` are ordinary source names, not literal forms. Their occurrences are message sends under name resolution. Canonical null for omitted initializers or empty bodies is supplied directly by the language; it does not depend on a declaration named `null`.

Operational definition: SOS evaluation and return rules.

3.1. Number Literals

```
numberLiteral : '-'? NUMBER ;
NUMBER        : NATURAL ('.' NATURAL)? ;
fragment NATURAL : [0-9]+ ;
```

Examples: `42`, `-7`, `3.14159` and `-0.5`. Numbers use decimal notation; arithmetic and comparisons are messages such as `x.plus(1)` and `x.greaterThan(0)`. The syntax does not settle the numeric domain, precision or overflow behavior. The prototype's supported subset is documented under implementation limits.

3.2. String Literals

Single and double quotes produce the same runtime kind of value. Both support:

Escape	Meaning
`\\`	Backslash
`\"`, `\'`	Double or single quote
`\/`	Slash
`\b`, `\f`	Backspace, form feed
`\n`, `\r`, `\t`	Newline, carriage return, tab
`\uXXXX`	Unicode escape with four hexadecimal digits

```
"hello"
'it\'s fine'
"line one\nline two"
"\u0041" // A
```

3.2.1. Multi-Line Strings

Raw newlines are allowed inside either quoted form. They preserve all whitespace between the quotes, including indentation; `"a\nb"` and the corresponding two-line literal have the same content. See the lexer in vh.g4.

3.3. Hole Literals

`?` is an explicit incomplete expression, valid wherever an expression is expected. It has its own AST node and differs from both an omitted initializer and canonical null. A missing initializer supplies null; a hole remains unresolved.

3.3.1. Descriptors on Holes

Use parenthesized descriptor syntax, for example `(? : expected(Controller) : reason("integration input"))`. Descriptors can inform diagnostics or a separately selected analysis tool; they do not silently change raw execution.

3.3.2. Runtime Semantics

A hole supplies no normal value by itself. Reaching it must invoke the execution context's explicit hole policy. The concrete bootstrap reports `UnresolvedHole`. Interactive suspension, externally supplied replacements, abstract values, branching and synthesis are possible richer tooling policies, not implemented guarantees. Their interpretation must be specified by that context; accepting partial syntax does not imply an abstract interpreter.

3.3.3. Hole Identity

Each syntactic occurrence denotes a distinct hole. Tools should retain a stable internal identity, normally based on AST/source identity, for diagnostics, annotations, filling, traces or replay. A descriptor-supplied name or qualified source path may identify a hole to a tool. Two occurrences of `?` do not become one hole merely because their spelling is equal.

3.4. Lambda Literals

`fun` and `λ` introduce lambdas. Parentheses around the parameter list are optional when it is empty. Bodies are semicolon-separated statement lists; an empty body and a trailing semicolon are allowed. Formal lists follow the shared comma policy: a trailing comma adds no parameter, while an interior missing parameter is an error.

```
fun { 42 }
fun(x) { x.plus(1) }
λ(x, y) { x.plus(y) }
```

A lambda captures its evaluation scope, current receiver, contribution origin and lexical return home. It does not capture its later caller's scope. Captured activation data can outlive the creating invocation. A lambda supplies exact `apply/N` at its declared parameter count; binding invocation uses the ordinary eligibility rules. Optional descriptors follow the closing brace.

A lambda is a native object with its own identity and the inherited root Object protocol. Distinct evaluations create distinct objects. Object-method sends use the lambda as receiver; its `apply/N` body instead uses the captured receiver and return home. Inheritance neither changes lexical `self` nor exposes captures. There is no public Lambda factory or ambient Lambda binding; the private representation is described in Core Structures.

3.4.1. Implicit Return

Methods and lambdas return their final expression's value on normal completion. An empty body returns canonical null. This normal result returns from the current invocation; it does not perform a non-local return.

3.4.2. Early Return with ^

`^ expression` first evaluates its operand, then returns to the lexically enclosing method or initializer home. Lambda invocation creates no new home.

Creation context	Target of ^
Method body or a lambda created there	That method invocation
Initializer or a lambda created there	That initializer contribution invocation
Supplied outside lambda	Its original home, even when called during construction

A missing or retired home causes a dead-context error. Captured data does not preserve return authority. A return crossing foreign execution is governed by the boundary agreement. The default agreement requires a live home reachable without crossing an active Alien callback barrier; supporting agreements must preserve these native home and initializer rules. Successful unwinding retires intervening invocation homes. Operand failure propagates as failure rather than completing a return.

Class expressions retain definition-time bindings, but each later initializer invocation creates a fresh home. An outside lambda can unwind construction to its own live accessible home; it is never rebound to the initializer.

3.5. Class Literals

The `class` keyword creates a named or anonymous class value. See class literal syntax and the distinction between member declarations and class expressions.

3.6. Object Literals

`{ body }`, optionally prefixed by mixin and superclass clauses such as `<: M(p) < S(q) { body }`, realizes a fresh anonymous zero-parameter class and immediately sends its primary `new/0`. It has no constructor parameter list or factories; contributor arguments and initialization follow the class rules. See object literals.

3.7. Initializer completion

A class or object body may end with `|` and an optional ordinary statement list. This section executes after that contribution's ordered slot initializers:

```
class Config(input) {
    saved = input;
    value { saved }
    |
    ^ "done"
}
```

Primary construction returns the allocated receiver, not `"done"`. No member, method, factory or slot declarations follow the delimiter; ordinary statements, including `var` and class expressions, are allowed. An empty `|` section is valid.

Descriptor lists also use `|`. Terminate a preceding described member, method or factory with `;` to separate post-slot code: `value { saved } : Tag; | performWork`.

Each contribution invocation has a fresh private return home:

- **Slot phase:** returning to that home is an error, including during the final slot expression. The phase ends only after its final value is written.
- **Post-slot phase:** normal completion or `^ value` completes that contribution and ignores the value. Subsequent contributions still run.
- **Every exit:** normal completion, early return, failure and unwinding retire the home. Escaped data remain usable without retaining return authority.

Constructor parameters are visible in both initialization phases, under the construction scope rules. An outside lambda retains its original home. The default Alien callback barrier remains in force; any future crossing support must preserve these initializer phase restrictions. Failures neither replay initialization nor roll back effects or retract an escaped receiver. See the accepted policy.

4. Objects and Classes

This chapter defines class and object literals, member declarations, composition, and factories. Name Resolution defines lexical receivers; Message Sends defines invocation; Literals defines initializer completion and returns. Backend records belong to Runtime Object Representation.

Operational definition: SOS class and construction rules.

4.1. Top and the Class Algebra

`Top` is the empty constructor of the mixin–class algebra. At the level of instance-side composition, the inductive structure is:

```
ClassShape ::= Top | Apply(Mixin, ClassShape)
Object       = Apply(ObjectMixin, Apply(M1, ... Apply(Mn, Top)))
```

Like zero in an inductive definition of natural numbers, `Top` supplies the base case. Every nonempty class shape is a mixin applied to another shape, so even `Object` follows the same construction rule. `Apply` here is mathematical composition, not the source `apply` selector or a factory invocation.

`Top` contributes no methods, slots, member declarations, definition scope, capabilities or initialization. Lookup ends there with a miss and its layout is empty. Only `Object` has `Top` as its direct superclass; other native class chains reach it through `Object`. The selected seed supplies Object's owned contribution and any direct mixin applications. All these root contributions are stateless: no instance slots, constructor parameters, initializer bodies, or routed argument expressions. When there are no direct applications, its shape is simply `Apply(ObjectMixin, Top)`. State in Object is indefinitely deferred; a provider module may still hold capabilities in its own slots. The VM requires no instance methods on Object; one native seed supplies the protocol below.

The algebra describes composition structure, not class identity. A realized class retains its owned contribution, ordered direct applications, superclass, routes and metaclass. Expanding its shape neither manufactures intermediate class objects nor erases contribution scopes or occurrence identities. The formal rules give this projection precisely.

`Top` is a distinguished semantic base, not an instantiable class or a language-visible value in the current contract. It has no metaclass, factory, owned mixin to import, or public mirror reference. The class/metaclass rules below apply to realized classes. No built-in `Top` binding or new reserved name is introduced, and omitted superclass clauses still select `Object`. The empty **top activation** for module entry is a separate execution frame. Aliens acquire no native hierarchy through this definition.

The VM may represent the terminal case without allocating an object: the implementation refinement states the mapping and the conditions for reconsidering runtime reification.

4.2. The `class` Keyword

`class` distinguishes an instantiable class from an immediately evaluated object:

Form	Result
`class { body }` or `class() { body }`	Anonymous zero-parameter class
`class(params) { body }`	Anonymous class with constructor parameters
`class Name { body }` or `class Name() { body }`	Named zero-parameter class
`class Name(params) { body }`	Named class with constructor parameters
`{ body }`	One object, allocated and initialized on each evaluation
`<: Mixins(args) < Base(args) { body }`	One object with the selected mixins and superclass

4.2.1. Class Literal Syntax

```
namedClassLiteral
    : CLASS IDENT arguments? mixinApplications superclassApplication
      LB classBodyElement* initializerBody? RB (COLON descriptorList)* ;
anonymousClassLiteral
    : CLASS arguments? mixinApplications superclassApplication
      LB classBodyElement* initializerBody? RB (COLON descriptorList)* ;
classLiteral         : namedClassLiteral | anonymousClassLiteral ;
arguments            : LP formalList? RP ;
mixinApplications    : (WITH expressionList)? ;
superclassApplication: (EXTENDS expression)? ;
classBodyElement     : namedClassLiteral SEMICOLON? | methodDecl SEMICOLON?
                     | slotDecl SEMICOLON | factoryDecl SEMICOLON? ;
factoryDecl          : FACTORY methodDecl ;
methodDecl           : IDENT BANG* (arguments (COLON descriptorList)*)?
                       methodBody (COLON descriptorList)* ;
initializerBody      : PIPE statementList? ;
```

`WITH` is `<:` or `with`; `EXTENDS` is `<` or `extends`. Empty parameter parentheses are optional. Class bodies contain slots, methods, member classes and factories, followed by an optional final `|` statement section. Slot declarations require semicolons; methods, members and factories permit them. Descriptor chains retain their annotation syntax.

4.2.2. Object Literals

```
objectLiteral    : mixinApplications superclassApplication
                   LB objectBodyElement* initializerBody? RB (COLON
↪  descriptorList)* ;
```

```
objectBodyElement: namedClassLiteral SEMICOLON? | methodDecl SEMICOLON?
                 | slotDecl SEMICOLON ;
```

An object literal has no constructor parameter list or factories. Its optional mixin and superclass clauses have the same syntax, order and meaning as class clauses: mixins precede the superclass; `<:`/`with` and `</extends` are synonyms. Omitting the superclass selects structural root `Object`, without name lookup. Slot declarations still require their terminating semicolon.

```
<: M1(p1), M2(p2) < S(p3) { x = 5; }
< S(p) { x = 5; }
with M(p) { x = 5; }
```

Each evaluation realizes a fresh anonymous zero-parameter class, then immediately sends its generated primary `new/0` using ordinary exact message dispatch. Thus `<: M(p) < S(q) { body }` is equivalent to `(class <: M(p) < S(q) { body }).new`, with a factory-free body. Successful evaluation returns the initialized object. Repeated evaluation creates distinct class, metaclass and instance identities; it does not cache a singleton. Contributor factories and custom `apply` methods are not called or imported.

All constructor-clause rules apply. Targets resolve left-to-right for direct mixins, then the superclass, in the literal's evaluation scope before allocation. Routed arguments run later in its own initializer activation, immediately before their contribution is initialized. In particular, `self` in a target computation denotes the enclosing receiver, while `self` in a routed argument denotes the new object. Ordinary initializer lookup also applies: an own slot can shadow an enclosing name and is still uninitialized during routing. For example, `< S(x) { x = 5; }` cannot use its own pending `x` as an argument; use a distinct captured binding or a qualified outer receiver when that is intended.

The anonymous class captures the literal's evaluation scope, including deliberate creation inside another initializer. Contributors retain their own definition scopes and initializer-only constructor parameters; parameters are not added to instance-method bindings. Each contribution keeps its normal initialization order, occurrence, storage rules and private return home. Anonymous objects introduce no new `outer` qualifier.

An object literal remains an expression, not a valid normal module entry. Nested literals may use explicitly supplied module inputs, under the existing module restrictions.

4.2.3. Zero-Parameter Classes vs Singletons

For `class Config { debug = false; }`, `Config()` invokes its `apply/0` factory; `Config.new` and `Config.new()` each perform primary construction once. For `config = { debug = false; };`, the slot already holds the initialized object. See zero-arity invocation.

4.3. Root Object Protocol

In the Current Native Seed, native objects, including lambdas, inherit these ordinary methods from root `Object`:

```
equals(other) { _vm.same(self, other) }
```

```
hash { _vm.identityHash(self) }
toString { "an Object" }
```

These are normative defaults of this seed. Another seed may choose different root methods without changing lookup, invocation or boundary laws. Replacing an application kernel does not replace the structural root. The startup installation contract requires complete reference substitution and collection before publication, including class/contribution metadata affected by root adoption.

Root Object does not supply `same` or `identityHash`. Reliable identity and classification observations belong to an explicitly supplied IdentityMirror capability. The mirror observes subjects without invoking their methods. Defining these selector names on an application object cannot intercept mirror observations or the root defaults' direct use of `_vm`; no selector is thereby reserved on Aliens.

`equals` defaults to native identity equality, including identity recovered from an owned Expat. This seed's canonical null and Booleans participate with their native identities. The default `hash` returns an integer stable for the identity's lifetime; equal identities have equal hashes, but collisions are permitted. Hashes are neither addresses nor persistent identifiers across runtime sessions.

`equals`, `hash` and `toString` remain overridable. Equality overrides should define a reflexive, symmetric, transitive relation with compatible hashes. The defaults provide identity-derived results without a separately supplied mirror; the mirror grants observation independent of subject behavior.

The default `toString` returns exactly `"an Object"`. It reveals neither a class name nor an instance or class identity hash. Named and anonymous instances, class objects, metaclasses and lambdas retain this generic answer unless they override it. Canonical values keep their explicit `null`, `true` and `false` overrides. Diagnostic detail is therefore chosen by the object's own protocol.

`_vm` is a narrow seed capability stored on the provider module and retained through Object's definition scope. It is not an inherited Object slot, ambient module binding or accessor for IdentityMirror. Its operations cross the normal Alien/Expat boundary and expose no backend representation. Aliens retain their own protocols and do not acquire these methods by inheritance. Scalar Object behavior and identity remain subject to the scalar design decision.

4.4. Capabilities and Definition Scope

The capability sources do not change definition scope: calling a class does not capture its caller's scope. Named and anonymous classes follow the same rule. Examples with custom contributors or enclosing bindings assume an appropriate enclosing context; isolated modules and tooling evaluation have distinct entry contracts.

Constructor parameters are immutable bindings available only during initialization. Methods cannot resolve their own constructor arguments, even when called during initialization. Store persistent dependencies in slots. Lambdas and class expressions deliberately evaluated in an initializer may capture its parameters. Each contribution receives its own argument bindings; composition does not replace its captured definition dependencies.

4.4.1. Member Declarations and Class Expressions

A named class in body-declaration position declares a **member class**:

```
class Container(input) {
    value = input;
    class Member { read { value } }
    Captured = class { read { input } };
    NamedCapture = class Snapshot { read { input } };
}
```

`Member` binds to the enclosing receiver and its declaration/application origin. Its zero-argument accessor lazily realizes and caches one class per enclosing receiver and declaration occurrence. Repeated access returns that identity; different enclosing receivers or application occurrences obtain distinct classes. The member binds to the actual enclosing receiver while retaining the selected occurrence's captured lexical ancestry at every depth, including when selected through `super`. Its factories retain that same definition parent. Ordinary messages to the enclosing receiver still honor overrides; retaining ancestry does not restrict virtual sibling selection to the originating contribution. See lexical receiver selection.

The member can use stored `value`, but cannot acquire `Container`'s constructor binding `input`. Members are available regardless of their position among slot initializers.

`Captured` and `NamedCapture` are slots initialized by class expressions. Both capture the evaluation scope, including `input`. An expression's optional name is declaration metadata, not a binding introduced in the surrounding body. Anonymous forms occur only as expressions; named forms can be expressions or member declarations. Lowering must preserve this distinction.

Member and slot names share a namespace; duplicates in one body are errors. A member also conflicts with a zero-argument method of the same name and has no setter. A stored class is callable through its current value; a member advertises the selected declaration's constructor and owned `factory apply` arities before realization. Wrong-arity invocation evaluates no contributor targets. Selected realization failures, including cycles or abandonment during non-local return, are cached without replay. See member invocation for selection and overriding.

4.4.2. Initializer Bodies

The optional final `|` section executes after the contribution's ordered slots. Constructor parameters remain available. No member, method, slot or factory declarations follow the delimiter; ordinary statements, including local `var` and class expressions, may occur there. Its ordinary result or permitted early-return value is ignored. The complete phase, home-lifetime and failure rules are in Initializer completion.

4.5. Inheritance

`< Base(args)` selects a native class and inherits its complete instance-side chain: owned contribution, applied mixins and ancestors. Omitting the clause uses structural root `Object`, without looking up the name `Object`. An instance or Alien is not a class target;

no implicit `instance.class` conversion occurs. Native `super` cannot be a contributor target.

Construction runs the superclass's complete initializer schedule on the newly allocated receiver. Inheritance imports no factories; factories belong to the declaring class.

4.6. Mixins

`<: A(args), B(args)` applies the **owned reified contribution** of each selected native class: directly declared methods, slots, member classes, constructor formals and own initializer. It imports no source superclass, applied mixins, factories or argument routes. The definition scope remains captured. Resolving the source class itself still realizes its targets normally; applying its owned contribution does not evaluate those original constructor argument routes.

For example, if `A <: M1, M2 < S1`, `B <: M3, M4 < S2`, and `C <: A < B`, the contribution order is:

```
own(C), own(A), own(B), own(M3), own(M4), then the complete S2 chain
```

`own(X)` means X's owned contribution. No intermediate classes are introduced. Repeated applications share the reified definition but have distinct application occurrences and initializer invocations. A contributed method's `super` resumes from its application occurrence; see Name Resolution. Public mixin extraction and reflection are not part of the current language.

4.7. Constructor Clauses

A terminal ordinary invocation in a superclass or mixin clause has construction semantics. Its selector is read with an exact zero-argument send during class realization to select the contributor; its argument expressions are retained for each later construction. That terminal operation does not call a factory. Bare targets route zero arguments, and parentheses around the whole clause do not change its meaning. Other target expressions are evaluated normally during realization and must produce a native class.

The special exact-send form `self(args)` is therefore an ordinary target computation: `< self(x)` evaluates `x` and sends exact `apply/1` during realization, then routes no construction arguments. By contrast, `< self.apply(x)` reads exact `apply/0` during realization and routes `x` to the resulting contributor during each construction. In the first form, `x` uses the surrounding definition scope; in the second it uses the new class's initializer scope. These forms are not interchangeable.

4.7.1. Computed Contributor Targets

For `< registry.choose(kind).Base(x)`, realization evaluates `registry.choose(kind)` and reads `Base`; construction later evaluates `x`. Receiver subexpressions perform ordinary computations in the surrounding definition scope. The class being realized does not yet supply its own named `outer` qualifier there. Routed terminal arguments run

later in that class's initializer scope and may qualify it; both contexts retain their enclosing named declarations. Direct mixin targets run left-to-right, then the superclass target. Their choices are fixed for the realized class. This applies to class expressions and cached member classes.

4.7.2. Construction Order

For `class C(x) <: A(a), B(b) < S(s) { ... }`, successful primary construction:

1. Allocates one receiver with fresh uninitialized storage for its complete layout.
2. Evaluates `s` and runs S's complete initialization schedule on that receiver.
3. Evaluates `b` and runs B's owned initializer, then does the same for `a` and A.
4. Runs C's own ordered slots and final body, then returns the allocated receiver.

Each argument list is evaluated once, left-to-right, immediately before its private target invocation. Argument expressions run in the declaring contribution's initializer activation, with its constructor parameters bound. Superclass and applied-mixin calls receive their own bindings and fresh homes. The declaring routes and owned body share one logical home, established before routing begins. Routing precedes slot completion, so returning to that home from a route cannot finish construction early. Each contribution completes its slots and final body before the next contribution runs.

These calls are private identity-bound routines, never selectors named `initialize` or public operations on arbitrary receivers. Initializer writes bypass overridable setters. No component instances are allocated, no effects are replayed or rolled back, and escaped partial receivers are not retracted. Return and failure handling follows Initializer completion.

4.7.3. Ordered Overrides and Shared State

Own declarations have precedence, followed by direct mixins left-to-right, then the superclass recursively. At each contribution, selector/accessor choice follows Message Sends. A member override selects the whole declaration and its origin; it does not merge nested families automatically. Duplicate slots, members or exact method selectors within one body remain errors; an explicit method may override a generated slot accessor.

Every slot retains its logical identity `(applying class, occurrence, local slot)`. **Same-named contributions share storage.** Initialization runs in the reverse contribution order above, so later writes replace earlier values. Every occurrence still initializes, including repeated applications. Surviving methods may therefore observe shared state; this policy does not promise independent stateful mixins. Getter/setter eligibility and mutability belong to the selected declaration, even when storage is shared. Renaming, explicit state mapping and deep family composition require future rules.

4.8. Factories

A `factory` method is sent to the class object; an ordinary method is sent to its instances. Both use ordinary method invocation and lexical scope.

4.8.1. The Primary Factory (`new`)

Every source class and root `Object` has a generated, non-overridable primary `new/N`, where N is its constructor arity. Successful invocation allocates a fresh receiver, executes the complete construction schedule and returns that receiver. Its selector uses ordinary lookup; only its generated body contains private construction operations. Object literals automatically send this primary `new/0` to their newly realized anonymous class.

`factory new` is forbidden. An ordinary instance method named `new` is allowed. This restriction preserves a primary entry that guarantees fresh allocation and the complete initialization schedule, while `apply` and named factories retain control over their results. It does not make an arbitrary `new` send an allocation primitive. Inheriting `Object` does not add `new/0` to a class with parameters, and neither inheritance nor mixin application imports contributor factories.

4.8.2. The Callable Protocol (`apply`)

At constructor arity N, the default `apply/N` makes an exact `self.new/N` send. An owned `factory apply` at N replaces that default; factories at other arities coexist with it. A custom factory controls its result and causes no automatic allocation or initialization. It may call `new`, return a cached instance, or return another value.

```
class Point(xInput, yInput) {
    x = xInput;
    y = yInput;
    factory apply() { new(0, 0) }
    factory copy(point) { new(point.x, point.y) }
}
```

`Point(2, 3)` uses default `apply/2`; `Point()` uses the custom `apply/0`; `Point.new(2, 3)` performs primary construction directly. General callable selection belongs to Message Sends.

4.8.3. Named Factory Methods (`factory`)

Named factories such as `Point.copy(p)` are ordinary class-side methods with full result control. In a factory, `self` is the class object; lexical sends and qualified `outer` use the factory's declaration origin and the class's definition scope. A factory on a member class can therefore access its bound enclosing receiver. Invoking it does not capture the caller or instantiation service.

4.8.4. Bootstrap Metaclasses

Each source class C has a distinct metaclass `C class`, containing only C's owned factories, with direct superclass `Class` and structural class `Metaclass`. Factory `super` continues at `Class`, independently of C's instance superclass. The root relationships are:

Identity	Structural class	Direct superclass
`Object`	`Object class`	`Top`

Identity	Structural class	Direct superclass
`Class`	`Class class`	`Object`
`Metaclass`	`Metaclass class`	`Class`
Each of the three root metaclasses	`Metaclass`	`Class`

Thus `Metaclass` and `Metaclass class` form a shared two-object class-of cycle. `Top` terminates superclass chains without participating in this class-of graph. `Object` retains its empty initializer and `new/0`/default `apply/0`. `Class`, `Metaclass` and metaclass objects have no public construction factories; the common instance protocol of `Class` is initially empty. Structural links create no public `class` or `outer` selector, ambient binding or reflection API.

5. Slots and Accessors

Slots hold object state and expose it through generated accessor messages.

Operational definition: SOS accessor rules.

5.1. Slot Declaration

```
class Point(a, b) {
    x = a;
    y = b;
    label;
}
```

A slot declaration ends with `;`. Its = introduces initialization, whereas expression assignment invokes a setter. An omitted initializer supplies canonical null; storage awaiting its initializer is uninitialized, and reading it is an error.

5.2. Contribution Slots and Storage

Each contributed slot retains its application occurrence and local position. The actual receiver's complete layout maps these logical identities to storage. The current policy shares one location among same-named slots, while preserving their separate initializers and effects. A selected accessor uses its own logical identity in that layout.

Declaration initialization writes through this mapping without calling an overridable setter. Later initializer writes to shared storage replace earlier ones. Sharing and construction order are specified in Objects and Classes; explicit sharing policies and contribution-private access remain deferred.

5.3. Synthesized Accessors

Source instance slots and `var` locals supply:

Send	Selector	Result
`x`	`x/0`	Current stored value
`x!(v)`	`x!/1`	Store and return `v`

`x/1` and `x!/1` are different selectors. `x(args)` is invocation syntax, not a third synthesized accessor.

5.4. The `apply` Protocol and Callable Objects

Generated getters may supply invocation through their returned value. Eligibility, exact arity, zero-argument behavior and retained selection evidence are defined by Message Sends; the Apply guide introduces callable values.

5.5. Setter Return Value

A generated setter returns the new value, not `self`. Consequently `p.x!(2).y!(3)` sends `y!/1` to `2`. Use `p..x!(2)..y!(3)` to keep both messages on `p`. A custom setter returns its body result; assignment syntax does not force it to return the argument or perform a storage write.

5.6. Shadowing Accessors

An explicit method with the same selector overrides a generated accessor. Generated-accessor invocation eligibility then follows the masking rule.

Calling `x!(v)` inside a custom `x!/1` dispatches normally, potentially back to that method; it does not expose the overridden storage primitive. There is no public raw-slot operation. Use a distinct backing slot when needed:

```
{
    _x = 0;
    x { _x }
    x!(v) { validate(v); _x!(v) }
}
```

The underscore is only a naming convention, not access control.

5.7. `var` Locals

`var count = 0;` declares a mutable local with getter/setter messages on a backend-private activation object. All locals in a source scope are reserved before its initializers run. A reference to a later local therefore selects that local and fails if it is still uninitialized; it does not fall through to an outer name. Duplicate local declarations in the same statement list are errors; a local may shadow a parameter or a binding from an enclosing scope. This supports mutual recursion when the lambdas run after their bindings are initialized.

5.8. Parameter Immutability

Each formal parameter list requires distinct names. Parameters have immutable getter accessors and no synthesized setter. A request for `parameter!` therefore cannot mutate the parameter; ordinary lookup may find another applicable setter or fail. Use a `var` local for a mutable copy. Constructor-parameter lifetime is specified with construction scopes.

5.9. The ! Convention

`!` suffixes are allowed only on method/message selectors, including repeated suffixes. Slot, parameter and local names are plain identifiers. A slot `x` thus generates unambiguous selector `x!`.

The suffix convention signals mutation or effects; it is not a static effect system. A plain-named slot may hold an effectful lambda, and invoking that lambda need not display `!` at the call site. An explicit forwarding method such as `commit!(v) { callback(v) }` can communicate that intent. User-defined bang methods need not be setters.

5.10. Descriptors on Slots

Descriptors may annotate slots and locals. They do not change raw accessor behavior or make a slot immutable.

6. Message Sends

A selector is a name/arity pair. Ordinary access, invocation and assignment all send messages; there is no public direct field access or separate binding namespace. Receiver origins are defined in Name Resolution.

Operational definition: SOS selection rules.

6.1. Implicit Message Send

Form	Request
`x`	Exact `x/0`: return the selected accessor or method result.
`x()`	Invocation at arity zero.
`x(a, b)`	Invocation at arity two.
`x!(v)`	Invocation of `x!/1`, a distinct selector from `x/1`.

Parentheses remain semantically significant through lowering and dispatch. In particular, invocation cannot always be lowered to getter-then-apply.

6.1.1. List Separators

A single trailing comma is permitted in a nonempty argument or formal list, including constructor parameters and mixin application lists. `f(1,)` has one argument; `fun(x,) {x}` has one formal. Empty parentheses have no items. A leading comma or a missing item between commas is an error, not an omitted value or a new formal: `f(,1)`, `f(1,,2)` and `fun(x,,y) {y}` are invalid. Descriptor lists use their separately specified optional trailing pipe.

The current reader reports a nonblocking style warning for trailing commas. It preserves warnings on successful reads and still runs source checks; warnings do not alter arity or execution admission.

6.2. Self-Dispatch Message Send (Leading Dot)

`.m(args)` is equivalent to `self.m(args)`; both skip lexical receiver selection. Leading `..` likewise cascades on `self`. Prefer the explicit forms when nested scopes would make the receiver unclear.

6.3. Explicit Message Send (Dot)

`receiver.name` is an exact zero-argument send. `receiver.name(args)` invokes at the written arity. Evaluate the receiver first, then arguments once from left to right, then select

behavior. Dot chains send each message to the previous result: `a.b.c(3)` sends `c/1` to the result of `a.b`.

6.3.1. Invoking the Current Receiver

`self(args)` sends exact `apply/N` to the current receiver after evaluating its arguments once from left to right. `self.apply(args)` instead invokes `apply/N` under the ordinary candidate rules below. An exact `apply/N` method behaves the same in both forms; a generated accessor can distinguish them:

```
{
    apply = fun { 42 };
    exact { self() }
    invoked { self.apply() }
}
```

Here `exact` returns the stored lambda, while `invoked` returns `42`. With a one-argument lambda stored in `apply`, `self(42)` has no exact `apply/1` and fails; `self.apply(42)` may invoke that stored callable. Neither form invokes a callable returned by a selected ordinary method a second time.

6.3.2. No Operator Precedence

Arithmetic and comparisons use messages. Consequently `3.plus(4).times(2)` groups as `(3.plus(4)).times(2)`; use `3.plus(4.times(2))` for the other grouping. General operator syntax is deferred.

6.4. Dispatch Rules

The following rules own invocation selection for methods, generated accessors, lambdas, native callable objects and member factories. Invocation preserves the selected behavior's receiver, declaration origin, lexical captures and return home. Parameters and locals expose generated accessors on private activation objects; direct backend reads may implement these messages.

6.4.1. Candidate Selection

For `name/N`, examine each searched hierarchy contribution in order: owned contribution, direct mixins left-to-right, then the superclass recursively.

1. Prefer an exact operation `name/N`. At `N = 0`, a generated getter goes through the next rule instead of satisfying invocation by returning a value.
2. Otherwise consider the effective generated `name/0` accessor. A storage accessor qualifies when its value supplies exactly `apply/N`. A member accessor qualifies through its effective declaration's factory signatures.
3. Continue only on a miss. Exhausting the search produces a missing-selector diagnostic.

Exact-operation preference is local to each contribution: a nearer eligible accessor can beat a farther method. `super` starts after the executing contribution's exact occurrence.

Getter masking. The first `name/0` operation masks later getters in that hierarchy search, even when its candidate is ineligible. An ordinary overriding getter is never executed to discover a callable, and its hidden storage is never inspected. Later exact `name/N` methods can still match. A `super` search starts masking afresh at its redirected origin.

Callable capability. Native values qualify through an exact `apply/N` method, including inherited methods, or a lambda's declared arity. A stored `apply` accessor does not recursively make its containing object callable. A stored Alien can qualify through effect-free boundary selection of `apply/N`, including `ForeignCallable`. An explicit Alien receiver instead selects the requested foreign name/arity directly: no foreign getter is probed for a callable result.

Implicit invocation. First perform declaration-relative receiver selection, using these same candidates. Then dispatch the original invocation on that selected receiver. Method and eligible-accessor overrides both participate. Eligibility does not freeze the implementation, and final dispatch failure does not resume lexical search.

Evidence and failures. Retain an eligible accessor's value and selected callable operation through final dispatch. Revisiting the same candidate reuses them without rereading storage or resolving a foreign operation. Changing a stored value can change eligibility for a later invocation.

Miss: continue lookup	Error: stop the operation
Absent selector; wrong-arity lambda; non-callable value; unsupported member factory arity	Uninitialized read; foreign selection failure; selected realization or execution failure; boundary conversion/ownership failure

Eligibility errors precede dispatch: an uninitialized read or foreign selection failure remains fatal even when the receiver has an overriding `name/N` method. Neither errors nor non-local returns restart lookup or replay evaluation.

6.4.2. Invocation at Arity Zero

Declaration	Bare `f`	`f()`
`f { 7 }`	`7`	`7`
`f = fun { 7 };`	Lambda	`7`
`f { fun { 7 } }`	Lambda	Lambda
`f = 7;`	`7`	Miss at this accessor
`class f {}`	Cached class	Instance from `apply/0`

A selected method executes exactly once; its returned callable is not implicitly invoked. Write `obj.f().apply()` to invoke that result explicitly. Replacing a generated getter with a computed getter can therefore preserve bare reads while changing invocation behavior.

6.4.3. Member-Class Invocation

A generated member accessor advertises its effective declaration's constructor arity and owned `factory apply` arities. An overriding declaration supplies its own signatures. Flat metaclasses import no contributor factories, so candidate testing needs neither contributor lookup nor class realization.

`class Widget(x, y) { factory apply(x) { new(x, x) } }` supplies invocation arities one and two. `Widget()` is a miss without evaluating definition clauses, whether its cache is empty, successful or failed.

Metadata eligibility first selects the lexical receiver. If final dispatch chooses an overriding method, the member remains unrealized, even if its cache already holds a failure. When dispatch selects the member factory, commit to that candidate, realize it through its receiver-and-declaration-occurrence cache, then invoke exact `apply/N` with the saved arguments. Bare access also realizes through that cache. A selected failure is cached and propagated.

Realization evaluates direct mixin targets left-to-right, then the superclass target; their computations may have effects and must produce native classes, excluding metaclasses. Terminal constructor arguments remain deferred until construction. See class realization and the accepted decision.

6.5. Cascade (`..`)

A cascade evaluates its receiver once and sends every arm to that saved receiver. Arms execute left-to-right; each arm's arguments execute immediately before its send. Preserve each arm's exact-access versus invocation mode and any setter sugar:

```
p..x!(2)..y!(3)..describe
```

The intermediate results are discarded. By contrast, `p.x!(2).y!(3)` sends `y!/1` to the result of `x!/1`, which is `2` for a generated setter. A failed arm or non-local return prevents later arms and their arguments from running. Earlier effects are neither undone nor replayed.

Lowering uses a fresh private temporary and ordinary sends; it must preserve captures and return homes without exposing the temporary scope to programs.

6.6. Cascade Return Value

The whole cascade returns its last arm's result, not necessarily the receiver. To return an existing receiver, write `p..a..b; p`.

6.7. Implicit-Receiver Cascade (Leading `..`)

`..m..n` means `self..m..n` and skips lexical receiver selection. `m..n..p` instead evaluates implicit `m` first, then sends `n` and `p` to its result. Thus a leading cascade and a cascade whose receiver is an implicit send select different receivers.

6.8. Combining Dot and Cascade

Expression	Meaning
`a.b..c..d`	Save `a.b`; send `c` and `d` to it.
`(a..b..c).d`	Send `d` to the last cascade result.
`a..b..c.d`	Invalid without the parentheses above.
`a..take(b..x..y)..finish`	Arguments admit nested cascades directly.
`super.make..a..b`	Cascade on the result of the completed super send.

There is no `super..a..b` syntax: `super` is not a receiver value. Assignment has a separate cascade fence. The precise grammar is vh.g4.

7. Apply and Callable Values

`apply/N` is the callable capability at arity N. A lambda supplies it at its declared arity; a class's generated `apply/N` delegates to primary `new/N`; an ordinary object can define it as a method. Owned custom class factories may replace or add `apply` arities; see factories.

The language fixes `apply/N` as the protocol connecting invocation syntax to callable values, so methods and stored callables can participate in the same selection rules. Its implementation and result belong to the selected receiver; the spelling does not imply allocation, lambda execution or any other particular body on an arbitrary object.

```
{
    handler = fun(request) { request };
    run(request) { handler(request) }
}
```

Retrieving `handler` returns the lambda. `handler(request)` can use the generated getter and invoke its result. Exact methods and accessor routes compete under the selection rules, so invocation is not unconditional getter-then-apply. A computed getter's result can be called explicitly with `object.handler.apply(request)`.

Bare `object.m` sends `m/0`; it does not produce a bound method. For the distinction between `f` and `f()`, see zero-argument invocation. Member declarations use deferred factory selection.

Method reification is a future tear-off proposal. No tear-off syntax or public method-reification API is part of the executable language.

8. Syntactic Sugar

Sugar preserves ordinary selector dispatch, receiver origins, evaluation order and failure behavior. Name Resolution defines receiver forms; Message Sends defines invocation and cascades.

8.1. = Setter Sugar

In a declaration, = introduces slot/local initialization. In expression position, `name = expr` requests `name!/1`; it creates no binding and grants no direct storage access.

8.1.1. Desugaring

```
x = v          // x!(v)
p.x = v        // p.x!(v)
a = b = 3      // a!(b!(3))
```

Evaluate an explicit receiver first, then the RHS once, then invoke the setter. Chaining is right-associative. Custom and foreign setters use normal dispatch.

8.1.2. Valid Contexts

Sugar	Canonical form
`x = v`	`x!(v)`
`.x = v`	`.x!(v)`
`self.x = v`	`self.x!(v)`
`p.x = v`	`p.x!(v)`
`super.x = v`	`super.x!(v)`
`outer C.x = v`	`outer C.x!(v)`
`p..x=v..y=w`	`p..x!(v)..y!(w)`
`..x=v..y=w`	`self..x!(v)..y!(w)`

The cascade receiver is evaluated once. `super..` is invalid because `super` is not a value; an ordinary super send remains valid. Explicit Alien selectors such as `alien.outer = v` request the foreign selector `outer!/1`.

The selector before = must have no bang suffix. Write a canonical send for additional bangs; `x! = v` is not assignment sugar.

Assignments are expressions, including in arguments, returns and initializer bodies: `consume(x = 7)` and `var result = x = 7;` are valid.

8.1.3. Return Value

Assignment returns exactly the setter result. A generated setter stores and returns its argument. Custom setters may return something else or perform no write. If `b!/1` returns `7`, then `a = b = 3` passes `7` to `a!/1`. A missing setter fails normally; parameters gain no writable accessor from assignment syntax.

8.1.4. The Cascade Fence

An assignment RHS is a `nonCascadeExpr`, so an unparenthesized `..` ends it:

Expression	Grouping / result
`r..a=3..b=5`	Two setter arms on `r`.
`x = p..a..b`	`(x = p)..a..b`: cascade on the setter result.
`x = (p..a..b)`	Pass the last cascade result to `x!/1`.

This is a grouping boundary, not a ban on cascades after assignments. Declaration initializers and canonical invocation arguments accept full expressions: `var x = p..a..b;`, slot declaration `x = p..a..b;` and `x!(p..a..b)` all use the cascade's final result directly.

The authoritative syntax is in the `message`, `explicitMessage`, `expression` and `nonCascadeExpr` rules of vh.g4.

9. Name Resolution

Implicit sends first select a receiver from lexical declarations, then dispatch on that receiver. Lookup is fixed by the language; there is no global environment or platform-selectable lookup policy.

Operational definition: SOS lexical receiver selection.

9.1. Names and Contracts

An ordinary name selects behavior through message lookup; its spelling alone does not identify a value, grant authority or impose a protocol. The selected object's behavior and the contract with its clients give that name meaning. This applies equally to `null`, `true`, `false`, `Object`, `kernel` and application selectors such as `ifTrue` or `plus`. They introduce no ambient bindings. Parameters, locals, slots and methods may use these names, subject to the ordinary declaration rules. Without an applicable declaration, a send fails under the ordinary lookup rules.

Canonical null and Boolean identities, and structural root Object, are governed by the seed and core-object contracts. Those identities do not reserve source names for obtaining them. An omitted initializer supplies canonical null directly; an explicit initializer `null` sends `null/0` in its lexical context. A name or a matching protocol never establishes canonical identity at the foreign boundary.

The specification explicitly fixes grammar keywords and the receiver forms below. It also fixes the callable selector `apply/N`, the generated primary factory `new/N` and the setter selector mapping `x!/1`. These contracts connect syntax to messages; the selected operations still use ordinary dispatch. They do not give other selectors intrinsic behavior. Seed methods and explicitly supplied foreign/tooling capabilities have their own stated contracts, not meanings inferred from names on arbitrary objects.

9.2. Lookup Order

Search the current method/lambda's parameter and local accessors, then enclosing lexical scope objects from nearest to farthest. An activation contributes its own accessors. A lexical object scope contributes only declarations from its originating contribution, associated with its bound receiver. Arbitrary members added by a subclass do not become lexical declarations.

A selected contribution and its member classes retain their exact application occurrence and captured lexical ancestry at every nesting depth. Qualified `outer` and implicit lexical receiver selection follow that ancestry. Messages dispatch normally on the selected receiver, and `super` continues after the executing occurrence. Repeated applications of one declaration remain distinct; lexical traversal never reselects an earlier application on the same receiver.

For an exact send, the nearest scope declaring the selector selects its receiver; exact dispatch then honors overrides on that receiver. For invocation, use the candidate rules:

1. A declared exact `name/N` operation qualifies the scope, except that a generated getter at arity zero must establish callable eligibility.
2. Otherwise, if the scope declares `name/0`, select its effective getter on the associated receiver. An eligible generated accessor qualifies the scope. At arity zero, an ordinary method overriding that getter also qualifies directly.
3. A miss continues outward; an eligibility error stops selection.
4. Once qualified, dispatch the original invocation uniformly on that receiver, retaining successful accessor evidence.
5. If all lexical scopes miss, dispatch on current `self` through its owned contribution, direct mixins left-to-right, then superclass chain.

A subclass `name/N` alone cannot make an ineligible declared getter qualify. Once eligible, that receiver's overrides apply normally. An operation added on a different inner receiver cannot capture an enclosing dependency.

```
{
    class Base {
        f = fun(x) { 7 };
        run { f(1) }
    }
    class Child < Base { f(x) { 42 } }
}.Child().run // 42: Base selects the receiver; Child overrides f/1
```

If `Base.f` instead holds a non-callable or wrong-arity value, lexical search continues before the final self invocation. Getter masking, failure handling and retained callable evidence are specified once in Message Sends.

9.2.1. Capabilities and declaration scopes

The instantiating caller does not supply a class's lexical scope. Constructor parameters, member declarations and class expressions obey the construction scope rules. Normal module entry and explicit tooling scopes are distinguished in Program Structure.

9.3. `self` — Current Receiver

`self` denotes the current native receiver with fixed meaning. It parses as an identifier but is not an ordinary getter; native declarations cannot shadow it. It can be passed as a value and used in chains such as `self.x`. An explicit native receiver has no `self` selector. `alien.self` is instead an ordinary foreign selector with adapter-defined meaning.

`self(args)` is a special receiver form: it sends exact `apply/N` to the current receiver, with N equal to the argument count. It does not invoke a selector named `self`, perform lexical receiver selection or use a generated accessor to find a callable. `self.apply(args)` is an ordinary explicit invocation and follows the invocation selection rules. The distinction also affects constructor clauses.

9.3.1. Leading-Dot Shorthand for Self

`.m(args)` equals `self.m(args)`; `..m..n` equals `self..m..n`. Both bypass lexical receiver selection. See cascades.

9.4. `outer` — Qualified Lexical Receiver

`outer ClassName` selects the ordinary receiver for the nearest matching named lexically enclosing declaration, including the immediately containing class. Subsequent messages dispatch normally on that receiver.

```
class Counter(start) {
    n = start;
    class Stepper {
        step { outer Counter.n!(outer Counter.n.plus(1)) }
        current { outer Stepper }
    }
}
```

`outer Stepper` selects the current Stepper instance. The qualifier is declaration metadata, not a variable, evaluated accessor or dynamic class name. Repeated class names choose the nearest matching declaration. An unrelated subclass or superclass name is not thereby an enclosing qualifier.

Lambdas retain their enclosing declaration context. Named class expressions introduce qualifiers for their bodies; anonymous class expressions and object literals introduce no invented name. They preserve access to enclosing named declarations. Captured parameters and locals remain internal dependencies, not activation objects exposed by `outer`.

Contributed methods preserve their definition and application origins; applying a mixin elsewhere cannot substitute the receiving class's outer scope. Member classes also retain their bound receiver and declaration origin. This holds at every nesting depth: a member selected from the second of two applications retains that application's farther enclosing objects, even when both applications share the same declaration and immediate receiver. The repeated-parent example therefore returns `42`, preserving the selected `b.Family` ancestry. This is an intentional difference from Newspeak's deeper nearest-parent reselection rule.

`outer C` is a standalone receiver value: grouping, passing it as an argument and ordinary sends are allowed. It grants no public parent getter, activation reflection or additional access authority. Bare `outer`, unqualified `outer.message` and native `object.outer` are invalid. `alien.outer` is an ordinary foreign selector; repeated sends follow that adapter's protocol.

An unknown or non-enclosing qualifier is a compile-time semantic error, including in an unused method or lambda. The source checks validate the qualifier against the named declarations enclosing that occurrence before lowering or execution. This is lexical declaration lookup, independent of same-named parameters, locals or accessors.

A class's own qualifier is available in its body, factories and routed terminal constructor arguments. Contributor target expressions run in the surrounding definition scope, so they cannot use the new class's qualifier; surrounding named declarations remain available there. See constructor clauses.

The runtime retains a defensive check for directly supplied core trees and explicitly unchecked tooling input. It follows captured lexical scopes and rejects an invalid qualifier before evaluating arguments of a send using that receiver. Source validation does not replace the exact application ancestry with a new runtime binding mechanism. Qualifying a normal module's own declaration can select its instance; it cannot expose the instantiation service, tooling capabilities or empty top activation.

9.5. `super` — Lookup Redirector

`super.m(args)` retains `self` and starts lookup immediately after the executing contribution's exact application occurrence. It continues through later mixins and the superclass chain, preserving the send mode and using fresh getter masking. Repeated applications of one mixin remain distinct; lookup does not restart after the first occurrence of its definition.

`super` is not a value: standalone use, assignment and `super..` are invalid. Its native meaning cannot be overridden. The explicit selector in `alien.super` belongs to the foreign protocol.

9.6. Lexical, Self, and Outer Dispatch — Compared

Origin	Receiver/search
`m(args)`	Declaration-relative lexical selection, then self fallback.
`self.m(args)` / `.m(args)`	Current receiver's hierarchy only.
`outer C.m(args)`	Lexical receiver for C, then ordinary dispatch.
`super.m(args)`	Current receiver, after the executing occurrence.
`receiver.m(args)`	Evaluate receiver once; native hierarchy or Alien protocol.

These origins are language forms, not a universal protocol implemented by every value. Aliens acquire no native class, mixin or lexical outer.

9.7. Reserved Words

9.7.1. Grammar Keywords

Grammar keywords cannot be declaration identifiers. `outer` and `super` are also allowed as explicit foreign selector spellings.

Tokens	Purpose
`class`, `factory`	Class and factory declarations.
`var`	Local declaration.

Tokens	Purpose
`fun`, `λ`	Lambda literal.
`outer`, `super`	Native receiver forms.
`<`, `extends`	Superclass clause.
`<:`, `with`	Mixin clause.

9.7.2. Semantically Reserved Identifiers

Identifier	Restriction
`self`	Cannot name a native slot, parameter, local, member class, method or factory, or be sent as a native selector.
`new`	`factory new` is rejected because the primary source-class factory is reserved. Ordinary instance methods named `new` remain allowed.

These spellings parse as identifiers; the standard source checks reject invalid declarations before execution. See primary construction.

10. Recursion

Recursion uses ordinary late-bound messages and lexical captures; it requires no special recursive declaration form. It remains subject to normal selection, including overrides and callable eligibility.

10.1. Named Method Recursion

A method can send its own selector or another method's selector. Declarations are available regardless of textual order, enabling mutual recursion. Use `self.m(...)` when current-receiver dispatch is intended; implicit `m(...)` still performs lexical receiver selection.

10.2. Slot-Bound Lambda Recursion

A stored lambda can refer to the accessor holding it:

```
{
    fact = fun(n) {
        n.eq(0).ifTrue(fun { 1 }, fun { n.times(fact(n.minus(1))) })
    };
}
```

Its body resolves `fact` when invoked, after the slot is initialized. Arithmetic and branching here assume suitable capabilities. Normal expression results return through recursive invocations. A `^` would target the creating initializer, normally already retired; see return homes.

10.3. Local Lambda Recursion

A `var` local can similarly hold a recursive lambda. Its accessor belongs to the captured activation. A lambda's caller supplies no replacement lexical scope or implicit self-name.

10.4. Mutual Recursion

All locals in a source scope are reserved before initialization. Lambdas may therefore capture references to later local declarations, provided they run after those bindings are initialized. Reading a reserved but uninitialized local is an error, not an outer lookup. See locals.

10.5. Passing Recursive Lambdas as Arguments

Passing or returning a recursive lambda preserves its captured accessors. The caller's parameter name does not become visible in the lambda body. Captured data may survive the defining method, but its return home does not: ordinary results remain valid, while ^ to a retired home fails.

For name-based recursion, place the lambda in a captured slot or local. An unbound anonymous lambda has no automatically provided name for itself.

10.6. Tail-Call Optimization

The prototype uses explicit continuations for native execution but does not implement tail-call elimination. This is not a bounded-memory guarantee. Foreign callback depth and other execution limits are separate; see implementation and future TCO.

11. Descriptors

Descriptors attach optional metadata to syntax for external tools. They have no operational semantics in the language. A descriptor payload is arbitrary text; VarioHyve does not require it to be a VarioHyve expression.

11.1. Design Constraint

Removing all descriptors preserves raw execution, including results, effects, failures and execution budgets. The ordinary frontend recognizes only descriptor framing, groups and attachment sites. It does not tokenize the payload as program code, parse an expression from it, run source checks on it, or interpret it during lowering or execution. A syntax error in a well-framed payload cannot prevent the surrounding program from executing.

Consuming tools own payload parsing, validation and interpretation. They may use another language or explicitly request VarioHyve's frontend and runtime as capabilities. An annotation named `immutable`, `cached`, `public` or `module` alone grants no runtime semantics. Class, superclass and mixin clauses remain executable language constructs independently of descriptors.

11.2. Syntax

```
element : payload
element : first | second
element : first : second
```

`|` separates entries within one group; `:` chains groups. Entries retain exact text, including whitespace, comments and escape spellings; groups retain order and multiplicity. Payload text has no language-mandated meaning.

11.2.1. Short annotations

Short annotations require no extra delimiters:

```
class Arithmetic {
    add!(a: int, b: int): int { a.plus(b) } : documented;
    legacy(a: int, b: int) { a.plus(b) } : int;
    count = 0 : tracked | nonNegative;
}
```

A short payload ends at a top-level `:`, `|`, `,`, `;`, `)`, `}`, newline or end of input. Balanced parentheses, braces and brackets, quoted text, and comments protect separators inside them. This is lexical framing only: quote escapes are not validated, and names, operators and payload grammar are not interpreted. `<:` remains part of a short payload rather than starting another descriptor group. Whitespace around a short payload is retained, not trimmed.

To continue with another member on the same line after its short annotation, use `;`. A newline also ends a short payload. Multiline quoted text and balanced regions remain part of that payload. Terminate a described declaration with `;` before a final initializer body, because an adjacent `|` otherwise continues the descriptor group.

11.2.2. Method headers

Methods and factories with an explicit parameter list may also carry descriptor groups before the body: `add!(x: int): int { x.plus(2) } : something`. Both sets describe the method (or factory), in written order: header groups first, trailing groups second. The language assigns neither position a type or return contract. Formal-parameter descriptors still describe their individual parameters.

In a header descriptor only, the first unquoted, unnested `{` starts the executable body. Braces inside parentheses, brackets, quotes or an optional verbatim fence remain payload text: `answer(): tool({key: value}) { 42 }` and `answer(): [=[arbitrary { unfinished]=] { 42 }` are both valid. Pipes and chained colons work in headers as elsewhere. Comments shield their contents too.

Use explicit `()` for a zero-parameter header, such as `answer(): int { 42 }`. This preserves existing uninitialized slots: `answer: int {42};` remains a slot whose descriptor includes the braces. Existing trailing descriptors and omitted parameter lists, such as `answer { 42 } : int`, remain valid. Other attachment sites, including lambda descriptors, retain their existing framing.

11.2.3. Verbatim annotations

An optional fence permits arbitrary text, including unmatched brackets or quotes, newlines, and every short-form separator:

```
(42 : [=[@foreign { syntax: "unfinished; ) | anything]=])
```

At the beginning of an entry, after optional spaces or tabs, an opening `[` followed by one or more `=` characters and another `[` is matched by `]`, the same number of `=` characters, and `]`. The payload is exactly the text between these markers. Nothing inside it is parsed, decoded, balanced or normalized. Choose more `=` characters when the payload contains a closing marker with fewer: `[==[text containing ]=] remains intact]==]`. `[=[]=]` is an empty payload. Fences are optional at every descriptor attachment site and can be mixed with short entries in the same group. A closing fence is an explicit boundary, so another member can follow it on the same line without a semicolon.

The framing must identify where program code resumes. An unterminated fence, quote, block comment or unbalanced short-form enclosure is a framing error; use a fence when unmatched delimiters are intentional payload content. This boundary requirement does not validate the payload's own language. Empty short entries are retained; the optional

trailing | does not create an additional entry.

11.3. Where Descriptors Apply

Attachment	Example
Slot or local	`x = 0 : tracked;` / `var x = 0 : tracked;`
Method or factory	`move(dx, dy): result { ... } : cached;` / `factory make(): tag { ... }`
Class or object	`class { ... } : typed` / `{ ... } : module`
Lambda	`fun(x) { x } : annotation`
Formal parameter	`(x : nonNeg, y : nonNeg)`
Expression	`(3 : immutable : tracked)`

11.3.1. Expression-Level Descriptors

Parenthesized expression descriptors distinguish the annotated expression from its surrounding declaration:

```
x = (3 : annotation);  // describes the expression
x = 3 : annotation;    // describes the slot
```

Groups may chain at every attachment point. Text that resembles nested annotations inside a payload is still one opaque entry: `(42 : (Shape : Tag))` does not create an annotation on `Shape` in the surrounding program AST. A tool may choose to parse that entry later and own the resulting structure separately.

11.4. Semantics

Compilation hints, documentation, types, contracts and analyses belong to external tooling agreements. They cannot replace construction, lookup or capability rules. `(1 : class < super.Base {})`, `(1 : @unknown => syntax)` and `(1 : [=[unfinished (]=])` all execute as `1`. A tool's later decision to parse or execute an entry cannot retroactively change the original read result.

11.5. Metadata Retention

Surface and core artifacts retain raw entries and their groups. Core tools need no surface AST, parser object or source snapshot. Exact payload text is intrinsic metadata content; source locations and the enclosing document are optional provenance. Neither retaining nor erasing provenance changes payloads or execution.

An attachment describes a particular occurrence and role. Lowering preserves its identity, grouping, ordering and role when it removes a statement wrapper or rewrites a constructor target. Tools can find descriptors from surviving core elements and find described elements from descriptors. The runtime never reads this index. See inert core metadata for the implementation and explicit interpretation examples.

12. Foreign Semantic Values and the Alien/Expat Boundary

Foreign systems retain their semantics and execution mechanisms. An **Alien** makes a foreign value reachable inside VarioHyve; it supplies no native class, mixins or lexical outer. An **Expat** exposes native behavior to a foreign participant. Arguments, results, identity, failures and callbacks obey the same boundary contract in both directions, including language-visible VM services. Native lookup and receiver-origin semantics remain fixed.

The chapter follows values through selection, conversion and control transfer, then defines the VM-service and mirror capabilities. Operational rules are in the formal appendix.

12.1. Aliens — Foreign Semantic Values Inside VarioHyve

An Alien may denote an object, function, pointer, struct, resource handle, simulator model or remote endpoint. It need not support direct operations. Storing it in a slot does not make that slot's owner its lexical outer.

12.1.1. Alien Dispatch

After receiver and argument evaluation, an Alien send selects the adapter's exact `(name, arity)` operation. Native class/mixin lookup does not continue through the Alien. Selection must distinguish absence from failure, perform no operation or callback, and retain the selected operation for execution. A resolution or execution failure never restarts lookup or replays effects.

A foreign getter is not a generated native accessor: `alien.f(args)` does not read `f/0` to discover a callable. Write `alien.f.apply(args)` for that behavior. A stored Alien can qualify for native callable-binding invocation when its adapter advertises exact `apply/N`; the eligibility check's selected operation is reused. See Message sends.

Explicit Alien selectors `self`, `outer` and `super` use the foreign protocol. They do not navigate native scope. Native `self`, qualified `outer ClassName` and `super.message` keep their receiver-origin meanings; native declarations do not acquire those reserved selector names through this exception.

12.1.2. Arity Mismatch

An adapter defines how name/arity requests map to foreign behavior. Foreign methods, explicit capability operations and opaque handles need not share one object model. Overloads, defaults, ABI layout and lifetime are adapter concerns; automatic host reflection is not implied.

12.1.3. Alien Identity

Each adapter defines identity and lifetime for its foreign domain and preserves sharing on repeated compatible crossings. Numeric address equality alone is insufficient for pointer identity, provenance or reused allocations. Alienation does not synthesize an `eq` method. Concrete Java identity caching is an implementation policy, not a universal foreign identity law or an implemented C FFI.

12.2. Expats — VarioHyve Objects Outside VarioHyve

An Expat reenters native semantics through a behavioral interface. Its public contract exposes no store, address, activation, private initializer or VM representation. Adapters may privately retain what execution requires. The exact-send interfaces, `VHExpat.send` within an attached invocation and `ExpatReceipt.send` under an explicit retention receipt, include zero-argument access; they do not apply source invocation's callable-binding fallback. To use a stored callable, read the binding and send `apply/N` to the result explicitly.

12.2.1. Expat Surface

Without language access control, an Expat exposes the same source-visible behavior available to an external native caller, including generated accessors. Backend-private construction routines are not selectors. Proposed export control must preserve equality of the native external interface and the foreign export interface.

12.2.2. Expat Identity

A native object has at most one Expat proxy per compatible foreign runtime. Exporting it repeatedly through different paths returns the same proxy. Retention receipts are separate capabilities for this identity. Independent receipts may concern the same proxy; their own host identities are not the native object's identity. A proxy alone does not keep its native referent alive or authorize use after its invocation attachment expires.

12.3. The Symmetry

Direction	Arguments	Operation	Result
Native → Alien	Expatriate	Execute the selected foreign operation	Alienate
Foreign → Expat	Alienate	Exact native send	Expatriate

For compatible references with valid lifetime evidence, importing an owned Expat recovers the original native identity, and exporting an Alien to its originating adapter recovers its foreign identity. Cross-runtime references require an explicit bridge. Conversion by value instead follows a declared codec and may normalize host representations; it does not promise preservation of their object identity.

12.4. Operation Results

An Alien send returns the operation's ordinary result through the declared conversion protocol, just as a native send returns its method result. There is no automatic Alien-only `ok(value) | AlienError(error)` wrapper and no required sum result. Such an outcome protocol is one optional contract an operation may explicitly promise, equally for native and foreign implementations. The boundary does not infer failure from an ordinary returned object's shape, class name or truth value. Defined exceptional completions remain diagnostics under Error Propagation.

12.5. Primitive Marshalling

Numbers, strings, booleans and null may cross by value instead of through proxies. Each adapter must state its supported domains and reject unsupported conversions. A numeric encoding must not silently turn a pointer into an integer while claiming to preserve its provenance.

Canonical null and Boolean roles may hold native objects or Aliens. Assigning one of these roles adds neither methods nor a native class. The internal core null constant and empty expressions select canonical null directly; source names such as `null`, `true`, and `false` use ordinary message lookup.

Each world selects an explicit conversion profile. The Current Native Seed profile uses ordinary native objects and maps their exact identities to Java `null`, `Boolean.TRUE`, and `Boolean.FALSE`, without Expats. Incoming host scalars recover those canonical identities. Conversion never sends `toString`, `isNull`, `not`, or a truth-testing message to classify another object. Similar behavior or a source name does not make an object canonical.

The Host Seed instead preserves its foreign canonical payloads. A world may separately preserve numbers and strings as foreign values while using native canonical null and Booleans. Preserved values obey the originating adapter's identity laws; by-value codecs obey their declared value relation. Neither policy silently overrides the other. The supported Java subset is recorded in the implementation.

The startup installation contract requires a participating boundary to repair its managed references and retire provisional ownership before publication, consistently with the final codec. It cannot rewrite opaque foreign heaps or leave obsolete mappings that recover retired native addresses. This adds no live-replacement exception to published identity laws and supplies no general foreign garbage collector.

12.6. Explicit Wrappers

A native wrapper supplies its own identity, scope and behavior around foreign values. Multiple wrappers may share one Alien without sharing native identity. For example, a callable capability can operate on an opaque handle:

```
class Device(deviceHandle, reader) {
    handle = deviceHandle;
    readFn = reader;
    read(buffer) { readFn(handle, buffer) }
}
```

This is explicit adaptation, not a built-in calling convention. Persistent dependencies use slots under the ordinary constructor-scope rules.

12.7. Constructor Arguments and Optional Bundles

Aliens, native objects and bundles can be ordinary module constructor inputs. The same conversion contract applies in richer tooling contexts; it does not add capabilities to an isolated module root.

12.8. Control and Failure Agreements

12.8.1. Error Propagation

Selector absence is distinct from a selected operation's failure. Failures cross as defined diagnostics, never as retained host exception objects, classes or causes. The Java categories `FOREIGN_FAILURE`, `CONVERSION`, `OWNERSHIP` and `EXECUTION_POLICY` survive machine execution and callback propagation; native lookup/control errors use `LANGUAGE_FAILURE`.

Callbacks use ordinary native semantics and preserve state changes. A failed callback aborts its own native continuation, retires its invocation homes and reports a defined diagnostic to its foreign caller. It does not automatically fail the suspended native caller. Foreign code may handle the diagnostic, make further callbacks and return normally, or propagate a failure. The outer NF send observes the foreign operation's completion, subject to normal result conversion and execution limits. A propagated diagnostic preserves its kind; a later foreign failure is reported as that failure, not replaced by an earlier handled one.

The Java diagnostic also carries detached best-effort crossing evidence: direction (native to foreign, foreign to native, or host entry), phase, selector and arity, optional source site, and invocation progress. It distinguishes admission, selection, argument conversion, execution/callback and result conversion. Not dispatched describes this invocation only, not earlier argument effects. Completed records a normal invocation return even when result conversion then fails. A thrown foreign failure conservatively records completion unknown; these fields do not promise rollback, cancellation or safe retry.

The original failure kind, reason, source and semantic trace survive propagation. Each actually crossed boundary adds its own evidence, innermost first. Reusing the same crossing during propagation does not duplicate it; recursive calls with the same selector remain distinct. Crossing evidence uses the configured trace limit (five by default, or all), independently of semantic and private physical frames, and reports truncation. Handling a callback failure adds no evidence to later independent outcomes.

Selector absence remains a lookup result until lookup has finally failed; a foreign callable eligibility probe must not become fatal merely to produce a diagnostic. Explicit adapter rejection can carry `FOREIGN_REJECTION`; arbitrary exception text is never interpreted as rejection or as permission to retry.

Effects already performed are not rolled back. Handling a callback failure does not resume the aborted callback, revive its homes, replay the operation or grant NLR crossing. The suspended caller's homes remain live until its own completion or failure. Callbacks continue to share the active execution budget; handling an exhaustion diagnostic does

not replenish it. There are no native resumable conditions in the current language. See the accepted recovery agreement.

12.8.1.1. Java host exit limits

The adapter preserves a supplied `BoundaryException` and its defined kind. It translates other `RuntimeExceptions` thrown by `ForeignObject.resolve` into `FOREIGN_FAILURE` with a resolution reason. Effect-free lifetime metadata selection uses the same failure category and selection evidence, before argument conversion or invocation. Host scalar protocol resolution remains outside that translation catch. Before invocation, argument export annotates defined boundary failures without broadening its catch to other host failures. The existing invocation catch translates `Exception` from suspension, the selected operation, resumption and result import into `FOREIGN_FAILURE`; defined conversion/ownership/policy exceptions retain their original kind. Phase/progress evidence locates the translated failure without retaining its host exception object or cause.

JVM `Error` and native backend invariant failures outside these existing catches are not translated into ordinary language failures. No additional host-loss, cleanup or recovery contract is introduced here; abnormal host exit handling beyond today's adapter remains deferred.

12.8.2. Non-local Returns Across the Boundary

The boundary is the agreement between native and foreign participants. That agreement owns whether a non-local return may cross a foreign invocation and how the crossing is handled. The native language cannot prescribe how arbitrary foreign execution unwinds or otherwise transfers control.

The default agreement forbids crossing. A callback may return to an accessible home created inside that callback; an attempt to reach a live home across an active foreign-call barrier fails with `LANGUAGE_FAILURE`. The failure is reported to the foreign caller under the recovery agreement above. This is the only crossing agreement implemented by the current Java boundary.

A boundary may explicitly support crossing under a future agreement. It is responsible for implementing that agreement, including its observable control flow and treatment of intervening foreign work; the foreign-side mechanism is not fixed by VarioHyve. Every intervening boundary must support the requested transfer. Such support does not change the lexical target of `^`, revive a retired home, or bypass initializer phase restrictions. Crossing support and its transfer protocol are deferred; the default barrier must not be disabled without implementing a supporting agreement. See the accepted decision and examples.

12.8.3. Control Tokens and Execution Agreements

Ordinary native transitions require execution authority under an explicit execution agreement. **The current sequential agreement has one conceptual control token per live runtime.** It governs advancement of ordinary native continuations and their NF/FN control transfers. A suspended continuation has no independent authority to advance. The token

is neither a language value nor an additional capability that programs must pass; implementations need not reify it. A refinement argument must account for the authority using the implementation's actual synchronization and state.

The token may be idle at the boundary, assigned to a native continuation, or assigned to a foreign invocation. Independent FN entry acquires the existing idle token. NF transfers it to the selected foreign invocation; admitted callback entry transfers it to a new native continuation. Callback success or failure returns it to that invocation; foreign completion transfers it to the suspended native caller only after admitted descendants finish. Independent native completion returns it to the idle boundary. Nested calls retain suspended continuations and interaction state, not additional control tokens. Backend tooling may also yield after a requested step and later reacquire authority to advance its retained successor. Yielding is not native completion and does not retire homes; storing a configuration grants neither execution authority nor a snapshot of the mutable runtime.

This is an execution discipline, not exclusive physical access to all runtime memory. It permits three distinct forms of activity:

Activity	Authority and correctness obligation
Ordinary evaluation and callbacks	Execution authority under the active agreement; preserve continuations, admission and boundary transfers.
Representation maintenance and optimizations, including concurrent GC	Implementation refinement of the agreed semantics and observations; no second ordinary native continuation is implied by concurrent implementation work.
Explicit observation or intervention	A granted capability with a declared effect and coordination contract; it may intentionally change abstract program state without owning the ordinary control token.

An observation or intervention contract must specify its subjects and scope, permitted effects, consistency and coordination with execution, authority lifetime, and completion/failure behavior. Reading live state requires a defined consistency model. Changing a slot, pausing execution or replacing a continuation requires an explicit intervention contract; it cannot be justified as invisible maintenance. Such authority does not implicitly grant arbitrary evaluation, additional ordinary continuations, NLR crossing, dead-home revival or relaxed initializer rules. Language-visible VM services, debuggers and language monitors remain explicitly supplied Aliens using ordinary boundary sends. No general intervention API is implemented by this agreement. All agreements preserve fixed lookup, identity, return-home lifetime and initializer phase rules. NLR crossing still requires an explicitly supporting boundary agreement.

Implementations may use parallel work, concurrent collectors and boundary optimizations without a global representation lock. They must preserve logical identity, return-home liveness, roots and agreed observations, and coordinate their concrete data structures correctly. An optimization cannot silently replay foreign effects or change their specified ordering. Boundary bookkeeping, such as recording a pending request or an arriving completion, does not itself advance a native continuation.

A richer execution agreement may explicitly authorize simultaneous native transitions, including a model checker's request to execute proven independent transitions on a parallel VM. It must specify execution authority, shared-state and observation semantics, and applicable independence or synchronization conditions. Token uniqueness is a property of the sequential agreement, not a universal serialization requirement. The accepted extension principle and the remaining implementation work are separated in Concurrency and Parallelism.

12.8.4. Concurrency Boundary

The boundary supports synchronous same-thread callbacks and explicitly call-associated worker callbacks. Native execution remains serial over one runtime/store under the sequential agreement. Each NF invocation suspends and roots its native caller, transfers the conceptual control token, and records its call ancestry. An admitted FN callback acquires that authority, runs a separate native continuation, and shares the active budget. Nested NF calls repeat this protocol.

Foreign code captures a `CallbackContext` on its own active operation thread and may explicitly pass it to a worker. Its `send` requires an Expat from the same runtime with valid attachment or receipt evidence; the context grants permission to request admission, not receiver authority or possession of the control token. Contexts may be copied among workers; competing requests negotiate access to the unique token. Expats obtained before the NF invocation remain valid targets under open retention receipts. Unassociated entry on another thread during an active execution is rejected, as is callback entry during selection. Neither thread identity alone nor possession of an Expat grants admission to an outstanding foreign call.

Only callbacks associated with the innermost active foreign invocation may acquire native execution permission. Other associated requests wait for their turn without running native code. A descendant operation cannot synchronously use an ancestor context to wait on its own completion; that request is rejected. There is no FIFO or fairness guarantee. A future FIFO policy must respect call eligibility: a waiting ancestor request cannot block a needed descendant callback merely because it arrived first. Foreign completion closes admission, rejects queued or late requests and waits for already admitted descendants to finish before importing the result and resuming its native caller. A foreign result can arrive while a callback owns the token; arrival is recorded, and delivery waits for ownership transfer. Retained call contexts cannot attach to a later invocation. This preserves the existing NLR barriers and callback-local failure contract.

This protocol provides no preemption or termination guarantee: foreign work or an admitted descendant may never finish, and foreign dependencies may deadlock. Independent-entry scheduling, detached asynchronous native tasks and parallel native execution remain outside this contract. Execution limits belong to the implementation policy.

12.8.5. Lifecycle and Garbage Collection

The boundary registers **shared retention obligations**, independently of execution authority, mutation permission and resource validity. Several holders may retain the same referent. A holder releases its own obligation; it neither releases another holder's claim nor

closes a shared resource. Public raw addresses are not a substitute for a lifetime contract.

12.8.5.1. Invocation attachment

Expats exported as NF arguments are attached to that particular invocation by default. The attachment starts before exposure to foreign code and protects their native graphs until result conversion and all admitted descendants finish. Callback entry and return transfer the conceptual control token but do not end the enclosing invocation's attachments. Nested NF invocations register their own attachments independently. A live ancestor's attachment also protects use and transport within its descendant interaction tree; nested foreign helpers need not detach or re-export every alias merely to use it before the ancestor completes. This does not widen detachment permission: registration still cites an authorized argument or result of its owning invocation. Closing callback admission does not itself expire attachment or detachment authority: already admitted work may register authorized receipts while the invocation waits for its descendants and completes result conversion.

The selected operation declares which argument positions permit detachment. This metadata is available before argument conversion and operation execution; it does not cause automatic retention. An explicit detach registers an independent receipt while the authorized attachment is open, before publishing the retained reference. Permission belongs to an argument occurrence, including when several positions contain the same native object. Missing permission means attached-only. Undeclared or late detachment fails without creating a lasting root and without repeating selection or execution. Detachment also requires possession of the Expat being retained; naming an argument position through a callback admission context does not grant access to that argument. The boundary validates the supplied proxy against the occurrence.

An adapter may perform this registration on behalf of a foreign API that stores its arguments. The foreign implementation need not adopt VarioHyve's collector or a static ownership discipline. The declaration is a promise on which the boundary may act; a proof of foreign code is not required. Optional analysis or review evidence may support the promise, but grants no additional authority and does not replace runtime checks under this agreement.

12.8.5.2. Results, receipts and checked use

Native results from FN callbacks attach to the enclosing NF invocation. They may outlive the callback but need a separate declared callback-result permission and explicit detachment to outlive that invocation. Permission to detach a parent argument does not authorize detaching every object returned by its accessors. An independent host native result, including module construction, has no enclosing invocation: by-reference export delivers a host retention receipt. Public opaque boundary references likewise carry an explicit releasable hold. By-value codec results need no receipt; compatible Alien round trips follow the foreign domain's own retention rules.

A receipt supplies checked sending and transport for its canonical Expat. Closing it is idempotent and ends only that receipt's obligation. Acquiring an independent co-hold creates another receipt; copying one receipt shares the same obligation. Closing one co-hold leaves the others valid. The last release makes an otherwise unreachable native

graph eligible for collection, without requiring immediate collection.

Every mediated use and conversion validates the evidence supplied for that use. An expired attachment or closed receipt fails before native access even when another holder keeps the referent alive. Re-export does not revive a closed receipt. A bare canonical proxy is usable only under a currently applicable invocation attachment; outside that context it requires receipt evidence. Copies of a proxy used within the same valid attachment are ordinary aliases: the boundary cannot infer which foreign variable originally acquired that identity. Nor can it observe every hidden store into an opaque foreign heap. It diagnoses observable use without valid evidence, not necessarily the earlier hidden escape.

Lifetime violations use `OWNERSHIP`, independently of `EXECUTION_POLICY` for callback admission and `LANGUAGE_FAILURE` for retired return homes. Retention keeps captured data available; it never revives a return home, extends a `CallbackContext`, enables NLR crossing or replenishes an execution budget. Checked handles do not make arbitrary foreign raw-pointer operations safe.

Receipt release and admission coordinate so an already admitted use retains its target through completion; subsequent use of a closed receipt fails. Every handoff establishes the receiving root before dropping the sending protection, including an NF result that returns an attached argument unchanged. Partial argument conversion and defined exceptional completion close the invocation's own attachments. A failed callback does not close its enclosing invocation, and an outer failure does not revoke detached receipts already published by foreign code. Such retention is an effect, not a transaction to roll back. Broader host termination remains subject to the host exit limits.

12.8.5.3. Alien holds and cycles

An imported Alien establishes an adapter hold before publication. Native GC traces references to it normally and relinquishes that hold when the Alien becomes unreachable. Slots, captures and returns need no native detachment annotation. The adapter counts obligations between participants, not every native alias. Java may realize a hold as a strong payload reference; a foreign reference counted domain may use its own retain/release protocol. Expiring resources and temporary pointers still need an appropriate domain contract: native reachability cannot prolong an allocation that the foreign participant cannot retain.

Identity caches are non-owning indexes. A live observable proxy remains canonical without making its expired native graph a root. Reclamation and reimport must preserve the foreign referent's identity and identity hash while that identity remains live, including distinctions between restricted capability views. Expired receipts and diagnostic records must not retain their former targets. Releasing a hold is distinct from globally closing a resource. Observable foreign cleanup requires the adapter's safe cleanup agreement; it must not run arbitrary foreign code inside native tracing. GC makes no prompt resource-cleanup promise.

Cross-boundary cycles are legal by default. Without collector cooperation the boundary conservatively retains cycles that neither participant can prove unreachable. Every retained strong edge still obeys the attachment/receipt contract; allowing cycles does not authorize unregistered escape. An explicit release can break a cycle, but strong relationships must not silently become weak to obtain collection.

An optional future agreement may distinguish foreign external roots from edges whose

retaining owners are traceable, preserving opaque edge identities and provenance. Safe collection requires coordinated reachability and mutation observations, validation before reclamation, and agreed weak-reference, finalization and participant-loss behavior. Missing or incomplete cooperation retains conservatively. No universal foreign collector, introspection mechanism or cycle-collection algorithm is prescribed. See the accepted lifetime decision.

12.8.6. Foreign Kinds and Capability Contracts

An adapter may classify a value as callable, pointer, struct or endpoint; these are foreign contracts, not native classes. Analysis needs explicit assumptions about effects, callbacks, blocking, determinism and state changes. Unknown behavior stays opaque. General contract metadata, provenance registries and foreign reflection remain unimplemented; see the boundary assessment.

12.9. VM Services and the Trusted Seed

The host boundary has a public foreign facet and a privileged VM-service facet over one private owner. The public facet exposes only the ordinary transport, explicit mirror grant and call-associated callback admission. Trusted seed provisioning supplies narrow VM service capabilities; public clients have no operation for obtaining the provisioning facet or raw runtime representations. Object's identity service is provisioned only to its definition scope. The optional public `IdentityMirror` grants its own limited observations.

Both facets share identity maps, conversion, execution admission, budgets and GC roots. Current services use the same serial admission protocol. Privilege changes authority, not lookup, return or failure semantics; privilege alone grants no additional ordinary native execution. A future independently operating service requires an explicit execution or intervention agreement. The same capability view preserves identity across either route. Distinct restricted views have distinct identities even when they concern one resource. Deliberately exporting a privileged capability delegates that capability; a partition label does not silently attenuate it. No new privilege registry, selector allowlist or native invocation path is introduced.

Language-visible VM services use this same boundary; there is no privileged native-function invocation path. The trusted backend may privately allocate values, maintain stores and implement lookup/continuations. Those internal operations are not language services.

A service returning a native object must return its Expat with valid attachment or receipt evidence, so import recovers the object. Returning its raw backend representation as a foreign value is invalid. Any future representation-sensitive service requires explicit authority and private unwrapping without adding raw-world access to public Expats. See the VM boundary decision.

12.10. IdentityMirror

`IdentityMirror` is an explicitly supplied capability for observations that must not dispatch through the subject. A host obtains it with `runtime.boundary().identityMirror()`

and can pass it as an ordinary module constructor argument. It is not installed in the kernel or exposed by Object. The Java public interface uses the same `ForeignObject` selector protocol as other services; it grants no store, address or runtime access.

Operation	Result
`same(a, b)`	Whether supported values have the same logical identity
`identityHash(a)`	Integer stable for that identity's lifetime
`kind(a, name)`	Membership in the semantic kind named by a string
`isAlien(a)`	`kind(a, "Alien")`
`isNative(a)`	`kind(a, "Object")`
`instanceOf(a, C)`	Whether the actual native class/superclass chain contains class C by identity
`applies(a, C)`	Whether the native contribution chain contains C's owned mixin by identity

These operations execute no subject or target method, foreign resolver, computed getter or member-class realization. Their arguments are first evaluated using ordinary message-send rules, which may themselves realize a member class. All explicit sends on an Alien retain their foreign meaning, even for these same selector names. Object's ordinary equality and hash defaults are normative for the current runtime seed and remain available without granting the mirror itself. Another seed may choose different root defaults.

12.10.1. Identity and kinds

Identity comparison is reflexive, symmetric and transitive on its supported domain. It observes native identity or adapter-preserved Alien identity, never semantic equality or remote resource equivalence. Compatible round trips preserve identity and hashes; same identities hash equally, but collisions are permitted. Hashes are neither addresses nor stable across sessions. A wrapper and its payload differ; differently restricted handles must not be merged merely because they refer to the same resource. Ownership failures require an explicit bridge.

The supported kind labels are exactly `"Object"` and `"Alien"`, case sensitive. Object includes native instances, class objects, metaclasses, and lambdas. Native canonical null and Booleans belong to Object; canonical values implemented as Aliens belong to Alien. Canonical roles do not override representation kind. Alien also includes opaque foreign references, preserved host scalars, and VM services exposed through the adapter. It does not imply callability or trust. Classification is relative to the observing runtime; importing its owned Expat recovers a native object.

Current integer/string codec values belong to neither kind and have no native class or mixin membership. Identity operations on them, including mixed `same` queries, fail with `CONVERSION`; interning does not specify scalar identity. Preserved foreign scalars instead use their adapter's identity and Alien kind; see Possible Seeds and Worlds. Thus `isNative` is not implemented as the negation of `isAlien`. Object's default `equals` with a codec scalar still returns false because its receiver is native. An unknown kind label fails with `LANGUAGE_FAILURE`, a non-string kind label with `CONVERSION`, and a missing selector/arity follows ordinary lookup failure.

12.10.2. Class and contribution references

The second argument of both membership queries is a native class reference, obtained through ordinary messages or explicit capability provision. For an ordinary class, it is the same definition object usable in superclass and mixin applications. `instanceOf` uses that class's identity; `applies` selects that class's owned reified mixin, just as `<: C` does. It does not send an `ownedMixin` accessor or require a separately exposed mixin token. The query does not resolve a name or consult a class registry. Names and hashes, including old anonymous labels, are not targets.

`instanceOf` starts at the receiver's actual class and follows superclasses through root Object, inclusive. It returns at the first matching class identity. It does not treat a mixin application's source class as a superclass.

`applies` walks the same linear contribution order used for native dispatch: the class's owned contribution, its direct applications left to right, then its superclass's owned contribution and direct applications, continuing through Object inclusive. It returns true at the first match and false after the chain is exhausted. Matching compares the reified contribution's identity, not its syntax, name or semantic equality. An application contributes only the selected source's owned mixin; this query does not traverse that source's superclass or applications. Owned, inherited and repeated contributions are included without collapsing occurrence identities or executing construction routes.

Anonymous and named definitions follow identical identity rules. Distinct class evaluations and distinct per-enclosing-receiver member realizations remain distinct even when they share a declaration or name. Aliases to the same class select the same identity and owned mixin. No identity hash participates in membership, so hash collisions cannot change an answer.

For a class object, queries inspect its metaclass hierarchy and class-side contributions; they never switch to the represented class's instance hierarchy. The target rule remains the same: C denotes C itself or C's owned instance-side mixin, not its metaclass's contribution. In particular, `instanceOf(C, C)` and `applies(C, C)` are normally false. If a metaclass reference is explicitly supplied, it can select that metaclass or its owned class-side contribution. This does not add a public class/metaclass getter.

Both arguments undergo ordinary boundary ownership validation. A non-class target, including a string, canonical null, ordinary instance or foreign object, fails with `CONVERSION`; an unowned class fails with `OWNERSHIP`. Invalid targets fail even when the subject is an Alien or codec value. With a valid target, Aliens and codec values return false because they have no native membership.

These are structural identity tests, not proofs that a selector is supported or that a contribution's behavior remains effective after overriding. They do not establish trust in a subclass or authorize its behavior. The design note records the routing and capability tradeoffs.

Part III.

Possible Seeds and Worlds

13. Possible Seeds and Worlds

A seed is an object graph providing capabilities; its library contents are deliberately independent of the language definition. The VM specifies structural roles and startup invariants, not a mandatory library namespace or collection of primitive methods. A seed chooses the objects that occupy the VM's canonical roles and the behavior of its structural root. A world also chooses boundary conversion and the capabilities explicitly supplied to programs. These choices do not change lookup, lexical scope, construction, or the native rules for return homes.

The VM sends **no required selectors** to canonical null, true, or false. It supplies canonical null directly for empty expressions and omitted initializers; Boolean identities are used by the selected boundary conversion policy. Nor does the VM require any instance methods on root Object. `null/0`, `true/0`, `false/0`, and `Object/0` are the optional seed *provider's* discovery protocol, not methods required on the returned values.

Every native world retains classes, objects, activations, closures, and message dispatch even when all its data values are foreign. Root Object is a native class with **no superclass and no instance state**, whether owned or contributed by applied mixins. Its absent superclass represents semantic Top, not canonical null. Zero-parameter, stateless mixin contributions may supply behavior. Root initialization bodies and routed argument effects are unsupported; support for stateful roots is indefinitely deferred.

Installation finishes before publication under the atomic startup contract. It rewrites all retained references to retired seed identities and forces native GC. Selecting a new seed during application execution is unsupported. Installing a seed does not introduce ambient bindings into an isolated module.

The following chapters describe the Host Seed, one possible native seed, and a native-seed world with host scalars and strings. These are concrete choices, not an exhaustive taxonomy of valid worlds.

14. The Host Seed

The minimal host seed binds canonical null, true, and false to boundary-owned foreign values and installs an Object with an empty instance protocol. The Java host realization preserves the corresponding host payloads as Aliens. Those values need no `isNull`, conditional, equality, hashing, or display methods for the machine to execute. A client send succeeds only when the foreign adapter provides its selector; assigning a canonical role supplies no additional behavior or native superclass.

The supplied Java adapter offers null tests/display and Boolean selection methods matching the native library's selector inventory. These are optional adapter capabilities, not minimum seed requirements. In the unconfigured Host Seed, numbers and strings remain opaque unless an explicitly supplied foreign capability operates on them. The optional JavaHostKernel world adds receiver protocols such as `2.plus(3)` through a host adapter chosen before runtime publication.

The host seed can be used without loading a native library. Explicitly supplied foreign capabilities perform work, while native objects and closures coordinate them. Foreign scalar preservation avoids native data conversion; native coordination objects still use Expats when passed to a host capability. A preserved Java value does not acquire automatic Java reflection or a native number/string protocol.

`BootstrapRuntime(capabilities, BoundaryProfile.HOST_VALUES, adapter)` selects an explicit `HostValueAdapter` for preserved values. This is independent of the kernel bundle passed to a module: passing a bundle cannot install or change the runtime adapter. Its foreign selector resolution obeys the same no-effects, no-callback selection contract as `ForeignObject.resolve`; selected operations execute through the ordinary boundary. Known null/Boolean operations retain precedence, and explicit `ForeignObject` receivers retain their own resolver. The optional adapter adds no native class, slot, implicit namespace or I/O grant.

Host Boolean control behavior is a boundary agreement. If a host Boolean adapter executes a native thunk, that execution crosses the Alien/Expat boundary. A non-local return from the thunk can cross this call only when every intervening boundary implements the required transfer. A supporting host and boundary can permit it; foreign Boolean representation is not intrinsically a limitation. The current Java implementation forbids this crossing and reports `LANGUAGE_FAILURE`. Ordinary callback returns and returns to homes created inside the callback remain possible. See non-local returns across the boundary.

The Java bootstrap selects this world explicitly:

```
var runtime = new BootstrapRuntime(Map.of(), BoundaryProfile.HOST_VALUES);
Object result = runtime.evaluate("true.ifTrue(fun { 7 })", 1_000);
```

The tooling bundle supplies `true` for this expression. An isolated module must receive its capabilities as constructor arguments, even in the Host Seed.

15. JavaHostKernel world

JavaHostKernel is an optional Java-backed library over the Host Seed. The starter selects `HOST_VALUES` and an explicit `HostValueAdapter` before publishing its runtime. Its kernel bundle, factories, collections and data values are Aliens. Receiver sends such as `2.plus(3)` use foreign selector/arity dispatch; they do not introduce native Integer classes or alter native lookup.

Normal modules retain their empty top activation. The host explicitly passes the kernel to `class Program(kernel)` and then sends `run/0`. This is the starter's convention, not a required language entry selector. Literal receiver behavior comes from the selected world; named factories and Console come from explicit inputs. Iteration callbacks can use their own captured capabilities. Numeric operations grant no console, filesystem or other ambient authority.

15.1. Receiver adapter

`HostValueAdapter.resolve(receiver, selector, arity)` selects an operation over the original host payload. Selection must have no effects or callbacks and may return null only for a genuine miss. Selected failures never retry another adapter. Configured resolution receives the same reentry guards and detached failure translation as ordinary `ForeignObject` resolution. Such objects keep their own resolver; built-in null/Boolean selectors retain precedence.

The adapter is supplied at runtime construction, with no public replacement or global registry. Its decimal-literal support flag is captured once. Foreign payload identity and compatible round trips are unchanged; no value interning or extra language wrapper is used to attach behavior. Arithmetic results are ordinary host results and need not share the receiver's identity. Value equality does not establish IdentityMirror identity.

15.2. Numbers and strings

Integral literals retain the signed 64-bit domain. The configured world accepts decimal syntax as Java binary64 (`Double`), with Java rounding and underflow; literal overflow to infinity fails explicitly. Exponent syntax remains absent. Unconfigured worlds still reject decimal literals. `HOST_SCALARS` may use the same adapter while retaining native canonical values; `NATIVE_VALUES` rejects optional host receiver adapters and retains its existing codec.

Integer receivers provide `plus`, `minus`, `times`, `quotient`, `remainder`, comparison, `asDouble` and `toString`. Arithmetic uses Java signed 64-bit wraparound, division truncates toward zero, and division by zero fails. Byte, Short, Integer and Long payloads retain their imported identities; operations read their integral values and arithmetic returns Long results. Mixed integral/floating arithmetic requires explicit conversion.

Double/Float receivers provide `plus`, `minus`, `times`, `dividedBy`, comparison, `abs` and `toString`. Floating operations use binary64, including IEEE special results. String receivers provide concatenation, UTF-16 length, equality, trim and literal-delimiter split. No universal native Object display/equality method is implied. The API declarations and conformance tests define the exact selectors and input validation of each protocol.

Kernel factories supply parsing and mutable collections. Lists use zero-based indices; maps use string keys and insertion order. Elements preserve boundary identity. Collection iteration is sequential and visits a snapshot; integer iteration fixes its bounds before calling back. Both invoke native callbacks through Expats, preserving shared budgets and the foreign return barrier. List `add`/`set` and Map `put` declare the stored-value argument detachable and register a receipt before retaining a native object. Replacement releases only the collection's previous receipt. `get` returns the stored reference through ordinary conversion, preserving identity and the collection's hold. Snapshot iteration acquires independent co-holds for its native elements and releases them on normal or failed completion, so mutations during callbacks do not invalidate the remaining snapshot.

List and Map `close/0` idempotently release their native receipts and empty the collection; it returns host null. Further collection operations fail as foreign rejection. Closing a collection never closes unrelated foreign elements. The Java collection constructor likewise acquires independent co-holds for receipt inputs and rejects bare Expats lacking durable evidence. These are adapter retention obligations under the boundary agreement, not native ownership annotations or a universal collection protocol.

Console uses explicitly injected streams; line input returns host null at EOF.

15.3. API declarations

`JavaHostKernel.vh` is a module-like API document. Class-shaped declarations describe receiver protocols; method bodies containing `?` denote absent VarioHyve implementations. Reaching such a hole still fails normally. `_value` documents host state and supplies no slot or exported accessor on the real Alien. If someone instantiates the API sketch as source, its declaration has ordinary native slot semantics instead; that object is not the host value. An underscore is not access control.

Descriptors document host adapter entry points, help text and results. Tools read known literal metadata structurally; they do not evaluate it. Removing descriptors preserves raw execution, including unresolved-hole failures. Neither descriptors nor declarations install adapters, import capabilities or choose dispatch. Java implementations and the API document are checked together and shipped at the same version. This is a concrete tooling convention, not a general binding generator or interface-definition extension to the language.

16. One Possible Native Seed

The repository's Current Native Seed module is one library choice. It is an ordinary isolated module whose sole constructor input is a narrow host identity service. The module stores that service in its `_vm` slot because constructor parameters are not method bindings. Its members and ordinary object literals implement the native canonical values and root behavior. The provider module has state; the Object it supplies has none.

The builder sends `null/0`, `true/0`, `false/0`, then `Object/0`, once each. The returned Object class initially inherits the provisional structural root. After validation, adoption removes that superclass edge, redirects the provisional root and scalar references throughout the retained graph, retires obsolete bootstrap ownership, and collects. The provider's Object is then the actual root; its methods keep their original definition scope. No duplicate ancestor Object or forwarding wrapper is retained.

This module supplies the following protocols:

Provider	Selectors	Behavior
Null	`isNull/0`, `notNull/0`, `toString/0`	Returns canonical true, canonical false, and `"null"`, respectively.
True and false	`and/1`, `or/1`, `not/0`	Select an already evaluated argument or canonical Boolean according to the receiver.
True and false	`andThen/1`, `orElse/1`	Invoke the selected thunk with `apply/0`; otherwise return the receiver.
True and false	`ifTrue/1`, `ifFalse/1`	Invoke the selected thunk once; the unselected case returns canonical null.
True and false	`ifTrue/2`, `ifFalse/2`	Invoke exactly one of the two thunks with `apply/0`.
True and false	`isNull/0`, `notNull/0`, `toString/0`	Return false, true, and the receiver's Boolean spelling.
Object	`equals/1`, `hash/0`, `toString/0`	Identity equality, an identity hash, and `"an Object"`.

Null and Booleans inherit Object's methods. Object delegates identity operations to the supplied service over the ordinary Alien/Expat boundary. Its methods are overridable client defaults, not VM instructions; independent observations use an explicitly supplied IdentityMirror. No number, string, or collection class is supplied by this module.

Boolean branch methods execute as native methods. A thunk that returns to an accessible native home does not cross foreign execution merely because its receiver is a canonical

Boolean. If that thunk or its caller invokes foreign code, the boundary's return-transfer agreement still applies. The current Java agreement forbids non-local return through an active foreign call; using this native seed does not disable that barrier.

With the compatibility Java codec, these three native identities cross as host `null`, `Boolean.TRUE`, and `Boolean.FALSE`; incoming host values recover the same native identities. Classification uses exact identities, without sending methods or recognizing names. Supported integral values and strings use the existing by-value transport. This conversion profile is a world choice, not a requirement imposed on all possible seeds.

Number/string transport is selected before the provider executes and stays consistent across adoption. Canonical null/Boolean conversion becomes active after the replacement commits, so bootstrap provisioning does not depend on the native roles already existing.

The source module records this library's contract. Another provider may offer other methods or return Aliens for the scalar roles, provided it satisfies the structural and atomic-installation invariants. Missing library methods fail when sent; the machine does not preflight a Boolean or null protocol.

To load the checked source module through the public Java bootstrap API:

```
String source =
↪  Files.readString(Path.of("vh-source/bootstrap/current-native-seed.vh"));
var runtime = BootstrapRuntime.fromNativeSeedModule(
        source, BoundaryProfile.NATIVE_VALUES, Map.of());
Object result = runtime.evaluate("false.not", 1_000); // Java Boolean.TRUE
```

The convenience entry explicitly supplies the module's narrow identity service. For a different provider ABI, `fromSeedModule(source, profile, capabilities, inputs...)` supplies exactly the listed constructor inputs. Tooling capabilities are assembled after seed installation, so they are not ambient module inputs.

17. A World with Native Seed but Host Scalars and Strings

A world may install the same native null, Booleans, and Object while preserving host numeric scalars and strings as Aliens. Native control protocols are then available without assigning native classes or by-value wrapper semantics to numbers and strings. Canonical null and Boolean conversion remains explicit and independent of numeric/string preservation.

Supported number/string source literals use the configured Java import path. With host preservation they produce foreign Java values; Java integral literals remain limited by the parser's supported signed 64-bit domain. Preservation does not add arbitrary precision, Python literals, native arithmetic, or a reflected Java method interface. Programs use explicitly supplied foreign operations for those values, and repeated compatible boundary crossings preserve foreign identity instead of normalizing it by value.

This world can coordinate Java data directly while retaining native conditional control and Object defaults. Additional foreign domains require explicit ownership and translation agreements. Java Booleans and Python integers could coexist in a future multi-domain world, but this choice alone provides neither a Python adapter nor a cross-domain conversion bridge.

The public Java bootstrap selects this combination with `new BootstrapRuntime(capabilities, BoundaryProfile.HOST_SCALARS)`. To load the native source provider itself, pass `BoundaryProfile.HOST_SCALARS` to `BootstrapRuntime.fromNativeSeedModule`. The default `NATIVE_VALUES` profile instead retains the compatibility integer/string codec.

The public constructor also accepts an explicit `HostValueAdapter` as its third argument. Selecting JavaHostKernel's adapter enables its numeric/string receiver protocols and finite binary64 decimal literals while retaining native canonical null/Booleans. Omitting the adapter preserves the existing opaque scalar behavior. The native codec profile rejects this adapter configuration. See JavaHostKernel for the optional library contract.

Part IV.

Implementation

18. Runtime Object Representation

This implementation chapter describes `VisitorSemanticsVM`'s records and links. The language contracts are in Objects and Classes, Slots, Name Resolution, and Message Sends. These Java structures are backend-private; none is a required representation for foreign clients.

18.1. Core Structures

The following sketches abbreviate the current value types:

```
VMObject       = { ValueRef[] slots, slotLayout, classRef?,
↪  lexicalParentRef?, memberCache }
VMMixinClosure = { definitionScope, declarationName, mixinSyntax,
↪  parameters, initializer }
VMClass        = VMObject + { name, definition }
ClassDefinition = { complete structural snapshot }
snapshot       = { mixins, superclassRef, initializationEdges,
↪  instanceLayout, initializer }
VMMetaclass    = VMClass
```

`VMObject` has no per-instance method dictionary. Reified contributions retain method syntax and declarations; ObjectProtocol selects their operations. `VMClass.ownedContribution()` names `mixins[0]`; later entries are direct applications. Class objects currently have empty value-slot arrays: instance declarations belong to their contributions, and instance values belong to allocated receivers.

`VMMixinClosure` and method closures remain dedicated `VMValue` types. `VMLambdaClosure` is a `VMObject` with the shared native Lambda class and private function/captured-scope/receiver/origin/return-home fields. Its ordinary Object methods use the closure as receiver; intrinsic `apply/N` uses its lexical captures. No captured slots, internal closures or activations become public. Activations reuse `VMObject` storage but have no class reference. A missing Java reference is distinct from the language's canonical null object. Alien and scalar nodes are described under Java boundary representation.

Applying a reified contribution reuses its identity without importing the source class's construction or inheritance chain. Mirror membership selects the supplied class's owned contribution and compares that identity; it requires no naming metadata or reverse link to the owning class.

18.1.1. Store Entries

Evaluating a class expression creates a class/metaclass pair. A member accessor creates that pair on first successful realization. Construction creates its receiver; each object-literal

evaluation realizes a fresh anonymous class/metaclass pair with its selected contributors and sends primary `new/0` to create the receiver. These are object-graph counts, not total backend allocations: contributions, activations and continuations also occupy representation space.

`VMObject.MemberClassKey` contains applying class, contribution index and member declaration. The cache lives on the enclosing receiver, completing the semantic key. Each stable `MemberClassBinding` owns one private disjoint state: unrealized, realizing, realized(class reference), or failed(error). Its guarded `begin`, `complete` and `failPending` operations implement the formal cache transitions. `ClassTargetFrame` sequences target computations; `MemberClassReturnFrame` reports completion to the binding. Failure and unwind report abandonment; repeated cleanup retains the first failure rather than retrying realization. These transitions cover supported language failures and non-local abandonment; raw Java host/JVM exits remain subject to the host exit limits.

Observing a realizing binding reports a cycle without independently failing the original attempt. A foreign handler may recover from a reentrant callback failure while the suspended original continues. Only abandonment of that original realization caches its failure. Failure diagnostics remain detached data; later observation adds context without changing the retained diagnostic. Startup relocation prepares a replacement successful payload while preserving binding identity and the cache key's full declaration/application origin. See the lifecycle decision.

18.2. Contribution Occurrences and Layouts

The backend identifies an occurrence by `(applyingClass, mixinIndex)` and a logical slot by adding its contribution-local position. Inherited occurrences retain their own applying-class identities. Direct applications hold the selected `VMMixinClosure`, without importing the source class's construction record.

InstanceLayout collects owned/direct/inherited contributions and interns physical positions by slot name. Its immutable mapping is cached on the realized class. Instances allocate fresh arrays from its physical declarations. Logical identities survive name sharing; physical index order is an implementation choice.

```
selected slot: (applyingClass, mixinIndex, localSlot)
physicalIndex = actualReceiverClass.instanceLayout.physicalIndex(selected
↪  slot)
```

Accessor mutability and eligibility come from the selected contribution, not from the declaration chosen to represent the physical cell. Initialization uses the same mapping. Ordinary source sends still pass through lookup; only an already selected accessor or private initializer write uses a direct array operation. See the accepted layout decision.

18.3. Class Assembly

Private initialization routes distinguish a complete superclass coordinator from an applied owned contribution. Named route constructors validate the target and occurrence;

consumers use that distinction directly. A mixin route retains its positive application index, while superclass entry uses coordinator index zero. The representation's superclass sentinel is confined to the route owner.

ClassConstruction resolves contributor targets, reserves identities, creates each distinct metaclass and installs owned factories. ClassDefinition derives a complete structural snapshot from the ordered contributions and routes before the class is installed. Its immutable lists, layout and coordinator agree; `VMClass` exposes no independent setters or mutable contribution list. A definition owner belongs to one class identity. `VisitorSemanticsVM` retains backend entry points that delegate trusted seeding and anonymous assembly to it. `seedClassGraph` installs the six root identities implementing the flat graph, plus Lambda and its metaclass. Lambda extends Object and has no public factories. The metaclass has no additional constructor arity, argument routes or initializer fields. The factory contracts specify their public selectors.

Flat metaclasses use an explicit definition with no construction coordinator; this is distinct from an unfinished ordinary class. Identity reservation preserves bootstrap class-of cycles without publishing partially assembled class metadata. Assembly remains inside one machine transition, with no collector safepoint or language execution inside finalization. Context-aware allocation remains intact. These are trusted backend assembly APIs: correct class/metaclass handle pairing and whole-graph validity remain assembler responsibilities.

Exclusive startup replacement is a separate lifecycle. It prepares root-edge removal, complete layouts and replacement snapshots before rewriting the graph. `NativeRelocation` commits whole snapshots through package-private definition operations while preserving class and definition-owner identities. Normal finalization and startup use the same contribution-layout derivation; startup additionally checks inheritance cycles and unchanged physical storage. Generated code and captured references still undergo the complete graph repair required by startup installation. These structural snapshots are not persistent stores or deeply immutable syntax.

Finalization validates route identity/order and instance-contribution kinds. Omitting a superclass initialization route is permitted only when its complete chain has no parameters, slots, post-initializer bodies or routed argument expressions. This covers the implicit stateless root and primordial structural classes without silently omitting superclass work in backend assembly.

18.3.1. Representing Top without Reification

The specification's `Top` is the empty constructor of the class algebra. `VisitorSemanticsVM` deliberately does not allocate a `VMClass`, metaclass, mixin or handle for it. The implementation refines the specification with this field-specific representation:

```
decodeSuperclass(Object, absent Java reference) = Top
decodeSuperclass(C, reference to S)             = S
```

Only Object's superclass may use the absent case in a well-formed class graph. An absent lexical parent, receiver or class link has its own meaning; it does not denote `Top`. Neither Java absence nor `Top` is the canonical language null. The spec's `super(Object) = Top`

and the VM's `Object.superclassRef() == null` therefore describe the same terminal structure at different representation levels.

The correspondence is local and concrete:

Semantic empty case	Current implementation
`lin(Top) = []`; exact/invocation search over it misses	`ObjectProtocol` stops when the superclass reference is absent, after searching Object's owned and directly applied contributions. No extra operation is selected or executed.
`Top` has an empty layout and no contribution	Class-definition derivation uses an empty inherited layout when the superclass reference is absent; Object retains its owned and stateless directly applied mixins.
No initialization dependency on `Top`	Object has no superclass initialization edge. No terminal initializer, activation, return home or argument evaluation is introduced.

No current source or boundary operation observes a `Top` object. Reification would extend the value contract and is justified only by an accepted feature requiring such a first-class reference. Its exposure, identity, class-of and construction rules would then need specification. The Top decision records that criterion and the rationale for the present representation.

ClassConstructionChecks asserts the absent root superclass and flat metaclass graph. NativeSeedSourceTest checks the corresponding source recipe. These are implementation evidence; the general Java simulation proof remains an obligation.

18.3.2. Private Construction Code

ConstructionCode generates `CoreUserFunction` bodies for ordinary primary `new`, default `apply`, owned initializers and class coordinators. Coordinators sequence dependency calls using `CoreLet`, then evaluate their owned body directly in the existing initializer home. The language schedule and argument binding are specified in Construction Order.

Private operation	Required backend work
`Allocate`	Allocate one receiver using a bound class's cached layout.
`InvokeInitializer`	Evaluate receiver and arguments through frames; enter a bound complete coordinator or owned contribution with its occurrence and fresh home.

Private operation	Required backend work
`WriteSlot`	Resolve the active initializer's occurrence-local destination through the actual receiver layout before evaluating the value; retain the destination in a write frame.
`SlotsComplete`	Enable the post-slot phase after the final write has completed.

These operations have no selectors. `new` itself uses ordinary method lookup; no privileged construction lookup target is required. The construction decision records why the four operations are necessary.

18.4. Method Activations

`MethodActivation` records immutable parameter slots, the selected `VMMethodClosure`, receiver, caller continuation and a fresh return-home reference. Its lexical parent is a `LexicalObjectScope`, pairing the receiver with its selected contribution and linking to that contribution's definition scope:

```
method activation
  → lexical receiver scope(receiver, applyingClass, contributionIndex,
↪  definition)
    → definition.definitionScope
```

`InitializerActivation` likewise has constructor parameter slots and a fresh home, plus slot/body phase state. Instances never acquire an automatic link to their own constructor activation. A lowered `CoreLetExpression` creates a separate activation, reserves all locals first, and inherits receiver and home metadata.

18.4.1. Lambda Activations

`VMLambdaClosure` retains its function, captured scope, receiver, applying class and return target. Invocation allocates parameter slots over the captured scope and restores the caller afterward, without a fresh home. Let/method/initializer activations can remain captured after their homes retire. Native tracing retains their reachable data without reviving return authority. The first collector conservatively also traces activation continuation links, which can retain old caller graphs; see collection trade-offs.

18.5. Dispatch Origin Tracking

For lexical lookup and `super`, `VMMethodClosure` retains `(applyingClass, contributionIndex)` separately from definition scope. `MethodActivation.methodOrigin` and `LexicalObjectScope` preserve that selection. Let/lambda activations follow lexical links; initializers use their own occurrence. Origins are neither inferred from a mixin's source class nor copied to every activation.

19. Java Interpreter

`VisitorSemanticsVM` is the executable prototype. The older `VM` is not a semantic authority. This chapter records how Java realizes the language contracts and formal reference machine, including implementation restrictions. It does not claim a completed refinement proof.

19.1. Machine Components

The CESK-inspired configuration is `Configuration(control, environmentRef, runtimeWorld, continuationRef)`. The scope reference identifies an activation or lexical receiver view; `RuntimeWorld` supplies the store, boundary and seeded identities. There is no separate environment namespace.

Reference component	Java representation
Value `v`, store `σ`	Typed `ValueRef` keys and mutable `VMValue` entries in `ConcreteStore`.
Scope `ρ`	`Activation` or `LexicalObjectScope`, with generated accessor storage and declaration/application metadata.
Continuation `κ`	Linked `ContinuationFrame` entries addressed by `ContinuationRef`.
Return homes `H`	Activation addresses also identify homes; `ReturnHomeActivation` stores liveness and `InitializerActivation` stores phase.
Boundary `B`	`DefaultJavaBoundary` identity maps and suspended synchronous foreign calls.
Sequential control authority	Conceptual ownership is reconstructed from boundary admission and suspended-call state; there is no reified control-token value.
Failure control	`Control.Error` terminates the current native continuation; callback failure is exported to its foreign caller. No execution-wide failure marker is retained.

`Control` distinguishes expression evaluation, exact host send, committed foreign call, returned value and error. EvaluateStepVisitor handles expressions; ContinuationStepVisitor handles completion; ObjectProtocol selects and invokes operations. Its ordered loops, `getterSeen`, `Selection` and `EligibleAccessor` retain lookup evidence without repeating reads or selection. The language algorithm is specified in Message Sends.

The step API returns a set, but the concrete driver requires exactly one successor for each nonterminal native configuration. It stops at an error or a value under `HaltFrame`. Ob-

servable determinism additionally depends on supplied foreign behavior. A set-valued interface does not implement abstract execution. Retained or prepared configurations do not each hold execution authority. All ordinary entry and stepping paths for one runtime obey the same sequential control agreement. `step` acquires admission for its selected configuration, advances one transition and yields authority back to the backend driver. A nonterminal successor retains its ordinary home state and needs admission again before `step` or `run` can advance it; yielding does not mean final delivery.

Public `inject` prepares an unexecuted tooling configuration under this implementation's execution guard, because allocation mutates the serial concrete store. Unassociated entry during an active execution fails before allocation. This guard is coordination for the current representation, not evidence that preparing a configuration is itself an ordinary native transition. `evaluate` keeps preparation, execution and export inside one guard and budget. Raw backend helpers remain subject to caller coordination; this is not a generally thread-safe backend API.

19.1.1. Addresses and Allocation

`Store` separates `Value(VMValue)` and `Continuation(ContinuationFrame)` entries. `ConcreteStore` uses packed entry storage, a logical-handle-to-position map and fresh increasing identifiers. Reads return one entry; concrete `join` rejects conflicting entries. Address reuse is not a supported abstract execution mode. Concrete addresses are internal and are never required by source programs or boundary clients.

The native collector traces at complete configuration boundaries when occupancy reaches 75% of an elastic entry budget (16,384 initially; backend-configurable threshold from 70% to 80%). It compacts live entry references while preserving logical handles and Java object identity. The `GarbageCollector` interface separates policy from VM reference enumeration. This is storage compaction, not a replacement for the Java collector that manages the underlying objects. The native collector currently runs serially at safepoints; the execution agreement permits other correctly coordinated refinements, including concurrent collection. See native heap collection for roots, boundary retention, measurements and limitations.

Backend-only root provenance, shortest retention paths and Graphviz snapshots share the collector's root enumerator and graph tracer. Graph snapshots can start at the complete root set or selected addresses, including an idle seed before any execution configuration exists. Reports are non-owning observations rejected by the public boundary; no language selector or activation API is added. Audited field signatures, randomized graph oracles and GC-on/off execution comparisons guard coverage. See graph hardening for guarantees and limits.

19.1.2. Values

Runtime Object Representation describes objects, classes, contributions, scopes, caches and occurrence layouts. `VMUninitialized` is an allocated backend sentinel, corresponding to the reference machine's non-value marker. Canonical null and Booleans are role references: native objects in the Current Native Seed, and Aliens in the Host Seed. Integer/string codec transport and Aliens use separate private entries; preserving host scalars selects Aliens.

19.1.3. Contribution declaration indexes

Immutable core mixins expose unbound declaration positions by selector and arity. Lists of up to four declarations use direct scans; larger method, member and slot lists use immutable indexes prepared on first lookup and atomically published. Concurrent first readers may prepare equivalent indexes. Entries contain spellings, arities and local integer positions, never native references or cached receivers. The original declaration lists remain authoritative and preserve order.

`ObjectProtocol` still walks contributions and superclasses in semantic order, selects methods before members before slot accessors within a contribution, and binds the result to the current receiver and exact application occurrence. Physical slots are resolved through that receiver's layout. Mutable callable eligibility is checked on each invocation. This is not a polymorphic inline cache, and neither Alien resolution nor lexical receiver selection is bypassed.

19.1.4. Prepared binding metadata

Function syntax owns reusable immutable parameter slot declarations, prepared on first invocation (with a shared empty case); let syntax owns reusable binding declarations and step-to-slot positions. An immutable parameter record keeps its layout and validation result together. Structural name checks are prepared with that metadata but reported only at the existing function invocation or let evaluation point. Unused malformed raw core code therefore does not fail just because its metadata was prepared. Surface syntactic checks retain their separate earlier validation contract.

Every call and let evaluation still creates fresh activation storage. Captures, return homes, uninitialized values and ordered evaluation are unchanged. Shared metadata contains no activation or world reference. Graph inspection traces cached parameter metadata without preparing unused functions. Generated expressions retained by prepared slots participate in the native graph scan and startup pointer repair. This is a representation refinement of the binding rules, not a new scope model.

19.1.5. Continuations

Frames retain their next reference and the scope/operands needed on resumption:

Frames	Pending work
`ReceiverFrame`, `ArgFrame`	Receiver and ordered argument evaluation.
`LetFrame`	Ordered local binding and sequencing.
`CallReturnFrame`, `DoReturnFrame`	Scope restoration, method completion or delivery to the exact captured home.
`ClassTargetFrame`, `MemberClassReturnFrame`, `ApplyFrame`	Contributor resolution, member-cache publication and deferred member invocation.
`InitializerCallFrame`, `InitializationWriteFrame`, `InitializerReturnFrame`	Private-call operands, resolved slot writes and initializer completion.

Frames	Pending work
`HaltFrame`	Final delivery.

There is no `SeqFrame`: `CoreLetExpression` and `LetFrame` sequence code. `CallReturnFrame` also restores let/lambda scopes without introducing a home. Non-local return scans for its exact home before mutating liveness. Completion, failure and unwind retire homes and abandon pending member realization. Java callbacks start separate native continuation chains; their active foreign context provides the barrier required by the default boundary agreement.

19.1.6. State Transitions

Class realization installs generated `CoreUserFunction` bodies. Primary `new` uses ordinary method dispatch; private allocation/calls/writes/phase transitions are implemented by `ConstructionCode` and the same visitors. `CoreObjectLiteral` retains a zero-formal, factory-free `CoreClassLiteral`, including its contributor targets and argument routes. Evaluation sends exact `new/0` to that class expression, reusing ordinary realization, dispatch and generated construction. No object-specific construction operation or continuation is needed. Empty private-call argument lists still schedule machine steps. The representation and operation rationale are in Private Construction Code.

19.2. Explicit Store: The Key Commitment

A `VMObject` contains a mutable `ValueRef[]`; a slot holds a reference rather than a separately allocated binding cell. Initialization, setters, caches and home retirement mutate backend objects. Configurations share `RuntimeWorld`: retaining an earlier configuration does not preserve an earlier store state. Only reachable native entries survive a collection. Active and synchronously suspended configurations, seed objects, boundary handles and explicit backend root leases define reachability. Backend tools retaining a configuration or raw address across other executions must use `world.nativeHeap().retain(...)` and close the lease when finished; a Java variable alone is not a native heap root. Root registration and lease release must themselves occur under the current guard or while the runtime is exclusively idle. These backend helpers do not acquire coordination internally; leases protect reachability rather than authorizing execution or concurrent store access. Boundary roots follow the lifetime agreement: open invocation attachments, host/detached receipts, public references and admitted uses retain native referents. Canonical Expat proxies and identity caches supply no independent root. Alien wrappers follow ordinary native reachability; collection drops their Java payload holds without inspecting foreign graphs. Collection preserves captured lexical data and retired return-home state, without restoring return authority or introducing activation reflection.

19.3. Staged Seed Construction

`RuntimeSeed.host(store)` publishes the structural Host Seed directly. `RuntimeSeed.create(store)` loads the parser-independent `CurrentNativeSeed` module definition with the compatibility boundary profile. `RuntimeSeed.currentNative(store,`

`profile)` selects another boundary profile with that same native module. These are host construction choices; no language capability can reseed a published runtime.

`RuntimeSeed.fromModule(store, module, profile, arguments...)` accepts an ordinary core module definition and explicit constructor arguments. The `fromNativeModule` convenience entry supplies the narrow root identity service as its sole argument; parser integration lowers `current-native-seed.vh` and exercises that same installer. The parser-independent definition keeps the dependency-free bootstrap checks available; it does not install singleton method records by a separate semantic path.

Public source clients select the same choices through `BootstrapRuntime(capabilities, profile)`, `BootstrapRuntime.fromSeedModule(source, profile, capabilities, inputs...)`, or `BootstrapRuntime.fromNativeSeedModule(source, profile, capabilities)`. The last entry grants the narrow identity service explicitly; the generic entry grants only its supplied inputs. Tooling capabilities are assembled after adoption and are not lent to the seed module's definition scope.

The builder owns a private unpublished world through module construction, exact-once discovery, validation, reference substitution, obsolete boundary-owner retirement and forced collection. It returns only a completed world. Failure closes provisional access and returns no usable partial world. Platform assembly occurs after this step. The full installation invariants are in Program Structure.

19.4. Module Entry

`BootstrapRuntime.instantiateModule` checks a class literal's declared arity and module restrictions, evaluates it in a fresh empty top activation, then uses an exact Expat `new` send. Definition, construction and synchronous callbacks share one transition budget. It returns an `ExpatReceipt`; the host explicitly sends any entry message and closes its receipt when finished. The intermediate class receipt is released after construction. Host sends contribute an empty caller scope.

`BootstrapRuntime.evaluate` is the distinct expression harness, using the assembled kernel as parent and receiver. Its supplied capability context may become part of returned closures/classes. The normative isolation contract is in Program Structure; the richer contexts are described in Tooling.

`MinimalRuntimeWorld` installs ordinary immutable bundle slots named `null`, `true`, `false` and `Object`, holding the corresponding seed identities. Additional supplied capabilities cannot replace those slots in this assembler. These are this tooling bundle's assembly rules, not reserved language names; nearer declarations can shadow its slots through ordinary lexical lookup. Normal modules do not receive these bindings implicitly.

19.5. Java Boundary

19.5.1. Runtime diagnostics

`BoundaryException.diagnostic()` returns immutable `Diagnostic` data: the existing failure kind and short message, a stable reason, an optional source site, semantic trace and related declaration/observation sites. Native reads, returns, construction, unsupported features and limits have specific reasons. Legacy constructors and `kind()` / `getMessage()`

remain supported. A diagnostic retains no host exception cause, native value or execution configuration.

Named source-entry overloads on `BootstrapRuntime` preserve a source label and revision digest in an optional `SourceMap`, separate from executable core nodes. Compilation projects parser/lowering evidence into copied scalar `SourceOrigin` records and explicitly transfers this table to the runtime. The runtime owns weak identity associations, not source snapshots, surface trees or ANTLR data; escaped core code keeps its evidence available after later evaluations without making unreachable code a metadata root. The same core tree can execute in a world with no evidence and must produce coherent source-free diagnostics.

Sites use zero-based lines and UTF-16 columns with exclusive ends, matching the shared source basis. Optional bounded excerpts are copied during compilation and rendering never reopens source files. Generated code may inherit supplied declaration evidence; shared singletons remain unattributed. Missing evidence is reported explicitly, never fabricated from AST strings or another execution's source. Metadata does not change execution, effects, transition budgets, failure kinds or cleanup.

The bootstrap CLI prints defined runtime failures to stderr and exits with status 1. It renders detached diagnostic data rather than a Java exception stack. `--diagnostics=semantic` is the default; `--diagnostics=vm-developer` additionally prints private physical frames, addresses and implementation names. `--trace-frames=N` selects a nonnegative frame limit, default five; `--trace-frames=all` requests the complete trace. `--no-source-excerpts` omits copied snippets. Parser/source-check failures remain a separate frontend path.

`DiagnosticFormatter.summary` provides one escaped line; `format` adds locations, optional excerpts, semantic frames and boundary markers. Missing locations and excerpts are explicit. Rendered labels are bounded to 512 UTF-16 units and messages/excerpts to 4096, including a visible truncation marker when needed; the structured data and legacy short message remain intact. Rendering escapes controls and preserves Unicode boundaries. The editor adapter accepts only matching source revisions and does not introduce execution on editing.

`DiagnosticOptions` on the source runtime configures native capture and persists for later Expat calls. `DISABLED` preserves execution and explicitly supplied foreign evidence while omitting native enrichment. Method/lambda/factory and initializer frames describe actual invocations, without let-scope frames. Finite capture bounds both diagnostic scanning and output; truncation is explicit. Semantic cleanup always covers every affected continuation. Repeated access to a failed member realization retains its original trace and adds the new access as related evidence, without retrying realization.

Developer reports contain detached scalars and remain backend-private. They are available through the explicit tooling entry and are rejected by the value boundary; callbacks preserve public evidence without exporting physical VM frames. Step exhaustion remains `LANGUAGE_FAILURE`; callback-depth and admission limits remain `EXECUTION_POLICY`. See the diagnostics decisions and evidence.

19.5.2. Boundary implementation

`ForeignBoundary.identityMirror()` returns a stable, boundary-owned capability. It is supplied explicitly to modules and observes identity and membership through the ordi-

nary mirror protocol, never a special evaluator dispatch branch. The narrow root Object service shares private identity helpers without exposing the mirror. Native identity hashes follow the stable native identity; preserved foreign values use their payload identity so collecting and recreating an Alien wrapper does not change the hash of a still live foreign referent. The concrete hashing algorithm is a backend policy, not a portable identifier.

DefaultJavaBoundary owns the Alien/Expat contract. `BootstrapRuntime.boundary()` returns a narrow `PublicFacet`; package-private `VmServices` provisions the seed identity service. Both delegate to the same private owner. Canonical Expat and Alien indexes preserve compatible round trips without acting as native roots. Native GC reclaims unreferenced Alien wrappers and prunes their identity-cache entries, relinquishing the Java payload holds. Foreign heaps remain opaque. Startup retirement advances an export epoch, preventing old construction proxies from becoming valid under later attachments to a surviving identity.

`VHExpat` is the canonical proxy; a bare proxy is usable through its current invocation attachment. `ExpatReceipt` supplies durable lifetime evidence for checked sending and transport, independently of proxy identity. `expat()` returns that canonical proxy, `retain()` creates an independent co-hold, and idempotent `close()` ends only this holder's claim. Independent host native results, including evaluation, module construction and the tooling kernel, use receipts. Host clients pass open receipts when reimporting those targets; extracting a bare proxy does not transfer the receipt's authority.

Selected `ForeignOperations` default to attached arguments. The effect-free `detachable(int... positions)` declaration permits explicit detachment at zero-based argument positions; `detachableCallbackResults()` separately permits retaining callback results. `ForeignCallable` supplies corresponding fluent declarations. `CallbackContext.detachArgument(int, VHExpat)` and `detachResult(VHExpat)` register receipts while the authorized invocation is open, validating the supplied proxy against the authorized occurrence. An admission context alone supplies no receiver authority. `ForeignOperation.retainArgument(List, int)` is the simple adapter helper; non-native arguments keep their foreign/by-value transport. Its `release(Object)` helper closes a native receipt only, never an unrelated resource.

Callback native results attach to the enclosing NF invocation. A worker combines its `CallbackContext` with an attachment or uses the `send(ExpatReceipt, ...)` overload. Callback admission, receipt validity and return-home liveness remain separate checks. `EXPIRED_ATTACHMENT`, `RELEASED_RECEIPT` and `UNDECLARED_DETACHMENT` use `OWNERSHIP`; admission uses `EXECUTION_POLICY` and dead-home failure uses `LANGUAGE_FAILURE`.

Host/detached receipt records and admitted-use holds are the actual roots. Java Cleaner assistance may eventually release an unreachable host receipt, without its cleanup registration strongly retaining the receipt. Explicit close ends the hold deterministically. Released receipt records retain no native target graph. This is local retention management, not cross-boundary cycle collection: otherwise unreachable native/foreign cycles remain legal and conservatively retained until an edge is released or a future cooperating collector can prove them unreachable.

Integer/string interning is a non-owning native cache: collection removes cache entries whose native transport values were reclaimed. An open public boundary reference retains its value until `close()`; `ForeignBoundary.Reference` is `AutoCloseable`. Conversion by value does not promise host identity.

The selected boundary profile defines conversion. `NATIVE_VALUES` maps exact native null/Boolean identities to Java `null`/`Boolean` without Expats. `Byte`, `Short`, `Integer` and in-range `BigInteger` normalize to `Long`; `Long` and `String` use by-value transport. Other numeric forms are rejected by that codec. `HOST_VALUES` preserves host null, Booleans, numbers, and strings as Aliens. `HOST_SCALARS` combines native null/Boolean conversion with preserved host numbers and strings. Preservation supplies foreign identity, not Java reflection or native class dispatch. Raw VM representation values are rejected by all profiles. See Possible Seeds and Worlds for the corresponding seed choices.

For the specified callback admission protocol, `DefaultJavaBoundary.Call` records the suspended configuration, parent call, originating thread, admission state and active callback count. `NativeHeap` transition leases root suspended configurations until their native thread resumes; closed calls clear their configuration and ancestry references. `Control.ForeignCall` retains the selected Alien receiver as well as arguments; native tracing follows both. The receiver's adapter hold therefore survives selection and active invocation even when the selected operation does not capture the receiver in its Java closure.

`CallbackContext.send` uses a permit to serialize native execution and short representation-sensitive services. `invoke` releases permission around the selected foreign operation and reacquires it before result conversion. Selection retains native permission and prohibits callbacks. Worker callbacks run on their own host thread, so the blocked caller needs no callback pump. The coordinator's native owner, active call and idle state collectively realize conceptual token ownership; neither copied contexts nor nested permits are individual tokens.

The innermost call admits callbacks and completion waits for admitted descendants, as required by the contract. The implementation uses `wait`/`notifyAll`, with no FIFO or fairness guarantee. Waiting admission can be interrupted with `EXECUTION_POLICY`; required caller restoration waits without abandoning native state and preserves the thread's interrupt flag. No foreign-thread affinity or termination guarantee is added. The shared guard also protects this concrete store's representation-sensitive operations; it is an implementation constraint, not a universal physical-memory token rule.

The callback nesting limit is 64; pending requests do not consume nesting depth. The default no-crossing NLR agreement is implemented by checking home liveness and the callback continuation, then delegating crossing rejection to `DefaultJavaBoundary.nonLocalReturnFailure()`. `Control.Error` aborts the callback and retires its homes before the boundary exports the specified diagnostic. The shared `Execution` retains only the step budget, not a sticky failure marker; foreign recovery does not replenish it. Exhaustion prevents further native transitions without vetoing a terminal foreign result that needs no further step. Host exception classes and message text are not language interfaces.

19.5.3. Stack Safety

The native driver loops over explicit frames, so language method/lambda calls and nested initializers do not recursively call the Java evaluator. Synchronous foreign calls and callbacks do use the host stack. A transition budget cannot preempt a blocking foreign operation; termination remains an adapter concern. The callback limit, retained allocations and possible resource exhaustion prevent any claim of unlimited execution.

19.6. Supported Subset and Remaining Work

Area	Current implementation status
Numbers and strings	Signed 64-bit integers and decoded strings. An explicit host-preserving adapter may enable finite binary64 decimal literals and receiver protocols; unconfigured worlds still reject decimals. JavaHostKernel is an optional foreign library, not native arithmetic/string classes. See its contract.
Source checks	Independent configurable surface-AST traversals reject statically inadmissible forms before lowering; the bootstrap runner and LSP use the standard set. Parser-only and direct core entry remain available. See source checks.
Qualified `outer`	Frontend validation rejects unknown/non-enclosing qualifiers before execution, including unused bodies. Runtime selection still scans captured lexical scopes by name and preserves exact application ancestry at every depth; core nodes do not yet carry bound declaration identities.
Holes	A reached hole is fatal (`UnresolvedHole`); no configurable concrete policy.
Composition	Whole-declaration member override and the specified shared-name state; no deep family merge or adaptation algebra.
Non-local returns across foreign execution	Boundary-owned agreement; only default no crossing is implemented. A supporting transfer protocol is deferred.
Tail calls and reclamation	Native tracing and entry compaction at occupancy safepoints; invocation attachments and explicit receipts replace permanent boundary pins, and unreachable Alien wrappers release Java payload holds. No TCO, cooperative foreign cycle collection or hard memory bound.
Execution agreements	Sequential ordinary execution with scoped worker callbacks. The conceptual token is not reified; parallel native execution, concurrent native GC and general monitor/intervention capabilities are permitted extension directions, not implemented APIs.

Area	Current implementation status
Analysis and persistence	No sound abstract interpreter, persistent snapshots, rollback or portable runtime persistence.

These limits are implementation restrictions or identified open work, not permission to replace accepted semantics. Future TCO must preserve captured homes and barriers; abstract execution needs sound domains, joins, transitions and foreign-effect models beyond an allocator change. See Future Directions and the outer comparison.

19.7. Refinement of the Formal Machine

A refinement relation must decode addressed frames and suspended Java calls into the reference stack; map mutable entries to the current `σ`; and preserve scope, occurrence, cache, `H`, `B` and active failure control. The allocated uninitialized sentinel maps to the private marker. Fresh identities and physical slot positions may be renamed while preserving identity, sharing and logical mappings.

The member binding's private unrealized/realizing/realized/failed states decode directly to the formal cache's absent/busy/ready/failed cases. Its shared identity connects cache entries and addressed completion frames; idempotent abandonment adds no retry or extra language effect. Detecting a reentrant cycle and abandoning the owning attempt remain distinct operations.

Class assembly installs a complete definition whose ordered contributions, superclass, routes, logical layout and generated coordinator decode to one formal realized-class record. Private snapshot preparation is administrative work inside a transition; no source-visible partial class or extra target/argument evaluation is introduced. Startup can replace a complete snapshot under its unpublished-world contract; this does not provide persistent configurations or live class mutation.

It must reconstruct control ownership from the coordinator according to runtime admission. Representation maintenance may stutter; declared interventions would need their specified effects in the target relation. The present refinement target includes neither a parallel execution agreement nor an independent intervention relation.

Object's absent superclass uses the field-specific `Top` decoding.

Administrative steps may stutter, and one foreign Java call can contain many native callback steps. Neither fact permits hiding effects, changing failure kinds, replaying selection or ignoring divergence. A comparison must include the prototype's scalar subset and execution limits, or explicitly exclude runs that hit them. No simulation proof is claimed; the formal appendix records the remaining obligations.

`bash scripts/check-bootstrap.sh` runs focused backend checks with JDK 17+ and Python 3. `./gradlew test` additionally exercises parsing and the complete project with its configured JDK 23 toolchain. Relevant witnesses include UnifiedInvocationTest, ClassCompositionTest, InitializerReturnHomeTest, NativeValuesTest and ModuleEntryTest. The admission regressions in BoundaryReentryChecks cover competing callbacks and pending completion, rejection of unassociated preparation before allocation, leased idle preparation followed by stepping, and concurrent foreign computation and consumption of previously published immutable diagnostics while native execution owns the guard.

20. Tooling Architecture

This chapter defines reusable frontend and tooling contracts, followed by the starter editor's supported scope. Descriptors define inert annotation syntax; runtime diagnostics define detached failures. Tooling does not add an operational meaning to either annotations or syntactic observations.

20.1. Optional Metadata and Source Coordinates

Each layer requires only its semantic input. Surface syntax stands alone without ANTLR tokens or contexts; core syntax stands alone without surface syntax; the runtime requires core code but no parser or source information. Required execution information missing from core is a defect in core/lowering; the runtime must never consult surface syntax to compensate. Position maps, core-to-surface mappings and descriptor associations are optional tooling data outside the trees. Execution diagnostics may use separately supplied evidence, but its absence must preserve coherent failure kinds, reasons, messages and semantic traces. It is a defect to fabricate a source location or recover absent surface/parser data through another execution's metadata.

Declaration-relative scopes, contribution/application identities and return homes are required semantic information, not optional source provenance. Removing diagnostic evidence must not remove these language distinctions.

The implementation shares the `reader-infra` 2.0.0 `SourceText` coordinate basis: zero-based lines, zero-based UTF-16 columns and absolute UTF-16 offsets, with half-open ranges. LF, CRLF and CR each delimit one line. ANTLR code-point indices are converted at the reader boundary; lowering, runtime source projection and LSP consume the resulting coordinates without another conversion. EOF is a zero-width position at the actual end of the snapshot; recovery ranges are bounded by that snapshot, never by synthetic token spellings. Human-readable CLI formatting alone adds one for conventional line/column display.

The coordinate contract follows the LSP position and range contracts, selecting UTF-16. Revision digests hash the exact UTF-16 code units.

20.2. Inert Core Metadata

`vh.syntax.core.metadata` stores opaque descriptor data independently of surface syntax, parser objects and runtime values. `CoreSyntax(root, descriptors)` pairs an executable core root with an optional `CoreDescriptorMap`; an artifact without descriptors uses an empty map. Core nodes hold no pointer to it. `CoreLoweringResult.core()` exposes this artifact without retaining the frontend tracing maps.

`CoreDescriptor.Entry.text()` returns exact raw payload text. Identity-bearing entries and groups retain nesting, ordering and repeated occurrences; equal text does not merge entries. `CoreDescriptorAttachment` adds a stable occurrence and role,

such as EXPRESSION, STATEMENT, RETURN or SLOT. Queries return immutable snapshots. descriptorsOf(owner) preserves groups; descriptorEntriesOf(owner) and descriptorTextsOf(owner) explicitly flatten those groups. describedElements(descriptor) and attachmentsOf(descriptor) provide reverse queries. Text that resembles nested annotations inside a payload is opaque; it creates no inner AST or reverse index automatically.

```
var root = num("42");
var map = new CoreDescriptorMap();
var entry = new CoreDescriptor.Entry("@foreign any syntax");
var attachment = map.attach(root, CoreDescriptorAttachment.Role.EXPRESSION,
                            List.of(entry));
var syntax = new CoreSyntax(root, map);
syntax.descriptors().descriptorEntriesOf(root); // [entry]
syntax.descriptors().describedElements(entry);  // [root]
syntax.descriptors().ownerOf(attachment);       // Optional containing root
```

A member class's declaration and class literal intentionally share the same entry/group identities through CLASS_DECLARATION and CLASS attachments. A local declaration has one CoreBindStep and one LOCAL attachment. A final ordinary statement moves its attachment to the surviving expression, retaining STATEMENT. A final return keeps RETURN on its surviving return operation, separate from annotations on the returned value. Rewritten constructor roots move their attachments to the application edge. Relocation preserves attachment identity and role, rather than cloning payloads or leaving orphaned owners.

DescriptorProjection only copies raw leaves and groups, caching occurrence identities. Optional MetadataOriginMap traces entries, groups and attachments to surface nodes; no metadata entry retains this map. CoreOriginMap separately records executable origins and aliases for eliminated wrappers. directlyFromSurface() selects produced roots; originsOf() returns recorded associations; originOf() retains the preferred diagnostic origin. Adding an alias must not replace an expression's narrower diagnostic origin.

20.2.1. Explicit interpretation by tools

VarioHyve is one available payload language, not the required one. A tool chooses which entries to interpret and may call existing capabilities separately:

```
// Opt into VarioHyve syntax, without source-admissibility checks or
↪  execution.
var parsed = Reader.parseExpression(entry.text());
// Alternatively opt into chosen checks, including the standard collection.
var checked = Reader.readExpression(entry.text(),
↪  SyntacticChecks.standard());
// After explicitly accepting a successful result, obtain ordinary core
↪  syntax.
var lowered = new
↪  SurfaceToCore().lower(checked.result().toOptional().orElseThrow());
// Execution is another explicit choice, with tool-provided
↪  capabilities/budget.
var runtime = new BootstrapRuntime(Map.of("types", toolTypes));
var meaning = runtime.evaluate(entry.text(), 10_000);
```

These are the normal APIs; no duplicated descriptor AST, automatic parse cache, or separate descriptor evaluator exists. A tool can use another parser, skip checks, analyze without execution, or supply an explicit type module/capability when executing. Attaching metadata imports neither the enclosing program's lexical scope nor its authority into the tool's interpretation. Any such binding must be part of the tool's explicit agreement and capability inputs.

Payload parsing returns an independent read result with payload-local zero-based UTF-16 ranges. Its errors belong to the tool and never invalidate the surrounding program's read result. When retained, a `SimpleDescriptor` source range denotes its exact payload substring, excluding verbatim fence markers. A tool may use that evidence to translate local ranges back to the containing document; without it the tool reports payload-local diagnostics. Whole-source snapshots and locations remain optional even though an entry retains its own required text.

The default editor highlights descriptor islands as opaque annotations. Its API-help consumer is an explicit opt-in interpretation, described under Starter Editor.

20.3. Source Checks

The frontend rejects language-invalid constructs established from source alone, including in unexecuted method and lambda bodies. It does not evaluate code or resolve ordinary sends against a presumed runtime world. Dynamic calls, computed contributor targets, holes and valid numeric syntax remain admissible even when a particular execution will fail. Descriptor payloads are excluded under the descriptor contract.

`Reader.readExpression` parses a complete expression and runs selected checks before lowering; the bootstrap source runner and LSP use this checked entry. `Reader.parseExpression` is the parser-only entry for syntax tools and fragments without their enclosing context. Results separate admission, diagnostics and analysis completion:

Result	Retained information
Completed, no errors	Successful surface tree and positions. Warnings remain in `diagnostics()` and do not skip later checks.
Parser error	Diagnostics and a recovery AST when available; source checks do not run.
Source-check error	Diagnostics, complete surface tree and positions as a partial result.
Cancellation/resource limit	Distinct `completion()`, no successful execution tree and a tooling warning; source validity is unknown.

Callers must inspect `ReadResult.diagnostics()` even on success. The bootstrap execution entry currently consumes admission but does not print successful-read warnings; CLI analysis clients can consume the same Reader result directly.

Each `SyntacticCheck` owns its rule and traversal. `SyntacticChecks` selects an immutable ordered collection using `standard()`, `none()`, `with(check)` and `without(id)`. Independent rules share a diagnostic collector, not semantic state. The check catalog lists standard

rules and their stable identifiers. Disabling a check is tooling configuration, not a different language contract. Runtime guards remain necessary for direct core or unchecked inputs; source checks prove neither binding nor executability.

20.3.1. Bounded reading and delimited fragments

`ReadLimits` selects source-size, delimiter-depth, parse-tree-depth, work and elapsed-time limits per request. These are implementation policies, not language limits. Default batch reads use structural and work bounds without a wall-clock deadline (`Duration.ZERO`), so ordinary execution does not fail merely because the machine is slow or busy. Interactive reads retain their 750 ms deadline; callers may explicitly select a positive elapsed-time limit. Cancellation remains active in both policies. The reader checks cancellation during lexing, token prediction and between source checks, bounds token storage and prediction configurations/caches, and guards tree depth before mapping or recursive consumers. Prediction caches belong to the request. SLL parsing with bail falls back to ordinary LL recovery within the same budget. A stack exhaustion is reported as incomplete analysis rather than escaping as a JVM error. A caller-supplied check must bound and cooperate inside its own work; these APIs cannot preempt arbitrary foreign Java code.

`Reader.parseMethodBody("{ ... }")` invokes the same delimited-body rule used by methods, factories and lambdas and requires end of input. It returns statements, positions and diagnostics without synthesizing a class, formals, receiver or return home. It does not run enclosing-context source checks. The overload taking a parent string and start/end offsets returns a `FragmentResult` with explicit parent-snapshot range translation. The fragment retains its own exact source; it must never borrow the full document's revision digest for execution results.

This permits small, independently selected regions to be parsed now. Automatic incremental replacement is not yet implemented: it needs reliable token-level boundaries through comments, strings and descriptor islands, rebuilt ancestor nodes and ranges, and invalidation of affected source checks. A parsed statement list alone does not establish those conditions or the enclosing lookup context.

`vh.syntax.analysis.SnapshotAnalysis` owns replaceable document analysis outside the LSP: it coalesces pending edits, cooperatively cancels obsolete work and uses two workers. Exact submission identity distinguishes even reopenings with the same revision number. Consumers recheck that identity under their publication lock before applying a result. The LSP invalidates old syntax/runtime ranges before scheduling work; queries await their snapshot without holding that lock. A resource outcome appears as a document-level warning, never a red syntax error.

20.4. Analysis Evidence and Client Policies

Analysis belongs to the frontend or explicitly selected tools. The LSP adapts requests, ranges and presentation; the same APIs serve CLI analysis, tests and documentation tools. It owns no language facts or second operational semantics. A future VarioHyve-specific protocol could use these APIs too, but is not implemented.

`AnalysisResult<T>` reports `must ⊆ actual ⊆ may` for a stated query domain, snapshot, model and accepted assumptions. These qualifications travel with the result; a guaranteed fact must not be presented as an unconditional claim about an arbitrary foreign sys-

tem.

Bound	Meaning under the recorded assumptions
Inside `must`	Definitely present.
Outside a sound `may`	Definitely absent. An open upper bound excludes nothing.
Inside `may`, outside `must`	Unresolved: possible, not guaranteed.

Equal finite bounds are exact. An informative upper bound alone is an overapproximation; an informative lower bound alone is an underapproximation; both provide mixed knowledge. Empty lower and open upper bounds mean unknown. Suggested candidates are separate: a finite completion list is not an exhaustive upper bound, and absence from it proves nothing. Evidence explains syntax, repairs, profiles and contract locations. Results with different domains, snapshots or assumptions cannot be combined without justification; no automatic cross-analysis proof/trust aggregation exists.

Initially, exact facts concern structural declaration occurrences in a complete matching syntax snapshot. A selector declared in a local mixin does not establish receiver selection, accessor eligibility, dispatch, execution success or absence of dynamic references. There is no conventional static binder inferred from enclosing declarations. Independently justified source checks remain separate. Starter API suggestions have empty runtime `must`, open runtime `may` and **possible** labels explaining the selected documentation contract.

Alien providers may supply analysis-specific contracts about selectors, results, effects or callbacks. A consumer decides whether it trusts the identified provider's promise and evidence for its purpose. Universal fully proved bounds are not required: proofs have model/checker assumptions, while tests, attestations or accountable promises may suffice for one mission and not another.

Evidence separates these records:

- Provider attribution, which is not authentication; a file path establishes neither.
- The promised contract, applicability and optional revision. An absent revision is not an immutable identity.
- Evidence kind/artifact. A proof claim is not a verified proof.
- The consumer's purpose and rationale, with an unassessed, accepted-for-purpose or rejected trust assessment.

Accepting evidence neither runs an analysis nor promotes candidates to `must`. Recording an assessment does not mutate reported bounds. Missing/rejected evidence leaves usable knowledge open where no independent accepted model supports a bound; it does not block ordinary execution or replace foreign semantics. Descriptors may carry agreements but establish no trust merely by being present. The implementation carries records, not a universal trust policy or proof checker.

Client purpose	Required evidence
Red errors	Established language violations or definite failures under stated accepted assumptions. Unknown/incomplete/possible failures and rejected provider evidence alone are not source errors.
Completion	Favor coverage; distinguish guaranteed, possible and unknown. Claim exhaustiveness only with a justified upper bound.
Navigation	Jump to an execution target only when uniquely established; otherwise show alternatives or unknown. Starter API links navigate selected documentation.
Refactoring	Require coverage of affected references and preservation of relevant behavior. Unique navigation or absence of diagnostics is insufficient; unknown dynamic references cannot be silently skipped.

Bounded ambient authority may enable finite upper bounds through explicit module inputs and accepted models of reachable capabilities. Unmodeled foreign behavior and reflection remain open where necessary. AAM derivation, semantic dispatch analysis and guaranteed refactoring are deferred; this protocol implements none of them.

20.5. Runtime Diagnostics

An explicit evaluation may produce a detached runtime diagnostic. Optional origins and excerpts describe the compiled snapshot and its revision digest; the runtime chapter defines failure payloads, semantic traces and source-free formatting.

Lowering preserves the most specific recorded occurrence for generated code. Each cascade send uses its original message arm; failures in its receiver or arguments retain their narrower origins. Enclosing fallback evidence must not overwrite those sites. Constructor-target rewrites and synthetic return wrappers follow the same rule. Generated temporaries remain private, and absent provenance still produces a coherent source-free failure.

`RuntimeDiagnosticAdapter` in `vh-lsp` maps a diagnostic to an LSP error only when the caller supplies a document URI, source name and revision digest that match its primary site. Zero-based, end-exclusive UTF-16 ranges are already LSP ranges. The stable reason becomes the diagnostic code; matching declaration, call and definition sites become related information. Absent primary origins, stale revisions and unmapped cross-source sites are not assigned invented ranges. The caller is responsible for verifying the document snapshot before publication.

The adapter itself neither executes nor publishes. The language server records each open document's URI, version, absolute source name and digest of its exact UTF-16 code units.

The explicit `vh/runtimeDiagnostics` notification publishes a runner result only when all four match the current snapshot, then combines it with source-check diagnostics. Editing clears prior runtime results immediately; closing clears all results. Open, edit and save never execute source.

20.6. Starter Editor

The VS Code extension selects the JavaHostKernel starter world explicitly. Run File saves a local module and launches `vh.host.Main run` in a separate JVM; Test File selects the SUnit runner. The module receives its kernel (and, for tests, testing library) through ordinary constructor inputs. Console input and output use an integrated terminal separate from LSP traffic. Stop Program terminates that run's process. Restart Language Server restarts editor assistance. Starting Java or running programs requires workspace trust. Run/Test uses its own Java/runner preparation; unavailable or still-starting assistance does not block execution. Runtime Problems publication is best effort when the language server is available; the terminal retains execution results.

Source analysis and API assistance support local files and untitled VarioHyve buffers; only file execution requires a saved local path. The status item separates restricted/unavailable assistance, server startup, pending analysis, completed source checks and incomplete/failed analysis. A completed status requires a matching document URI, version and source digest; empty Problems alone is not proof that analysis ran. Source-error/warning counts remain distinct from completion. The checkmark means source checks completed, not that execution will succeed. TextMate highlighting remains available without Java or workspace trust; its word selection includes legal bang selectors. Opaque descriptor payloads have the dedicated semantic token type `descriptor` and TextMate scope `comment.block.descriptor.vh`, with comment-style theme fallback so inert metadata is distinguishable from executable parameters and names. Themes and user token customizations retain control of the actual colors; payload content is never highlighted as program declarations.

The language server distinguishes shutdown from exit. Shutdown releases document state and cancels pending diagnostic publication before replying; later requests receive `InvalidRequest`. The subsequent exit notification terminates the server process, with status 0 after shutdown and 1 otherwise. The process entry point owns termination; embedded services can be closed without exiting their host JVM. Client disconnection also releases the launcher and document services.

The packaged runner writes a detached diagnostic sidecar for the executed snapshot. The extension verifies it against the launch snapshot and the current document before forwarding it; the server independently verifies the snapshot. Source changed while a run is active therefore cannot receive obsolete ranges. An explicit run may still report its original failure in the terminal.

Run/Test retains the requested document across saves, Save As, server startup and disk verification. A newer Run/Test supersedes older preparation and execution; Stop also cancels a launch that has not spawned yet. Snapshot changes during preparation require another explicit run. Switching the active editor never changes the file selected by a pending command.

The program console provides line input with Backspace, Enter, EOF and interrupt. Unsupported terminal navigation/control sequences are consumed as whole sequences, in-

cluding when split across input events, rather than leaking their bytes into program input. Multiline pasted text submits successive lines. This is a line console, not a terminal emulator with cursor-based editing.

Completion, hover, signature help and API-definition navigation read the bundled class-shaped API contract. Its consumer parses only the documented literal `host`, `doc` and `returns` schema; malformed/unknown payloads are ignored, and annotations are never evaluated. The analysis evidence policy defines the conditional status of these suggestions and documentation links.

The declared starter convention supports:

- Literal receivers and documented result chains.
- The plain module's first constructor input during slot/final initialization, including nested lambdas, regardless of parameter spelling.
- Direct unmodified slots retaining that input or one of its selected services in instance code. Constructor parameters do not become method bindings and factories do not receive instance-slot evidence.

Only enclosing routine parameters/locals shadow this structural evidence; unrelated routines do not. A lambda's own declarations can invalidate it. Any matching executable setter marker conservatively invalidates retained-slot help, even if it might target a different object/local: resolving writes would require operational analysis. Absence of such a marker proves neither absence of mutation nor absence of overrides.

Named receivers across nested classes/objects, invoked getters, mutable aliases, slot-to-slot propagation, composition and unknown dynamic receivers receive no guessed protocol. Editing repairs may fill a missing selector/argument or close delimiters for parsing; arbitrary recovery does not establish a protocol. These are structural filters for the starter convention, not lexical selection, accessor resolution or execution simulation.

Folding uses mapped class, method, factory, lambda and initializer ranges, plus multiline opaque descriptor ranges. Selection returns one entry per requested cursor, in the same order. Unmapped text, including trailing comments or an empty document, receives a zero-width fallback. Parent selections strictly contain their children; absent, duplicate or incompatible recovery ranges are skipped.

20.6.1. Independent scratch evaluation

Evaluate Selection in Scratch copies exactly one nonempty selection into a separate, editable untitled VarioHyve document and evaluates its complete text. **Evaluate Scratch** reruns that document. Each invocation uses a fresh process with the declared starter tooling context: `kernel` is JavaHostKernel, alongside the bootstrap `Object`, `null`, `true` and `false`. These are explicit tool inputs, not ambient module authority. No receiver, locals or return home is captured, and no earlier evaluation state is reused. Each invocation's effects remain real and are not rolled back.

The ordinary checked expression entry remains in force. The tool introduces no wrapper, relaxes no source check and executes no unrelated source text. Multiple statements can be written explicitly as `(fun { ... }).apply()`. Native/foreign object results get generic labels; displaying a result performs no extra message send or arbitrary foreign `toString`. Returned native receipts are released.

The host entry is `vh eval <file> [--source-name <label>] [--diagnostics <file>]`.

The label is diagnostic identity, never a path to reopen. The editor writes the exact scratch snapshot to a temporary UTF-8 transport file and uses the scratch URI as its source name; invalid UTF-16 is rejected rather than silently replaced. Runtime Problems are published only to the matching scratch URI/version/digest. The original document and its diagnostics are unaffected. Stop and newer explicit execution cancel pending scratch launches too; temporary files are removed when the run ends or preparation is cancelled.

Scratch context membership lasts for the editor session and is not inferred from a saved filename. Saving the text does not turn ordinary Run File into expression evaluation. Object inspection, persistent sessions, customizable contexts and debugger activation evaluation remain independent future features.

20.7. Evaluation Contexts

A tool can evaluate with explicitly supplied objects or select a debug activation. The context uses activation objects, ordinary lexical lookup and the same Alien/Expat and return rules. It introduces neither a separate Environment namespace nor a global service registry. Language-visible debugger and mirror services must be explicitly supplied boundary capabilities. Internal activation selection does not make activations public language values.

Operation	Context
Module instantiation	Empty top activation; explicit constructor inputs
Tool expression evaluation	Explicitly supplied objects, such as sample data or a console
Proposed debugger evaluation	Selected activation's bindings plus explicitly supplied services

A tool must identify its context and additional capabilities. It cannot silently enrich normal module entry. Results retain actual lexical captures, including classes that resolve superclass/mixin targets there; export and persistence must preserve or explicitly account for those dependencies.

The Java entry paths implement module and supplied-expression evaluation, not a general selected-activation debugger API.

20.8. Execution, Observation and Intervention

The execution agreement distinguishes ordinary evaluation, representation maintenance and explicitly authorized observation/intervention. Tools use that contract rather than gaining execution rights from possession of a configuration. Backend preparation and retention follow the Java coordination rules.

For example, an immutable published observation can be consumed while ordinary execution continues. Reading live slots needs a specified consistency model; changing one is an intervention whose effective ordering must be defined. Replacing or resuming a continuation also needs an ownership-transfer protocol so its previous owner cannot con-

tinue. These examples do not imply a general monitor or selected-activation intervention API in the current prototype.

A model checker is another explicitly supplied tool: a supporting parallel VM could honor its request for independent simultaneous transitions under the parallel-agreement requirements. The execution capability, proof evidence and concrete implementation must meet that agreement; a saved configuration alone supplies none of them.

20.9. Descriptors as the Naming Bridge

Intrinsic declaration names identify many syntax elements. A descriptor can name an otherwise anonymous expression, for example `(compute(x) : intermediate)`. It attaches information without changing execution. Name resolution for tool references is separate from language message lookup.

20.10. External Annotation Files

Each tool may keep a separate external file associating syntax identities with its own facts: a type, interval, units assumption, proof obligation or diagnostic. Formats and interpretations belong to the tool. Adding or changing annotations does not alter raw execution, supply runtime capabilities or establish that an asserted property holds. Different tool views need not use a shared format.

20.11. Qualified Naming

Paths such as `module.Class.method.local` are proposed tooling conventions, not language selectors. Tools must resolve collisions and preserve references when source changes. Descriptor-supplied names can identify subexpressions, but a stable AST/source-mapping scheme is still required; spelling a name alone does not solve identity across edits.

20.12. Connection to Abstract Interpretation

The explicit-state visitor machine provides a starting point for analysis. AAM and ADI are research approaches, not modes enabled by an allocator or an "abstract kernel." The required derivation is described under Persistence and Analysis.

Annotations may supply domains or assumptions subject to the analysis evidence policy. General semantic analysis, mirror tooling and persistence remain future work.

Part V.

Future

21. Future Directions

This chapter separates accepted extension directions from unresolved proposals; neither implies an implemented feature or promised release. Extensions preserve the accepted capability inputs, lookup, construction and boundary contracts unless a later decision explicitly changes them. Tool annotations alone cannot change raw execution. Any default handler, mirror or scheduling service must be supplied explicitly, not discovered through an ambient `kernel`.

21.1. Operators and Control Structures

Proposed surface extension. Binary operators, conventional conditionals, loops, pattern matching and library DSLs could lower to the core's sends, lambdas and objects. A monadic extensible parser inspired by Lean 4 is one candidate. Precedence, hygiene, phase separation and preservation of evaluation order need specification. The current syntax already expresses control through ordinary messages and thunks; no extensible parser is implemented.

21.2. Named and Optional Arguments

Proposed; dispatch unresolved. Named arguments, defaults and varargs must coexist with exact name/arity selection and generated setters. A candidate `name = expression` argument form would need a distinct grammar context from assignment. Decisions include parameter-name binding, default evaluation scope and timing, duplicate/unknown arguments, and how omitted or variable arguments map to selectors. The current language has positional arguments only.

21.3. String Interpolation

Proposed syntax. Interpolation could lower to explicit conversion and concatenation messages. Delimiters, conversion protocol and evaluation order remain unspecified. It also depends on native string operations, which the prototype does not yet implement; see the scalar design note.

21.4. Collection Literals

Proposed syntax and libraries. Lists/maps could be constructed through ordinary supplied classes, with future literal sugar. Syntax must avoid conflicting with object literals. Collection classes, key equality and hashing, mutability and element evaluation order require contracts; example `List`/`Map` names do not denote ambient standard objects.

21.5. Mixin Contract Verification

Proposed analysis. A verifier could collect a contribution's `super` sends and check the required selectors at each actual application occurrence. The selected continuation of the dispatch chain, arity and invocation rules matter; a set of names alone is insufficient. Descriptors could state stronger contracts. Results belong to external tooling and do not replace runtime failure on an unsatisfied contract. See object-model dispatch and tooling.

21.6. Deep Family and Mixin Composition

Accepted direction; deferred semantics. Deep composition with state in member classes needs rules for nested declarations, constructor signatures, sibling references, state and ordering. The current whole-declaration member override and ordered contributions remain the reference. Preserve declaration identities, definition scopes and occurrence identities so later composition can distinguish repeated applications.

Possible operations include aliasing, renaming, hiding, exclusion, precedence and explicit state sharing/mapping. They are distinct: an alias need not rewrite recursive sends; hiding need not remove internal access; selecting behavior does not settle storage collisions. Effectful initializer routes require an execution plan—registrations and callbacks cannot be assumed to commute or roll back. One-object construction and the foreign boundary must survive any extension. See the accepted scope decisions and composition design space.

21.7. Trait-Style Conflict Resolution

Alternative under consideration. Detection of overlapping method selectors could require an explicit own override or contribution-specific delegation, instead of silently accepting declared precedence. That would change current ordered override semantics and is not accepted. Direct contribution delegation syntax and its receiver/origin rules are also unresolved. Reified definitions help detection, but future composition may require additional metadata.

21.7.1. `doesNotUnderstand` as a Message

Open design. Receiver interception through a `doesNotUnderstand` message could support proxies and delegation. Questions include recursive failure, typo masking, default behavior and analysis of dynamic selectors. Signaling a missing-message condition is another proposal. Neither is implemented: current missing dispatch is a fatal diagnostic and never invokes a DNU method.

21.8. Exception System

Proposed condition/restart system. Current fatal diagnostics are not catchable, resumable or retryable. The design goals are local recovery, escalation through callers, optional

resumption at the signal site with a supplied value, and an explicitly configured last-resort policy.

The Common Lisp-inspired proposal separates:

Role	Responsibility
Condition	Describe a problem at the signaling site.
Handler	Select recovery policy in an enclosing dynamic context.
Restart	Offer a named recovery operation near the component that can perform it.

One candidate scans active handler/restart frames while preserving the signaling continuation, then either resumes or explicitly unwinds. Direct frame mechanisms and a design built from delimited continuations/algebraic effects remain alternatives. Handler matching and ordering, restart extent and selection, resumption multiplicity, cleanup, home liveness and foreign-call barriers need precise rules. Two or three new frame classes would not by themselves specify this semantics or prove its analysis cost.

The proposed missing-message condition would replace fatal DNU only after an accepted change. Unhandled conditions could reach a deployment-supplied handler: a test runner may record failure, a simulator may choose a degraded value, and an analyzer may represent alternatives. These are distinct execution policies, not capabilities implicitly available to every module. Existing boundary failure kinds must survive propagation; changing resumability requires an explicit contract for external effects.

A later library could offer `try/catch`, `ensure` and assertion-like operations over the chosen mechanism. Cleanup on normal return, failure and non-local return must be defined before claiming that `ensure` is ordinary sugar. The investigation order is basic signaling/handlers, restarts and explicit abort, library forms, then analysis. No release numbers or implementation-size claims are committed.

21.8.1. AAM Compatibility

No condition-system abstraction has been derived. Finite source or frame forms do not imply finite execution state. An analysis must represent handler/restart instances, dynamic names, continuation alternatives, state updates and control outcomes, with sound matching and transfer rules. Joins combine alternatives; widening is a separate convergence technique when a domain needs it. See analysis obligations.

21.9. Tail-Call Optimization

Proposed investigation; not implemented. The objective is bounded control space for eligible native tail-call chains, including mutual calls and delegation. Explicit continuations already avoid one Java evaluator frame per native call; that is distinct from bounded store use or proper tail recursion.

A final send is only a candidate. Its callee may invoke a captured lambda whose `^` still targets the caller's live home. For `^ recurse()`, the operand executes before return delivery; collapsing frames afterward does not reduce peak recursive space. An optimization

must preserve home identity and retirement, initializer phases and remaining contributions, constructor results, pending member-cache completion, callback barriers, selected receiver/origin, evaluation order and exactly-once effects. Foreign calls need separate eligibility rules.

The proposed sequence is to measure the recursion catalog, identify core tail contexts and completion obligations, prototype a conservative native optimization, and compare results, traces, identity, failures, continuation depth and allocation. Captured bindings cannot be reused if doing so changes closure observations. Reclamation and retained data/caches need a separate space account; a growing list of deferred homes does not establish bounded control space.

Inlining needs the actual selected behavior or justified guards, not special selector names such as `ifTrue`. Debug history also has a cost: bounded summaries, recording and replay provide different guarantees. Proper tail recursion remains a stronger uncommitted space property. The research basis is Clinger, *Proper tail recursion and space efficiency*.

21.10. Mirror-Based Reflection API

Broader reflection remains proposed. Mirrors could offer controlled inspection or invocation on a target. Reflective operations belong on mirrors, not ambient methods of `Object`; the application need not itself implement reflection. Obtaining a mirror requires authority, and language-visible backend services still use the Alien/Expat boundary.

The API must decide which class, slot, method and origin information is exposed, which mutations are allowed, and how access restrictions, optimizations and lifetime are respected. Existing backend fields are implementation evidence, not a public representation contract or proof that no new metadata is needed. There is currently no public class getter or activation reification.

The implemented IdentityMirror protocol is the baseline. Further kinds, standalone mixin references, native scalar identity and broader reflection remain open. The design note records the current scope and limitations.

21.11. Access Control

Proposed policy; no language access control is enforced today. The candidate object-capability design treats an object's export surface as its external behavioral contract:

1. Private declarations by default, with explicit exports; getter and setter exposure can differ.
2. Inherited exports remain visible, while parent internals are sealed from subclasses. Whether a separate "heritable" visibility is useful remains open.
3. A mixin's exported behavior becomes internal to its applier unless re-exported; the mixin's own helpers remain available to that contribution but hidden from unrelated contributions.
4. Lexical nesting grants access to enclosing internals. The exact treatment of explicit sends to an `outer ClassName` receiver still needs specification.
5. Class-side `new`, `apply` and named factories follow the same export policy; external construction requires an exported path.

These are candidate changes to current lookup, not behavior enabled by existing annotations. Export syntax, especially for synthesized factories, is undecided. A tool-only descriptor could check conventions first; runtime enforcement would require an explicit semantic increment. Expats must then expose the same surface as external native callers. Origin metadata may assist enforcement, but sealing, virtual overrides, state access and composition require proofs and possibly new metadata. Export sets alone do not establish safe pruning in an analyzer.

The accepted `outer` operation still selects an ordinary declaration-relative receiver. This proposal creates neither a universal parent getter nor public activation reflection. Related Works places this proposal alongside Newspeak and other capability designs; no novelty or simplicity claim is assumed.

21.12. Method tear-offs

Proposed; neither syntax nor a method-reification API is implemented. Ordinary `obj.m` invokes `m/0`; it never implicitly returns a method. The current candidate `obj.m#N` would evaluate the receiver once, select an actual method `m/N`, and return a callable retaining that method, receiver and dispatch origin. Applying it would execute the selected method with its original `self` and exact `apply/N` arity. It would not select callable-slot fallback or member-class factory candidates.

Under that proposal, `obj.m#2(a,b)` obtains the method reference and applies it; a mismatched application arity fails. This requires no separate immediate-apply semantic operation. An earlier `$` suffix candidate omitted explicit arity; it is not accepted syntax either. Final grammar, bound versus unbound references, identity/equality of repeated references, interaction with overrides and `super`, and mirror integration remain open. The accepted distinction between access, invocation and callable values stays in Message sends.

21.13. Concurrency and Parallelism

Accepted execution-agreement extension principle; concrete parallel agreements and the composition layer remain future work. No scheduler is implemented. The current execution agreement already permits maintenance refinements and declared interventions while serializing ordinary native control. This section concerns richer agreements authorizing simultaneous native transitions. The Java world's mutable store and effectful foreign operations supply neither snapshots nor a pure state transformer.

21.13.1. Independent Transitions and Parallel Execution

A model checker with an appropriate capability may ask a supporting parallel VM to execute two independent enabled transitions concurrently. The agreement must specify execution authority, shared-state semantics, observations and the independence evidence it accepts. It need not mandate a proof format: evidence may be supplied with the request or established by a trusted analysis, but it must apply to the actual configuration and assumptions used.

For transitions t_1, t_2 enabled in C, the independence obligation is that each preserves

the other's applicability and both orders yield equivalent configurations and observable effects under the agreement:

$$C \xrightarrow{t_1} C_1 \xrightarrow{t_2} C_{12}, \qquad C \xrightarrow{t_2} C_2 \xrightarrow{t_1} C_{21}, \qquad C_{12} \approx C_{21}.$$

Equivalence includes the specified boundary effects and their ordering, not just equal final slot values. Arbitrary foreign operations cannot be presumed independent; their declared contracts or an explicit effect model must supply the necessary justification. Speculation, cancellation and replay need their own effect agreements.

Semantic independence and concrete concurrency safety are separate obligations. A parallel implementation must correctly coordinate its allocator, store, roots, continuations, caches and collector; mathematical commutation does not make today's Java data structures thread-safe. An implementation parallelizing work that refines an allowed sequential trace may retain the sequential abstract agreement. Authorizing additional ordinary native control requires the richer agreement and its authority discipline. Neither case changes fixed lookup, identity, return-home lifetime, initializer phases or the boundary's NLR policy.

This extension principle adopts no parallel VM implementation, scheduler, snapshot representation, model-checking algorithm or public transition-proof API. Those remain separate increments with their own specifications and tests.

21.13.2. Boundary-supported Non-local Returns

Implementing an agreement beyond the default no-crossing boundary requires a transfer mechanism and specified treatment of intervening foreign work. This is separate from parallel execution or scheduling. The NF/FN design note compares immediate escape and deferred delivery without selecting either as a mandatory mechanism.

21.13.3. The Semantic Language Interface (SLI)

The proposed SLI would expose machine stepping, input/output interaction and possibly inspection or snapshots to an external composition runtime. Actors, CSP, dataflow and parallel simulation are candidate compositions, with explicit scheduling, synchronization, time, routing and failure policies. Those policies must preserve native lookup and the message-level Alien/Expat contract; swapping schedulers does not prove application behavior unchanged.

A future boundary suspension could expose an operation request, then resume with a reply. It must define callback/reentrancy, identity, ownership, cancellation and failure behavior before replacing today's synchronous calls. Snapshot/restore must account for the native store, live homes, continuations and boundary caches, and explicitly exclude, model or compensate external effects. Replay and speculation require recorded inputs and suitable foreign contracts; they are not free consequences of sequential stepping.

The incremental composition proposal starts with bounded native state copying/comparison, explicit admitted state, a small SLI adapter and observations before shared-state composition. Gamine's SLI, communicating vats/actors, DEVS and virtual-actor runtimes are inspirations, not implemented integrations. Progress and sound analysis across composed machines remain separate proof obligations.

21.14. Persistence and Analysis

Deferred. Portable platform persistence must preserve identities, captures, composition origins and explicitly reconnectable foreign bindings. It is not Java object serialization and must not discover new authority while loading. See object-graph persistence.

An abstract execution requires domains, store/control abstractions, transfer rules and foreign-effect assumptions related to the reference semantics. A configurable allocator, supplied abstract object or descriptor does not provide that derivation. Native scalar protocols and foreign conversion must also have explicit contracts; current open choices are recorded in the value/analysis note.

Part VI.

Related Works

22. Connections and Comparisons

VarioHyve combines an object language with explicit boundaries for independently implemented behavior. These comparisons distinguish inherited ideas, alternative choices and possible verification techniques. Proposed integrations and proof directions are not implemented results.

22.1. Federation and Model Interoperability

Maier identifies operational and managerial independence as distinguishing systems-of-systems concerns. VarioHyve adopts this perspective when participants retain their semantics and authority. Federating engineering models nevertheless does not imply that their subject is a system-of-systems, or that the coordinator controls every participant. Maier (1998)

Openflexo's FML organizes virtual models, concepts, roles and behaviors, connected to resources through technology adapters. Its synchronization behaviors operate on models in their original technical spaces: both FML and VarioHyve can coordinate execution. FML supplies modeling abstractions; VarioHyve investigates an object-language boundary exposing selected operations as capabilities. An experiment should compare the same coordination in FML, including adapter and correspondence maintenance. Openflexo Core, model-mapping tutorial

Eclipse Epsilon supplies model querying, validation and transformation, with connectivity, Java embedding and workflow support. VarioHyve might combine an Epsilon validation with a simulator while retaining Epsilon's semantics. The experiment must establish whether explicit authority and interaction contracts improve that workflow over Epsilon-only or host-language coordination. Epsilon documentation

CorrLang and comprehensive systems specify relationships among heterogeneous endpoints, distinguishing identity from other commonalities. Their graph-based correspondences address entity and constraint alignment. An Alien's reference identity does not establish that two models describe the same physical entity; conversely, correspondence does not establish compatible callback or failure behavior. These agreements could complement VarioHyve's behavioral contracts. CorrLang introduction and comprehensive-systems references

22.2. Objects, Modules and Reified Composition

Smalltalk supplies the orientation toward communicating objects and minimal irreducible machinery. VarioHyve shares these concerns without inheriting a Smalltalk image or debugger through similar syntax. Ingalls, *Design Principles Behind Smalltalk* (1981)

Newspeak is the closer comparison: lexical nesting, late binding and modules without globals make dependencies construction inputs. VarioHyve likewise uses explicit mod-

ule inputs and preserves declaration origins in lexical selection and mixin application. It specifies its own callable eligibility, initialization and class-side construction. The detailed comparison records deliberate differences, including exact captured ancestry for deeper repeated parent applications, and remaining implementation limits; complete Newspeak conformance is not claimed. Bracha et al., *Modules as Objects in Newspeak* (2010)

Piumarta and Warth bootstrap an extensible object model from a small messaging seed. This informs VarioHyve's private initialization machinery, with a deliberate difference: native lookup and invocation remain fixed. Reified contributions preserve semantic information rather than granting replacement dispatch policies. Language-visible VM services use the ordinary Alien/Expat agreement. Piumarta and Warth, *Open, Extensible Object Models* (2008)

22.3. Authority, Reflection and Foreign Boundaries

Miller's *Robust Composition* connects object capabilities, cooperation and limiting interference. VarioHyve's explicit inputs and restricted service views follow this concern: public and privileged VM-service facets share transport without sharing every grant. This does not provide E's distributed execution, persistence or hostile-component security guarantees. Miller (2006)

Bracha and Ungar separate reflection from base behavior, encapsulate its implementation and align reflective concepts with the observed language. VarioHyve's supplied `IdentityMirror` follows this separation: observations belong to the mirror, leaving Alien selectors to their foreign protocol. Implemented identity and membership observations are a limited facility, not a general activation-inspection API. Bracha and Ungar, *Mirrors* (2004)

22.4. Execution Monitoring and Controlled Intervention

Kishon, Hudak and Consel compose language semantics with monitor specifications. Their monitoring semantics separates the program's answer from monitoring information, with preservation obligations; subsequent work develops profilers, tracers and interactive debuggers. Kishon, Hudak and Consel (1991), Kishon and Hudak (1995)

Drey and Teodorov distinguish observation from deliberate intervention, such as editing a variable during diagnosis. VarioHyve accordingly separates ordinary continuation authority, observation and intervention contracts. Its sequential control token does not universally lock representations. Observers need defined consistency; mutating monitors need specified additional effects. Neither implicitly changes lexical return-home rules. These are tooling contracts, not an implemented debugger. Drey and Teodorov (2016)

Implementation concurrency remains separate: Dijkstra and colleagues demonstrate collection concurrent with computation under established invariants. VarioHyve permits such semantic-preserving maintenance, without claiming a concurrent collector implementation or proof. Dijkstra et al. (1978)

22.5. Analysis and Verification of Composed Behavior

Abstracting Abstract Machines derives sound analyses through systematic machine abstraction. VarioHyve's explicit state is a starting point, not an analyzer: soundness must cover object scopes, contribution identities, return homes and foreign effects. Bounding addresses alone is insufficient. Foreign summaries must account for mutation, callbacks, failures and nontermination. Van Horn and Might (2010)

Abstracting Definitional Interpreters offers another derivation path: monadic interpreters can support collecting and abstract semantics while inheriting precise call/return matching from the defining language. This suggests an alternative analysis organization, not a property obtained automatically by VarioHyve's Java interpreter. Either route needs a sound relationship to the specified object and boundary semantics. Darais et al. (2017)

Interaction trees model recursive computation and external events; VarioHyve crossings would need intermediate callbacks, not just atomic results. Omnisemantics supports reasoning over every permitted nondeterministic outcome. Neither establishes that a foreign provider meets its contract. Xia et al. (2020), Charguéraud et al. (2023)

DimSum composes independently defined language semantics through events and protocol translations. Melocoton verifies OCaml/C interoperability with related language-local state. Both suggest obligations for conversions, roots and callback control; neither verifies VarioHyve's Java adapter. The research audit expands these comparisons, including alternative boundary calculi, while the case-study catalog defines integration experiments and baselines. DimSum (2023), Melocoton (2023)

Part VII.

Appendix

A. Formal Rules

This appendix gives the structural operational semantics (SOS) of the accepted core. Surface syntax and its explanatory examples are in **Specification**; Java representations and restrictions are in Java Interpreter. Unsettled parameters are listed at the end. No correctness or refinement theorem is assumed.

A.1. Signature and notation

$$v, \rho \in A, \quad \sigma : A \rightharpoonup \mathit{Node}, \quad u \notin A, \quad s = (n, k), \quad o = (C, i, \mu),$$
$$H : \mathit{Home} \rightharpoonup \{\text{method}, \text{slots}, \text{body}, \text{dead}\},$$
$$X = (\sigma, H, B),$$
$$\kappa ::= \text{halt} \mid f :: \kappa, \qquad K = \langle c, \rho, \kappa \rangle_X.$$

A contains private runtime identities; u is uninitialized storage. B is the boundary state, including identity maps and canonical null/Boolean role identities. Failure is carried by the active control and is reported at the nearest callback boundary or independent entry. Fresh identities are always distinct from existing ones. An omitted configuration component is unchanged; $X[\sigma(v).p := w]$ denotes a field update, not a source assignment. $\cdot$ concatenates sequences/stacks. For context-preserving steps, $\langle c \rangle_X$ abbreviates $\langle c, \rho, \kappa \rangle_X$ with the same ρ, κ on both sides.

Node or record	Fields relevant to execution
Obj(C, S, ρ_d, Γ)	Class, physical slots, definition scope, member cache.
Act(U, ρ_p, v, o, h)	Generated local/parameter accessors, lexical parent, receiver, occurrence, return home.
View(v, o, ρ_d)	Declaration-relative receiver scope; no parameter bindings or return home.
$\Lambda(\bar{x}, e, \rho_d, v, o, h)$	Native object of class Lambda, with code and private lexical captures.
$\mu =$ $(D, \rho_d, \bar{x}, T, \bar{S}, \bar{D}_m, e_b)$	Reified owned contribution: declaration, scope, formals, operations, slots, members, final initializer body.
Class$(\mu, \bar{o}, S, R, L, M)$	Owned contribution, direct applications, superclass, routes, complete layout, metaclass.
Meta$(\mu, \ldots)$	Class-side record; its fixed structural fields are given below. Not an eligible construction-clause target.
Alien(ϕ), Scalar(p)	Foreign identity or scalar payload; neither implies a native class.

Scopes use object accessors, not a separate binding environment. Every active scope carries receiver/origin/home metadata; a `View` has $h = \bot$. $\sigma(\rho).U$ is storage for generated accessors, not a source namespace. It retains declaration mutability: local bindings

have setters; parameters do not. The maps below abbreviate these tagged storage declarations.

A.1.1. Class algebra

Let $A_{\text{cls}} \subseteq A$ contain identities whose store records are `Class` or `Meta`. The superclass field ranges over $A_{\text{cls}} \cup \{\text{Top}\}$. The distinguished empty constructor $\text{Top} \notin A$ is not a store node, source value, lookup miss $\bot$, uninitialized marker u, or the canonical null identity.

The class algebra and its projection from realized classes are:

$$\begin{aligned} Q &::= \text{Top} \mid \text{Apply}(\mu, Q), \\ \text{shape}(\text{Top}) &= \text{Top}, \\ \text{shape}(C) &= \text{Apply}(\mu_C, \text{Apply}(\mu_{o_1}, \ldots \text{Apply}(\mu_{o_m}, \text{shape}(S_C)) \ldots)). \end{aligned}$$

For no direct applications the last equation is $\text{shape}(C) = \text{Apply}(\mu_C, \text{shape}(S_C))$; for Object without direct applications, $\text{shape}(\text{Object}) = \text{Apply}(\mu_{\text{Object}}, \text{Top})$. A root with stateless direct applications follows the general projection above. This is a projection of instance-side composition, not an equality of runtime identities: occurrences, scopes, routes and metaclasses remain in their records. It introduces no intermediate classes. Every realized superclass chain is finite and ends at `Top`; only Object has `Top` as its direct superclass.

`Top` has no owned contribution, scope, class-of link or superclass projection. There is no value, allocation, factory or initializer-entry rule for it. Source construction-clause targets must be realized non-metaclass identities in A_{cls}; the seed supplies Object's terminal superclass.

A.1.2. Core and machine controls

$$\begin{aligned} e &::= l \mid \text{null} \mid ? \mid \text{self} \mid \text{outer}(D) \\ &\quad \mid q.n(\bar{e})_\iota \mid \lambda\bar{x}.e \mid \text{class}(d) \mid \text{object}(d) \\ &\quad \mid \text{let}(\bar{b}; e) \mid \text{ret}(e) \\ &\quad \mid \text{alloc}(C) \mid \text{init}(\tau; e_r, \bar{e}) \mid \text{write}(j, e) \mid \text{slotsDone}, \\ q &::= \text{implicit} \mid \text{explicit}(e) \mid \text{super}, \\ \iota &::= \text{exact} \mid \text{invoke}, \qquad b ::= x \leftarrow e \mid _ \leftarrow e. \end{aligned}$$

The return form $\text{ret}(e)$ is source `^e`. Bare sends use `exact/0`; parentheses use `invoke/k`. Leading-dot sends lower to an explicit `self` receiver. The final four forms are private generated construction code. A class descriptor d retains owned declarations, contributor-target expressions and delayed routes. For $\text{object}(d)$, the descriptor is anonymous, with no owned constructor formals or factories; it may retain the same optional contributor clauses as a class descriptor.

The core form null is an internal constant, not the source identifier `null`. Empty bodies and omitted initializers lower to this form; source `null`, `true` and `false` lower to ordinary implicit sends. Their results depend on lookup and the selected behavior, not on this constant rule or a reserved source spelling.

$$
\begin{aligned}
c ::= {} & e \mid \mathrm{val}(v) \mid \mathrm{collect}(a; \bar{v}; \bar{e}) \mid a[\bar{v}] \\
& \mid \mathrm{call}(p, \bar{v}) \mid \mathrm{enter}(v, o, \bar{x}, \bar{v}, e, \chi) \\
& \mid \mathrm{steps}(\bar{b}; e) \mid \mathrm{return}(h, v) \\
& \mid \mathrm{member}(g) \mid \mathrm{foreign}(\omega) \mid \mathrm{callback}(v, n, \bar{f}) \\
& \mid \mathrm{error}(\epsilon) \mid \mathrm{abort}(\epsilon), \\
\chi ::= {} & \mathrm{method} \mid \mathrm{initializer}.
\end{aligned}
$$

a is a pending control with a value-list hole: a send, a private initializer entry, or class realization. Frames appearing below are exhaustive for these forms: `Recv`, `Args`, `Bind`, `Restore`, `Done`, `Return`, `Write`, `Cache`, `Apply`, `Module`, and `Barrier`. Their fields are given by their rules; the tail is κ.

A.1.3. Runtime admission and conceptual control

The following transition rules describe ordinary execution under the sequential execution agreement. Their runtime-level admissibility condition is one conceptual control token τ_c per live runtime, with one owner:

$$
\begin{aligned}
\mathrm{owner}(\tau_c) \in {} & \{\mathrm{idle}\} \\
& \cup \{\mathrm{native}(\alpha) \mid \alpha \text{ admitted}\} \\
& \cup \{\mathrm{foreign}(\iota) \mid \iota \text{ active}\}.
\end{aligned}
$$

Here α ranges over native continuations and ι over foreign invocations. The condition ranges over one runtime's evolving state and all its configurations, not separate tokens per configuration, stored version or thread. Ordinary native rules require ownership by their continuation; modeled foreign actions require ownership by their invocation. Transfers below are abstractly atomic, with associations retained in B and barriers. τ_c is ghost state, not a required field or language value; initializer target τ is unrelated.

Abstract event	Required ownership transfer
Accepted independent entry or tooling readmission	`idle` to the selected new or prepared native continuation
Committed NF invocation	Native caller to its fresh foreign invocation
Admitted callback	Current foreign invocation to the fresh callback continuation
Callback result or failure delivered at its barrier	Callback continuation to the associated foreign invocation
Foreign `done` or `fail` delivered	Foreign invocation to its suspended native caller, after admitted descendants finish
Independent final result or failure delivered	Native continuation to `idle`
Backend driver yields after a requested step	Native continuation to `idle`, retaining the successor for later readmission

All other ordinary steps retain ownership. Pending/rejected requests and completion arrival are adapter bookkeeping; `done`/`fail` below denotes delivery after descendant com-

pletion. The admission contract supplies nesting eligibility and rules out confusing a copied context with control ownership.

Backend one-step yield may leave a nonterminal successor without retiring its homes or producing a final observation. Later advancement requires readmission. The source evaluator's driver instead retains authority between native steps until a boundary transfer or completion.

This layer models serialized control effects, not arbitrary foreign computation, host queues, wakeups or fairness. Such work can continue while a callback owns the token. Refinement must reconstruct ownership from the implementation; these conditions are not a completed proof of the Java coordinator.

The execution agreement also permits maintenance refinements and declared interventions. Maintenance may stutter; an intervention changing abstract state needs its own specified transition. The rules below define neither an intervention relation nor a parallel agreement. Proofs must state which extensions, if any, they include.

A.2. Evaluation

Literal interpretation $B, \sigma \vdash l \Downarrow (v, \sigma', B')$ is fixed by the selected scalar profile; conversion failures use `error`. The internal null form returns the canonical identity v_{null}.

$$\frac{B, \sigma \vdash l \Downarrow (v, \sigma', B')}{\langle l, \rho, \kappa \rangle_X \longrightarrow \langle \text{val}(v), \rho, \kappa \rangle_{X[\sigma:=\sigma', B:=B']}}$$

$$\begin{aligned} \langle \text{null}, \rho, \kappa \rangle_X &\longrightarrow \langle \text{val}(v_{\text{null}}), \rho, \kappa \rangle_X, \\ \langle ?, \rho, \kappa \rangle_X &\longrightarrow \langle \text{error}(\text{Hole}), \rho, \kappa \rangle_X, \\ \langle \text{self}, \rho, \kappa \rangle_X &\longrightarrow \langle \text{val}(\sigma(\rho).v), \rho, \kappa \rangle_X. \end{aligned}$$

The `self` rule requires a receiver; absence is an error. For `outer(D)`, let ρ_j be the first declaration-relative view for lexical declaration D in the retained lexical chain. No such view is `InvalidOuter`. At every depth, traversal follows the retained lexical parent of each activation or view. A view preserves its exact occurrence and captured definition parent; the enclosing receiver's contribution chain is never searched for a replacement application of the same declaration. Implicit lexical selection uses this same chain, followed by ordinary dispatch on the selected receiver.

$$\frac{\sigma(\rho_j) = \text{View}(v, o, \rho_d) \qquad \mu_o.D = D}{\langle \text{outer}(D), \rho, \kappa \rangle_X \longrightarrow \langle \text{val}(v), \rho, \kappa \rangle_X}$$

$$\frac{w \text{ fresh} \qquad \sigma' = \sigma[w \mapsto \Lambda(\bar{x}, e, \rho, \sigma(\rho).v, \sigma(\rho).o, \sigma(\rho).h)]}{\langle \lambda \bar{x}.e, \rho, \kappa \rangle_X \longrightarrow \langle \text{val}(w), \rho, \kappa \rangle_{X[\sigma:=\sigma']}}$$

A.2.1. Receiver and arguments

`send` retains its mode and origin: $r = \text{lex}(\rho)$, $\text{recv}(v)$, or $\text{super}(v, o)$. In the following first rule, $q \in \{\text{implicit}, \text{super}\}$, and r is `lex(ρ)` for implicit sends and `super(σ(ρ).v,σ(ρ).o)` for `super`; an absent required receiver/origin is an error.

$$\langle q.n(\bar{e})_\iota, \rho, \kappa\rangle_X \longrightarrow \langle \text{collect}(\text{send}(r, n, \square, \iota); []; \bar{e}), \rho, \kappa\rangle_X,$$
$$\langle \text{explicit}(e).n(\bar{e})_\iota, \rho, \kappa\rangle_X \longrightarrow \langle e, \rho, \text{Recv}(n, \bar{e}, \iota, \rho) :: \kappa\rangle_X.$$

$$\langle \text{val}(v), \rho', \text{Recv}(n, \bar{e}, \iota, \rho) :: \kappa\rangle_X \longrightarrow \langle \text{collect}(\text{send}(\text{recv}(v), n, \square, \iota); []; \bar{e}), \rho, \kappa\rangle_X.$$

$$\langle \text{collect}(a; \bar{v}; []), \rho, \kappa\rangle_X \longrightarrow \langle a[\bar{v}], \rho, \kappa\rangle_X,$$
$$\langle \text{collect}(a; \bar{v}; e :: \bar{e}), \rho, \kappa\rangle_X \longrightarrow \langle e, \rho, \text{Args}(a, \bar{v}, \bar{e}, \rho) :: \kappa\rangle_X,$$
$$\langle \text{val}(v), \rho', \text{Args}(a, \bar{v}, \bar{e}, \rho) :: \kappa\rangle_X \longrightarrow \langle \text{collect}(a; \bar{v} \cdot [v]; \bar{e}), \rho, \kappa\rangle_X.$$

A class-target `Args` frame rejects each nonclass or metaclass result before advancing to the next target. Other argument lists retain values without executing their contents. These rules evaluate receivers/arguments once, left-to-right.

A.2.2. Local scope and sequencing

For $\text{let}(\bar{b}; e)$, allocate fresh a and $\sigma' = \sigma[a \mapsto \text{Act}(U, \rho, v, o, h)]$, where (v, o, h) are copied from ρ and $U = \{x \mapsto u \mid (x \leftarrow e_x) \in \bar{b}\}$. All local names are reserved before the first initializer; duplicate names fail.

$$\langle \text{let}(\bar{b}; e), \rho, \kappa\rangle_X \longrightarrow \langle \text{steps}(\bar{b}; e), a, \text{Restore}(\rho) :: \kappa\rangle_{X[\sigma:=\sigma']},$$
$$\langle \text{steps}([]; e), \rho, \kappa\rangle_X \longrightarrow \langle e, \rho, \kappa\rangle_X,$$
$$\langle \text{steps}((x \leftarrow e_0) :: \bar{b}; e), \rho, \kappa\rangle_X \longrightarrow \langle e_0, \rho, \text{Bind}(x, \bar{b}, e, \rho) :: \kappa\rangle_X,$$
$$\langle \text{val}(v), \rho', \text{Bind}(x, \bar{b}, e, \rho) :: \kappa\rangle_X \longrightarrow \langle \text{steps}(\bar{b}; e), \rho, \kappa\rangle_{X[\sigma(\rho).U(x):=v]},$$
$$\langle \text{val}(v), \rho', \text{Restore}(\rho) :: \kappa\rangle_X \longrightarrow \langle \text{val}(v), \rho, \kappa\rangle_X.$$

For $x = _$, the `Bind` rule discards the value and leaves storage unchanged. Source assignment remains a setter send. Empty bodies and missing initializers lower to the internal null form. Sequencing is a `let` with discard steps, not an additional binding discipline.

A.3. Selection

This section is the formal counterpart of message selection and lexical lookup.

$$\begin{aligned}
\text{lin}(\text{Top}) &= [], \qquad L_{\text{Top}} = \emptyset,\\
\text{lin}(C) &= [(C, 0, \mu_C)] \cdot \bar{o}_C \cdot \text{lin}(S_C),\\
\mathcal{R} &= \{\bot\} \cup \{!\epsilon \mid \epsilon \in \mathit{Error}\} \cup \mathit{Plan},\\
p &::= t \mid \text{memberApply}(g, k),\\
t &::= \text{get}(v, p_s) \mid \text{set}(v, p_s)\\
&\quad \mid \text{method}(v, o, \bar{x}, e) \mid \text{lambda}(w)\\
&\quad \mid \text{memberGet}(v, o, d) \mid \text{foreignOp}(\phi, op).
\end{aligned}$$

$\bot$ is a miss and $!\epsilon$ is an error. A plan is committed behavior. $T_{v,o}(s)$ is the instantiated declaration table at occurrence o: explicit methods override generated accessors at the

same selector. Slot targets contain $p_s = L_{\text{class}(v)}(o, j)$; activation targets contain their local slot. Generated getters are `get` and `memberGet`. A lambda has intrinsic `apply/k` at its declared arity; other selectors use its native Lambda/Object hierarchy. Scalar tables are parameter **S**.

The recursive equation for $\text{lin}(C)$ applies only to realized class records. The empty base contributes no occurrence or slot mapping. Reaching it therefore uses the empty-list miss rules below for both exact and invocation search, including an exhausted `super` suffix; `Top` itself is never a receiver.

A.3.1. Exact and invocation lookup

Exact search over an ordered contribution list is:

$$E(v, [], s) = \bot,$$
$$E(v, o :: L, s) = \begin{cases} T_{v,o}(s) & T_{v,o}(s) \neq \bot, \\ E(v, L, s) & T_{v,o}(s) = \bot. \end{cases}$$

For an Alien, $E(v, s)$ instead uses effect-free adapter selection $B \vdash \phi.s \Downarrow R$, translating a selected operation to `foreignOp(φ,op)` and retaining miss/error. No foreign getter is speculatively invoked. For native v, $E(v, s)$ starts at its complete hierarchy; an activation uses only its own accessor table. Write L_v for the receiver's search list: $\text{lin}(\text{class}(v))$ for native objects, including lambdas for non-`apply` selectors, and one accessor table for an activation. Lambda `apply/k` selects its intrinsic operation only at the declared arity; invocation retains the captures in the lambda rule below. Intrinsic apply has no invented method/contribution origin; inherited Object methods use their normal origin and the lambda object as receiver.

Exact callable eligibility excludes generated getters. Compute $E(v, (\text{apply}, k))$ once and match its retained outcome against these cases:

$$P(v, k) = \begin{cases} t & E(v, (\text{apply}, k)) = t \text{ and } t \text{ is not a generated getter}, \\ !\epsilon & E(v, (\text{apply}, k)) = !\epsilon, \\ \bot & \text{otherwise}. \end{cases}$$

Let η retain successful accessor evidence for this invocation. The following ordered equations define generated-accessor eligibility A; in the member row, $g = \text{memberGet}(v, o, d)$:

$$A(g, k, \eta) = \eta(g, k) \qquad \text{if defined},$$
$$A(\text{get}(v, p_s), k, \eta) = \begin{cases} !\text{Uninitialized} & \sigma(v).S[p_s] = u, \\ P(w, k) & \sigma(v).S[p_s] = w, \end{cases}$$
$$A(g, k, \eta) = \text{memberApply}(g, k) \qquad g \text{ a member}, \ k \in \text{arities}(d),$$
$$A(g, k, \eta) = \bot \qquad \text{otherwise}.$$

Here storage notation includes activation slots. $\text{arities}(d)$ contains the selected member declaration's constructor arity and owned `apply` factory arities; it neither consults the realization cache nor imports contributor factories.

Invocation search $I(v, L, n, k, b, \eta)$ carries the getter-masking bit b. In these rules set $t = T_{v,o}(n, k)$, $g = T_{v,o}(n, 0)$:

$$I(v, [], n, k, b, \eta) = \bot,$$

$$\frac{t \neq \bot \quad t \text{ not a generated getter}}{I(v, o :: L, n, k, b, \eta) = t},$$

$$\frac{t = \bot \text{ or generated} \quad \neg b \quad g \neq \bot \quad A(g, k, \eta) = R \neq \bot}{I(v, o :: L, n, k, b, \eta) = R},$$

$$\frac{t = \bot \text{ or generated} \quad b \ \vee \ g = \bot \ \vee \ A(g, k, \eta) = \bot}{I(v, o :: L, n, k, b, \eta) = I(v, L, n, k, b \vee (g \neq \bot), \eta)}.$$

For each occurrence, compute the unmasked $A(g, k, \eta)$ at most once and match its retained outcome across the alternatives. Premises short-circuit left-to-right: never read a masked getter. All searches start with $b =$ false. An ordinary `n/0` method masks older getters without being executed to discover `apply/k`. A nearer eligible accessor can precede a farther exact method. Errors terminate selection.

A.3.2. Lexical receiver selection

For each scope ρ_i, let D_i be its own declaration table and v_i its associated receiver: the activation itself or the receiver of a `View`. Define the eligibility judgment $\rho_i \vdash (n, k, \iota) \Downarrow R_i$ by this ordered table. $\checkmark(v_i, \eta)$ selects a receiver and evidence.

Condition	Eligibility result
`exact`, $D_i(n, k) \neq \bot$	$\checkmark(v_i, \emptyset)$
`invoke`, $D_i(n, k)$ exists and is not a generated getter	$\checkmark(v_i, \emptyset)$
`invoke`, $D_i(n, 0) \neq \bot$, $g = E(v_i, (n, 0))$ is an ordinary method, $k = 0$	$\checkmark(v_i, \emptyset)$
`invoke`, $D_i(n, 0) \neq \bot$, $g = E(v_i, (n, 0))$, $A(g, k, \emptyset) = p$	$\checkmark(v_i, \{(g, k) \mapsto p\})$
An evaluated selection premise yields $!\epsilon$	$!\epsilon$
Otherwise	$\bot$

Walk lexical parents until the first nonmiss. On $\checkmark(v_i, \eta)$, dispatch the **original** selector/mode on v_i; do not execute the declaring operation directly. Invocation uses $I(v_i, L_{v_i}, n, k, \text{false}, \eta)$. Exhausting the lexical chain starts the same search on current `self`, or misses if no receiver exists. A final dispatch error/miss never resumes lexical search. Eligibility is thus established before an override can replace the eligible accessor route.

Explicit native sends start at the full receiver hierarchy. `super(v,o)` starts strictly after the **executing occurrence** o, with fresh getter masking. The suffix is determined by occurrence identity, not by contribution equality. Explicit Aliens use adapter selection in either mode. Invalid native origins fail; foreign spellings `self`, `outer`, and `super` remain foreign selectors. Write $r \vdash (n, k, \iota) \Downarrow R$ for this complete selection judgment.

$$\frac{r \vdash (n, |\bar{v}|, \iota) \Downarrow p}{\langle \text{send}(r, n, \bar{v}, \iota), \rho, \kappa \rangle_X \longrightarrow \langle \text{call}(p, \bar{v}), \rho, \kappa \rangle_X}, \quad \frac{r \vdash (n, |\bar{v}|, \iota) \Downarrow \bot}{\langle \text{send}(r, n, \bar{v}, \iota), \rho, \kappa \rangle_X \longrightarrow \langle \text{error}(\text{MissingSelector}), \rho, \kappa \rangle_X}.$$

A selected $!\epsilon$ similarly enters `error(ε)`. Eligibility evidence retains the callable target itself, including a selected foreign operation; `call` never rereads its getter or repeats adapter selection.

A.4. Calls and generated accessors

$$\begin{aligned}
\langle \text{call}(\text{get}(v, p_s), []), \rho, \kappa \rangle_X &\longrightarrow \langle \text{val}(\sigma(v).S[p_s]), \rho, \kappa \rangle_X, \\
\langle \text{call}(\text{set}(v, p_s), [w]), \rho, \kappa \rangle_X &\longrightarrow \langle \text{val}(w), \rho, \kappa \rangle_{X[\sigma(v).S[p_s] := w]}, \\
\langle \text{call}(\text{method}(v, o, \bar{x}, e), \bar{v}), \rho, \kappa \rangle_X &\longrightarrow \langle \text{enter}(v, o, \bar{x}, \bar{v}, e, \text{method}), \rho, \kappa \rangle_X.
\end{aligned}$$

The getter requires a value other than u; otherwise it enters `error(Uninitialized)`. Setters exist only for writable declarations. For `enter`, arity must match. Allocate fresh a, ℓ, h:

$$\sigma' = \sigma[\ell \mapsto \text{View}(v, o, \mu_o.\rho_d), a \mapsto \text{Act}(\bar{x} \mapsto \bar{v}, \ell, v, o, h)],$$
$$H' = H[h \mapsto \begin{cases} \text{method} & \chi = \text{method}, \\ \text{slots} & \chi = \text{initializer}. \end{cases}$$

$$\begin{aligned}
&\langle \text{enter}(v, o, \bar{x}, \bar{v}, e, \chi), \rho, \kappa \rangle_X \\
&\quad \longrightarrow \langle e, a, \text{Done}(h, v, \chi, \rho) :: \kappa \rangle_{X[\sigma := \sigma', H := H']}.
\end{aligned}$$

A method's definition scope is the selected contribution's scope, never its receiver's own constructor activation. For a lambda, no fresh home is created:

$$\frac{\sigma(w) = \Lambda(\bar{x}, e, \rho_d, v, o, h) \quad |\bar{x}| = |\bar{v}| \quad a \text{ fresh} \quad \sigma' = \sigma[a \mapsto \text{Act}(\bar{x} \mapsto \bar{v}, \rho_d, v, o, h)]}{\langle \text{call}(\text{lambda}(w), \bar{v}), \rho, \kappa \rangle_X \longrightarrow \langle e, a, \text{Restore}(\rho) :: \kappa \rangle_{X[\sigma := \sigma']}}$$

Arity errors in a committed call are failures, not new lookup misses.

A.5. Class realization and construction

A.5.1. Definition, occurrences and layout

A descriptor's target list is $\bar{e}_d = [e_{A_1}, \ldots, e_{A_m}, e_S]$: direct mixins left-to-right, then an explicit superclass target. Omit e_S for implicit `Object`. Terminal constructor arguments remain in $d.R$.

$$\langle \text{class}(d), \rho, \kappa \rangle_X \longrightarrow \langle \text{collect}(\text{realize}(d, \rho, \square); []; \bar{e}_d), \rho, \kappa \rangle_X.$$

Given validated target classes $A_1, \ldots, A_m, S$, allocate fresh C, M. Let μ_C be d's owned instance contribution capturing ρ_d, $o_i = (C, i, \mu_{A_i})$, and $o_0 = (C, 0, \mu_C)$. The store extension is:

$$\begin{aligned}\sigma(C) &= \text{Class}(\mu_C, [o_1, \ldots, o_m], S, R_C, L_C, M),\\ \sigma(M) &= \text{Meta}(\mu_C^{\text{class}}, [], \text{Class}, [], L_M, \text{Metaclass}),\\ L_C(o, j) = L_C(o', j') &\iff \text{name}(o, j) = \text{name}(o', j').\end{aligned}$$

L_C numbers distinct slot names in first-occurrence order through $\text{lin}(C)$, retaining a mapping for every logical slot. All inherited occurrences keep their original applying class. The class-side contribution contains only owned factories and generated `new`/default `apply`. With this extension σ':

$$\langle \text{realize}(d, \rho_d, [A_1, \ldots, A_m, S]), \rho, \kappa \rangle_X \longrightarrow \langle \text{val}(C), \rho, \kappa \rangle_{X[\sigma := \sigma']}.$$

For implicit `Object`, insert that structural root without a source lookup. Declaration validity, reserved `new`, flat root seeding and factory signatures are the static constraints of Objects and Classes. For realized ordinary classes C, including Object, Class, Metaclass and the shared Lambda class (not the semantic terminal `Top`):

$$\begin{gathered}\text{super}(\text{Object}) = \text{Top}, \quad \text{super}(\text{Class}) = \text{Object},\\ \text{super}(\text{Metaclass}) = \text{Class}, \quad \text{super}(\text{Lambda}) = \text{Object},\\ \text{class}(C) = M_C, \quad \text{super}(M_C) = \text{Class}, \quad \text{class}(M_C) = \text{Metaclass}.\end{gathered}$$

Lambda values have class Lambda while retaining the record's lexical captures. Lambda's class has no public factories. Applications reuse the selected owned contribution's identity; membership observations compare that identity with the supplied class's owned contribution. IdentityMirror is an ordinary foreign capability governed by the observation contract; it adds no privileged machine transition.

A.5.2. Member-class cache

For $g = \text{memberGet}(v, o, d)$, the cache key is $k_g = (v, o, d.D)$, with $\Gamma(k_g) \in \{\bot, \text{busy}, \text{ready}(C), \text{failed}(\epsilon)\}$. Γ abbreviates the enclosing object's private cache in σ.

$$\begin{gathered}\langle \text{call}(g, []), \rho, \kappa \rangle_X \longrightarrow \langle \text{member}(g), \rho, \kappa \rangle_X,\\ \langle \text{call}(\text{memberApply}(g, k), \bar{v}), \rho, \kappa \rangle_X \longrightarrow \langle \text{member}(g), \rho, \text{Apply}(\bar{v}, \rho) :: \kappa \rangle_X.\end{gathered}$$

$$\frac{\Gamma(k_g) = \text{ready}(C)}{\langle \text{member}(g) \rangle_X \longrightarrow \langle \text{val}(C) \rangle_X}.$$

For an absent entry, allocate fresh ℓ with $\sigma(\ell) = \text{View}(v, o, \mu_o.\rho_d)$, and set $\Gamma(k_g) = \text{busy}$:

$$\langle \text{member}(g), \rho, \kappa \rangle_X \longrightarrow \langle \text{class}(d), \ell, \text{Cache}(k_g, \rho) :: \kappa \rangle_{X'}.$$

$$\langle \mathrm{val}(C), \rho', \mathrm{Cache}(k_g, \rho) :: \kappa \rangle_X \longrightarrow \langle \mathrm{val}(C), \rho, \kappa \rangle_{X[\Gamma(k_g) := \mathrm{ready}(C)]},$$
$$\langle \mathrm{val}(C), \rho', \mathrm{Apply}(\bar{v}, \rho) :: \kappa \rangle_X \longrightarrow \langle \mathrm{call}(t, \bar{v}), \rho, \kappa \rangle_X \qquad P(C, |\bar{v}|) = t.$$

`busy` raises `CyclicRealization`; `failed(ε)` raises that error without retry. `Apply` commits to the realized class's exact callable operation: miss/error fails, never resumes outer lookup. A member that loses virtual dispatch or fails arity eligibility never enters these cache rules.

A.5.3. Generated constructor code

Let $\tau = \mathrm{whole}(C)$ select the complete coordinator and $\tau = \mathrm{owned}(o)$ select one owned contribution. With ; denoting the already-defined discard sequencing:

$$\begin{aligned}
I_\mu &= \mathrm{write}(0, e_0); \cdots; \mathrm{write}(r-1, e_{r-1}); \mathrm{slotsDone}; e_b, \\
\mathrm{superRoute}(\mathrm{Top}, []) &= [], \\
\mathrm{superRoute}(S, \bar{a}_S) &= [(\mathrm{whole}(S), \bar{a}_S)] \quad (S \in A_{\mathrm{cls}}), \\
R_C &= \mathrm{superRoute}(S, \bar{a}_S) \cdot [(\mathrm{owned}(o_m), \bar{a}_m), \ldots, (\mathrm{owned}(o_1), \bar{a}_1)], \\
R_C &= [(\tau_1, \bar{\beta}_1), \ldots, (\tau_q, \bar{\beta}_q)], \\
I_C &= \mathrm{init}(\tau_1; \mathrm{self}, \bar{\beta}_1); \cdots; \mathrm{init}(\tau_q; \mathrm{self}, \bar{\beta}_q); I_{\mu_C}, \\
\mathrm{new}_C(\bar{x}) &= \mathrm{let}(z \leftarrow \mathrm{alloc}(C), _ \leftarrow \mathrm{init}(\mathrm{whole}(C); z, \bar{x}); z), \\
\mathrm{apply}_C(\bar{x}) &= \mathrm{explicit}(\mathrm{self}).\mathrm{new}(\bar{x})_{\mathrm{exact}}.
\end{aligned}$$

Names $z, \bar{x}$ in generated code denote exact accessor sends; z is fresh. An owned `apply/k` replaces the generated default at that arity. Object's superclass is `Top`, so it has no superclass dependency by `superRoute`. Any direct root mixins and its owned contribution are stateless, zero-parameter, and have empty initializers; an empty implicit Object call may be omitted. No route targets `Top` and no initializer activation, return home or effects are created for it. The base equation admits no arguments; there is no whole(Top) routine. A direct mixin imports neither its source class's routes nor its other contributors. Source factories otherwise use ordinary method calls.

$$\begin{aligned}
\langle \mathrm{alloc}(C), \rho, \kappa \rangle_X &\longrightarrow \langle \mathrm{val}(v), \rho, \kappa \rangle_{X[\sigma(v) := \mathrm{Obj}(C, u^{|\mathrm{im}\, L_C|}, \mu_C \cdot \rho_d, \emptyset)]}, \\
\langle \mathrm{init}(\tau; e_r, \bar{e}), \rho, \kappa \rangle_X &\longrightarrow \langle \mathrm{collect}(\mathrm{initEntry}(\tau, \Box); []; e_r :: \bar{e}), \rho, \kappa \rangle_X, \\
\langle \mathrm{initEntry}(\tau, v :: \bar{v}), \rho, \kappa \rangle_X &\longrightarrow \langle \mathrm{enter}(v, o, \mu_o . \bar{x}, \bar{v}, I_\tau, \mathrm{initializer}), \rho, \kappa \rangle_X.
\end{aligned}$$

Allocation requires a realized constructible C and fresh v. For `whole(C)`, $o = o_0$, $I_\tau = I_C$; for `owned(o)`, $I_\tau = I_{\mu_o}$. The coordinator's route expressions and owned body share this single home; each nested dependency call gets its own. An object literal realizes its anonymous class and sends ordinary exact `new/0`:

$$\langle \mathrm{object}(d), \rho, \kappa \rangle_X \longrightarrow \langle \mathrm{explicit}(\mathrm{class}(d)).\mathrm{new}([])_{\mathrm{exact}}, \rho, \kappa \rangle_X.$$

The ordinary receiver rule evaluates the class expression exactly once before dispatch. Its target expressions run in ρ; routed arguments run in the fresh declaring initializer

home after allocation. The resulting class and receiver are fresh on every evaluation. This introduces no additional private operation or object-specific continuation, and does not call `apply` or any contributor factory.

For a live current initializer home h with $H(h) = \text{slots}$, set $v = \sigma(\rho).v$, $o = \sigma(\rho).o$, and $p_s = L_{\text{class}(v)}(o, j)$ **before** evaluating the value expression:

$$\begin{aligned}
\langle \text{write}(j, e), \rho, \kappa \rangle_X &\longrightarrow \langle e, \rho, \text{Write}(v, p_s, \rho) :: \kappa \rangle_X, \\
\langle \text{val}(w), \rho', \text{Write}(v, p_s, \rho) :: \kappa \rangle_X &\longrightarrow \langle \text{val}(w), \rho, \kappa \rangle_{X[\sigma(v).S[p_s] := w]}, \\
\langle \text{slotsDone}, \rho, \kappa \rangle_X &\longrightarrow \langle \text{val}(v), \rho, \kappa \rangle_{X[H(h) := \text{body}]}.
\end{aligned}$$

Invalid private targets, arities or phases raise errors. No selector or Expat operation can name these controls.

A.6. Completion, return and failure

The rules below model the default boundary agreement, which prohibits non-local returns across foreign execution. Whether to support crossing belongs to the boundary agreement, not a universal native-language prohibition. A supporting agreement requires additional transfer rules; none are defined or implemented here. Lexical return targets, home liveness and initializer phase restrictions remain native rules.

$$\frac{(\chi, H(h), v') \in \{(\text{method}, \text{method}, w), (\text{initializer}, \text{body}, v)\}}{\langle \text{val}(w), \rho', \text{Done}(h, v, \chi, \rho) :: \kappa \rangle_X \longrightarrow \langle \text{val}(v'), \rho, \kappa \rangle_{X[H(h) := \text{dead}]}}.$$

An initializer finishing in `slots` raises `PrematureCompletion`. A source return first evaluates its operand, capturing $h = \sigma(\rho).h$:

$$\begin{aligned}
\langle \text{ret}(e), \rho, \kappa \rangle_X &\longrightarrow \langle e, \rho, \text{Return}(h) :: \kappa \rangle_X, \\
\langle \text{val}(v), \rho, \text{Return}(h) :: \kappa \rangle_X &\longrightarrow \langle \text{return}(h, v), \rho, \kappa \rangle_X.
\end{aligned}$$

Let $\kappa = P \cdot [\text{Done}(h, w, \chi, \rho_c)] \cdot \kappa_c$, with no `Barrier` in P. Successful delivery requires $H(h) = \text{method}$ or $H(h) = \text{body}$, according to χ. Let X' retire all homes in that prefix through the target and mark its pending `Cache` frames failed with `AbandonedRealization`:

$$\frac{\kappa = P \cdot [\text{Done}(h, w, \chi, \rho_c)] \cdot \kappa_c \qquad v' = \begin{cases} v & \chi = \text{method}, \\ w & \chi = \text{initializer} \end{cases}}{\langle \text{return}(h, v), \rho, \kappa \rangle_X \longrightarrow \langle \text{val}(v'), \rho_c, \kappa_c \rangle_{X'}}.$$

Failure cases for this default agreement, checked before any successful-unwind mutation, are:

Condition	Error
Missing or dead captured home	`DeadContext`
Live target lies below the first barrier (default no-crossing agreement)	`ForeignReturn`

Condition	Error
No matching completion frame	`DeadContext`
Reachable target still has $H(h) = \text{slots}$	`PrematureCompletion`

Only completion of the target contribution is skipped; its caller's subsequent contributions continue. A lambda's captured home is never dynamically rebound.

$$\langle \text{error}(\epsilon), \rho, \kappa \rangle_X \longrightarrow \langle \text{abort}(\epsilon), \rho, \kappa \rangle_X.$$

For an ordinary frame f (not `Barrier`), obtain X_f by retiring its home if it is `Done`, or caching `failed(ε)` if it is `Cache`; otherwise $X_f = X$:

$$\langle \text{abort}(\epsilon), \rho, f :: \kappa \rangle_X \longrightarrow \langle \text{abort}(\epsilon), \rho, \kappa \rangle_{X_f}.$$

There is no rollback, lookup retry, or revival of a retired home. Aborting a callback retires only frames above its barrier; the suspended caller remains live. Cache failure caused by a successful return does not abort that return.

A.7. Alien/Expat transitions

The adapter state `B` includes call association and admission lifetime. The Java realization may obtain a callback from the operation thread or from a worker holding an explicit call context; admission serializes their native transitions. Each admitted callback belongs to the current foreign invocation, retains its suspended caller and budget, and creates the barrier below. Closing a foreign invocation disallows further callback actions for it and joins admitted descendants before delivering `done` or `fail`. Host requests rejected before admission do not create a native callback continuation. These rules describe the serialized native interaction; they do not formalize arbitrary parallel foreign actions, host waiting queues or a general scheduler.

These transitions use the default no-crossing agreement: callback entry installs `Barrier`, and the foreign action relation has no non-local-return transfer action. A future supporting boundary must define that transfer and its observable interaction with foreign execution; merely removing `Barrier` is insufficient.

The adapter provides stateful, identity-preserving conversions $B, \sigma \vdash \bar{v} \Uparrow (\bar{f}, B')$ and $B, \sigma \vdash \bar{f} \Downarrow (\bar{v}, \sigma', B')$, or failure with the state reached before failure. Lists convert left-to-right. When the selected profile defines a canonical codec, null/Booleans convert by exact role identity; foreign-preserving profiles retain Alien payloads. Other scalars follow **S**. Compatible identity round trips satisfy $\Downarrow\Uparrow v = v$ and $\Uparrow\Downarrow \phi = \phi$. These are boundary obligations, not privileged native service operations.

Conversions and foreign actions also satisfy the lifetime agreement. Adapter state `B` records invocation attachments, declared detachment permissions and open retention receipts. Export establishes its protecting root before publication; import validates the supplied evidence and establishes the receiving root before sender protection ends. Callback exports attach to the enclosing NF invocation; independent host exports carry receipts. Completion closes only that invocation's attachments after descendants and result conversion, including defined failure paths. Detachment and release are boundary

bookkeeping, not additional ordinary native control tokens. These rules abstract the collector and foreign retention machinery; they do not formalize cooperative collection of cross-boundary cycles.

$$\frac{B, \sigma \vdash \bar{v} \Uparrow (\bar{f}, B')}{\langle \mathrm{call}(\mathrm{foreignOp}(\phi, op), \bar{v}), \rho, \kappa \rangle_X \longrightarrow \langle \mathrm{foreign}(op(\phi, \bar{f})), \rho, \kappa \rangle_{X[B:=B']}}.$$

The foreign interaction-state relation $B \vdash \omega \rightsquigarrow (a, B')$ has actions $a ::= \mathrm{done}(f) \mid \mathrm{fail}(\epsilon) \mid \mathrm{next}(\omega') \mid \mathrm{send}(v, n, \bar{f}, \omega')$. It may block or diverge; it need not be deterministic. The state ω records foreign behavior and resumption; it is not the unique control token. Several suspended barriers can retain distinct foreign interaction states. Actions `done` and `fail` below mean completion delivery after the admission condition above is satisfied, not the physical arrival of an answer from a foreign worker. Its completion rules are:

$$\frac{B \vdash \omega \rightsquigarrow (\mathrm{next}(\omega'), B')}{\langle \mathrm{foreign}(\omega), \rho, \kappa \rangle_X \longrightarrow \langle \mathrm{foreign}(\omega'), \rho, \kappa \rangle_{X[B:=B']}},$$

$$\frac{B \vdash \omega \rightsquigarrow (\mathrm{done}(f), B_1) \quad B_1, \sigma \vdash f \Downarrow (v, \sigma', B_2)}{\langle \mathrm{foreign}(\omega), \rho, \kappa \rangle_X \longrightarrow \langle \mathrm{val}(v), \rho, \kappa \rangle_{X[\sigma:=\sigma', B:=B_2]}},$$

$$\frac{B \vdash \omega \rightsquigarrow (\mathrm{fail}(\epsilon), B')}{\langle \mathrm{foreign}(\omega), \rho, \kappa \rangle_X \longrightarrow \langle \mathrm{error}(\epsilon), \rho, \kappa \rangle_{X[B:=B']}}.$$

For callback action `send(v,n,fs,ω′)`, allocate an empty activation a with no parent, receiver or home:

$$\frac{\begin{array}{l}\langle \mathrm{foreign}(\omega), \rho, \kappa \rangle_X \\ \quad \longrightarrow \langle \mathrm{callback}(v, n, \bar{f}), a, \mathrm{Barrier}(\omega', \rho) :: \kappa \rangle_{X'}, \\ B, \sigma \vdash \bar{f} \Downarrow (\bar{v}, \sigma', B')\end{array}}{\langle \mathrm{callback}(v, n, \bar{f}), a, \kappa \rangle_X \longrightarrow \langle \mathrm{send}(\mathrm{recv}(v), n, \bar{v}, \mathrm{exact}), a, \kappa \rangle_{X[\sigma:=\sigma', B:=B']}}.$$

Here X' includes the adapter step and fresh activation. Installing the barrier before import gives conversion failure the same failure path as the callback body. A previously handled failure does not prevent a new callback.

$$\frac{B, \sigma \vdash v \Uparrow (f, B')}{\langle \mathrm{val}(v), \rho', \mathrm{Barrier}(\omega, \rho) :: \kappa \rangle_X \longrightarrow \langle \mathrm{foreign}(\omega[\mathrm{result}(f)]), \rho, \kappa \rangle_{X[B:=B']}}$$

$$\langle \mathrm{abort}(\epsilon), \rho', \mathrm{Barrier}(\omega, \rho) :: \kappa \rangle_X \longrightarrow \langle \mathrm{foreign}(\omega[\mathrm{failure}(\epsilon)]), \rho, \kappa \rangle_X.$$

Export failure enters `error` with the barrier still present. The failed callback reports its diagnostic to the foreign continuation. Foreign code can handle it, make another callback and complete normally, or propagate a failure. The outer native caller observes that foreign completion after conversion; no shared fatal marker overrides it. Native mutations and callback-home retirement survive reentry, while suspended caller homes remain live until their own completion or failure. This does not resume the aborted callback or permit NLR crossing.

A.7.1. Conversion failures

Import failure has form $!(\epsilon, \sigma', B')$; export failure has form $!(\epsilon, B')$, with unchanged native store. At each site the reached state is retained:

$$\frac{B, \sigma \vdash l \Downarrow !(\epsilon, \sigma', B')}{\langle l \rangle_X \longrightarrow \langle \text{error}(\epsilon) \rangle_{X[\sigma := \sigma', B := B']}},$$

$$\frac{B, \sigma \vdash \bar{v} \Uparrow !(\epsilon, B')}{\langle \text{call}(\text{foreignOp}(\phi, op), \bar{v}) \rangle_X \longrightarrow \langle \text{error}(\epsilon) \rangle_{X[B := B']}},$$

$$\frac{B, \sigma \vdash \bar{f} \Downarrow !(\epsilon, \sigma', B')}{\langle \text{callback}(v, n, \bar{f}) \rangle_X \longrightarrow \langle \text{error}(\epsilon) \rangle_{X[\sigma := \sigma', B := B']}},$$

$$\frac{B \vdash \omega \rightsquigarrow (\text{done}(f), B_1) \quad B_1, \sigma \vdash f \Downarrow !(\epsilon, \sigma', B_2)}{\langle \text{foreign}(\omega) \rangle_X \longrightarrow \langle \text{error}(\epsilon) \rangle_{X[\sigma := \sigma', B := B_2]}}.$$

$$\frac{B, \sigma \vdash v \Uparrow !(\epsilon, B')}{\langle \text{val}(v), \rho', \text{Barrier}(\omega, \rho) :: \kappa \rangle_X \longrightarrow \langle \text{error}(\epsilon), \rho', \text{Barrier}(\omega, \rho) :: \kappa \rangle_{X[B := B']}}.$$

A.8. Entry, observations and remaining parameters

Normal module definition starts at an empty top activation, with structural superclass `Object` and no mixins; forbidden top-level contributor clauses fail before evaluation. Instantiation supplies existing argument values to exact `new/k`, without capturing the service's scope. Tools may explicitly supply a different starting activation. See module entry. Independent entry requires successful runtime admission from `idle` under the control agreement above. Construction of an entry configuration does not itself grant execution authority. For a valid module descriptor d, d's arity must equal $|\bar{v}|$. Allocate fresh a with $\sigma' = \sigma[a \mapsto \text{Act}(\emptyset, \bot, \bot, \bot, \bot)]$:

$$\begin{gathered}\text{module}(d, \bar{v}; X) \rightsquigarrow \langle \text{class}(d), a, \text{Module}(\bar{v}) :: \text{halt} \rangle_{X[\sigma := \sigma']}, \\ \langle \text{val}(C), a, \text{Module}(\bar{v}) :: \text{halt} \rangle_X \longrightarrow \langle \text{send}(\text{recv}(C), \text{new}, \bar{v}, \text{exact}), a, \text{halt} \rangle_X, \\ \text{tool}(e, \rho; X) \rightsquigarrow \langle e, \rho, \text{halt} \rangle_X.\end{gathered}$$

These are admitted independent entry points, not source operations. The entry arrow acquires the existing idle token for the new continuation. The same admission premise applies to independent exact Expat sends; callbacks instead use the foreign invocation's admission path. A rejected module descriptor has no definition transition; it reports its validation error. Delivery of a `Final` observation returns control to `idle`; retaining the final configuration does not retain execution authority.

$$\text{Final}(K) = \begin{cases} \text{success}(v) & K = \langle \text{val}(v), \rho, \text{halt} \rangle_X, \\ \text{failure}(\epsilon) & K = \langle \text{abort}(\epsilon), \rho, \text{halt} \rangle_X. \end{cases}$$

Parameter / obligation	Boundary of this formalization
S — scalar profile	Numeric domains, overflow and native string/number methods are underdeveloped. The prototype supplies signed-64-bit/string transport, not the full intended scalar protocol.
B — adapter relation	Each adapter must specify effect-free selection, conversion, foreign actions and failure kinds satisfying the boundary contract.
Execution authority	The global sequential admission condition must be refined by every ordinary entry, callback and completion path. Host queues, fairness, parallel execution and intervention relations require additional contracts; token uniqueness does not prohibit concurrent implementation maintenance.
Static validity	Complete surface-to-core elaboration and declaration-validity judgments remain to be mechanized; invalid/unsupported forms must fail, not acquire substitute semantics.
Proofs	Preservation/progress, determinism modulo fresh identities and fixed foreign choices, descriptor erasure, capability confinement, and Java simulation remain obligations.

The intended implementation relation $R(J, K)$ reads the current Java store, decodes addressed frames and suspended callbacks, and preserves origins, homes, caches and active failure controls. It also reconstructs global control ownership from the Java coordinator and suspended calls; a permit object or copied callback context is not individually a control token. In the superclass field of root Object, it decodes Java's absent reference as semantic `Top`, with no corresponding store identity. Other absent links retain their field-specific meanings. The terminal representation preserves the empty lookup, layout and construction cases. The relation is restricted to the supported scalar and sequential execution profile, with no unspecified interventions:

$$R(J, K) \wedge J \longrightarrow_J J' \implies \exists K'.\, K \longrightarrow^* K' \wedge R(J', K').$$

Administrative stuttering must preserve result/error and boundary traces and must not hide divergence. The concrete choices and limitations are recorded once in implementation refinement.